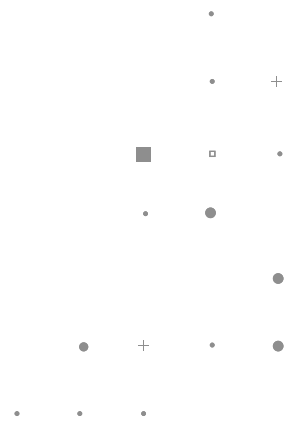
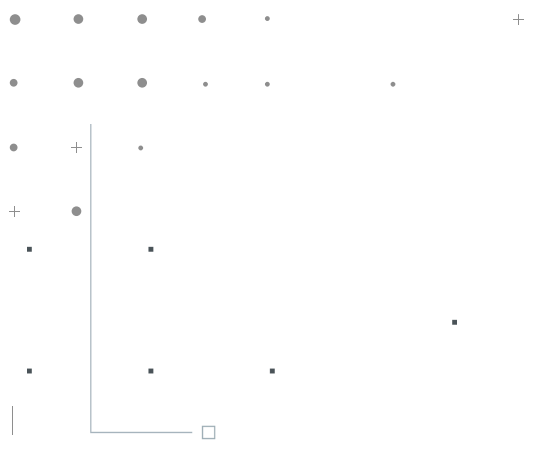


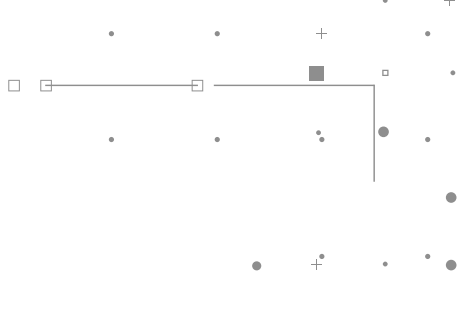


FIAP





Computational Thinking with Python



AGENDA

1

Apresentação do professor

2

Objetivo da Disciplina e Bibliografia

3

Plano Semanal de Aulas

4

Metodologia

5

Avaliação

EMENTA

Os **Algoritmos** estão inseridos em quase todas as áreas da nossa vida. Podemos encontrá-los em computadores, notebooks, tablets, smartphones, TVs inteligentes, robôs e muitos outros equipamentos.

Entender como **construir soluções eficientes** para diversos problemas computacionais em uma das principais linguagens da atualidade: **Python**.

Dominar a base necessária para programar em outras linguagens como: Java, C#, C e muitas outras.

Criar programas e aplicativos que interajam com os usuários e os auxiliem no seu dia a dia.

Desenvolver conhecimento com o **aprendizado orientado a dados**.

OBJETIVOS

Apresentar e aplicar os conceitos de **Lógica de Programação** utilizando a **Linguagem de Programação Python**

HABILIDADES

Durante a disciplina espera-se que o aluno desenvolva as seguintes habilidades:

- Elaborar algoritmos.
- Codificar programas utilizando técnica de lógica de programação com a linguagem Python.

COMPETÊNCIAS

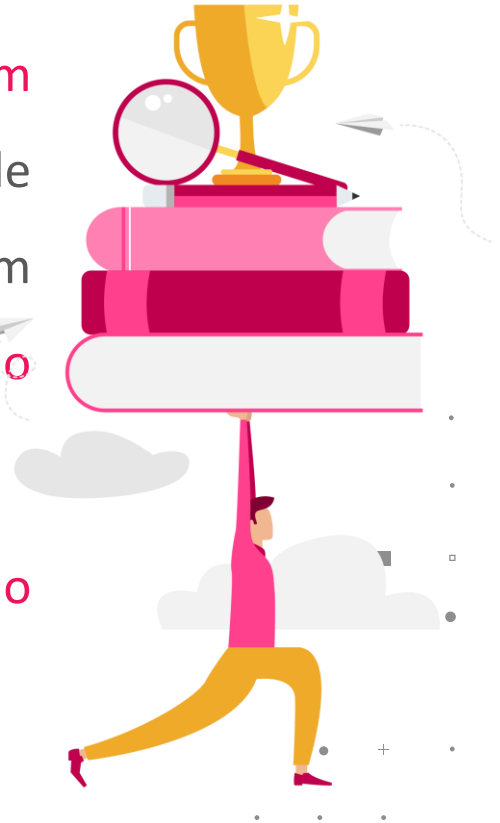
Ao término da disciplina os alunos deverão ser capazes de:

- 1 - Implementar algoritmos de programação.
- 2 - Utilizar linguagem de programação em ambiente de desenvolvimento.
- 3 - Entender os princípios de Codificação

Metodologia

A metodologia usada utiliza aulas teóricas e práticas em laboratório mescladas com uma grande quantidade de exercícios em cada unidade do conteúdo programático, além de jogos, projetos em grupo e trabalhos de avaliação continuada.

Além disso, a disciplina participa de um projeto integrado com outras disciplinas, a ser desenvolvido ao longo do ano.



CONTEÚDO

1º SEMESTRE

~~1. Conceito de algoritmos~~

~~2. Fluxograma | Pseudocódigo | Linguagem de programação~~

~~3. Tipos de dados~~

~~4. Inteiros | Reais | Texto | Lógico~~

~~5. Casting: int() | float() | str() | bool()~~

~~6. Variáveis e operadores Aritméticos~~

~~7. Comandos de entrada, saída e processamento de dados~~

~~8. print() | input()~~

~~9. Estruturas de Decisão e Operadores~~

CONTEÚDO

1º SEMESTRE

~~10. Operadores Relacionais e Lógicos~~

~~11. Decisão simples (if) | decisão composta (if else) | decisão encadeada (if elif else)~~

~~12. Estrutura de Seleção~~

~~13. Escolha: match case~~

~~14. Estruturas de repetição~~

~~15. Condicional: while~~

~~16. Contadora: for~~

~~17. Tratamento de erros~~

~~18. Comando: try except else finally~~

2° SEMESTRE

~~1. Subalgoritmos~~

~~2. Funções | Procedimentos~~

3. Parâmetros: Reais e formais | default | args

4. Estrutura de dados unidimensional

~~5. Estruturas de dados do tipo: vetor (dados homogêneos) | lista (list) | tupla (tuple)~~

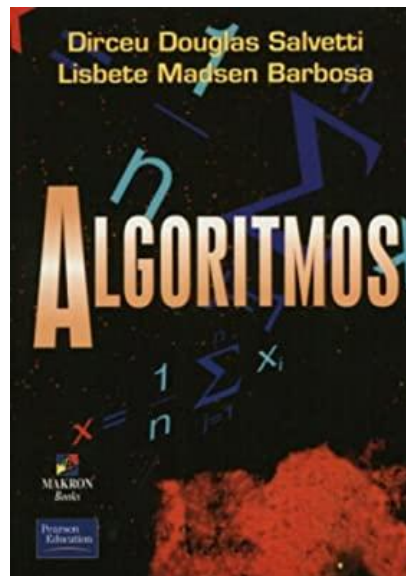
6. Estrutura de dados homogênea bidimensional tipo matriz (dados homogêneos)

7. Busca e ordenação em vetores / listas

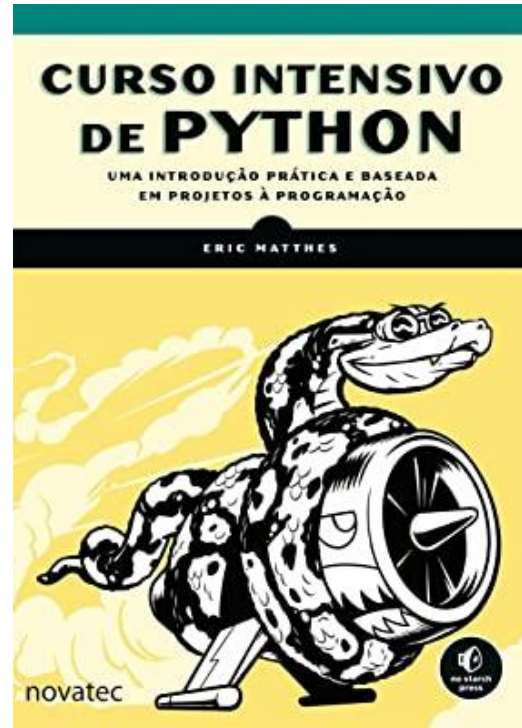
2° SEMESTRE

- 8. Conjuntos (set)
- 9. Dicionários
- 10. Tabelas de memória (listas x dicionários)
- 11. Arquivos
- 12. Manipulação de arquivos texto
- 13. Manipulação de arquivos CSV

BIBLIOGRAFIA BÁSICA



BIBLIOGRAFIA COMPLEMENTAR



HORÁRIO DE AULA

SEGUNDA FEIRA TEM AULA DO EDER (PRESENCIAL)

8h00 ÀS 09h40 e 10h00 ÀS 11h40 – manhã – 1ESPI

19h20 ÀS 21h00 e 21h15 ÀS 22h55 – noite - 1ESPV - 1ESPW - 1ESPY



AVALIAÇÃO

As notas semestrais na FIAP são compostas:

- A média final para aprovação deve ser maior ou igual a 6,0 pontos. Essa nota será resultante do novo sistema de avaliação da FIAP, assim composto:
- Média 1º semestre: Challenge Sprint (2 atividades) e Checkpoints (2 a 3), representando 40% da nota semestral; uma Global Solution, representando 60% da nota do semestre. Essa nota representará 40% da média anual.
- Média 2º semestre: Challenge Sprint (2 atividades) e Checkpoints (2 a 3), representando 40% da nota semestral; uma Global Solution, representando 60% da nota do semestre. Essa nota representará 60% da média anual.

CÁLCULO DE MÉDIA ANUAL

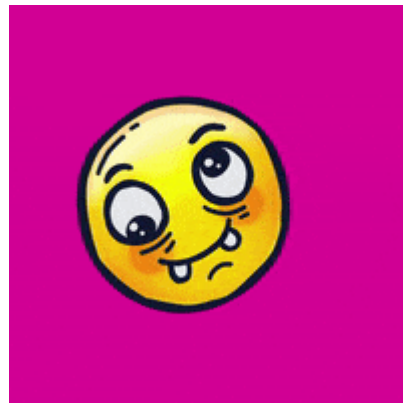
A média anual é ponderada, ou seja, os semestres possuem pesos diferentes:

$$MA = (MS1 \times 0.4 + MS2 \times 0.6)$$

CRITÉRIOS DE APROVAÇÃO

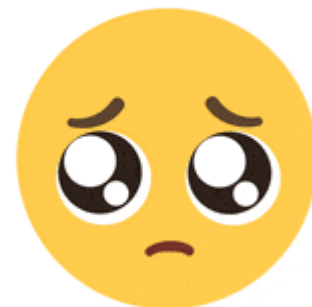


Média Anual	Situação
0 a 3.9	Reprovado
4.0 a 5.9	Exame
6.0 a 10	Aprovado



CASO O ALUNO FIQUE DE EXAME:

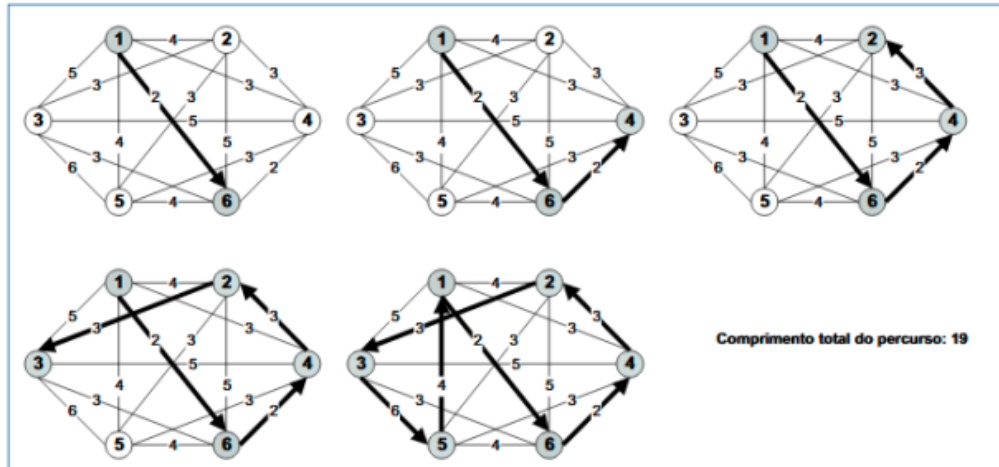
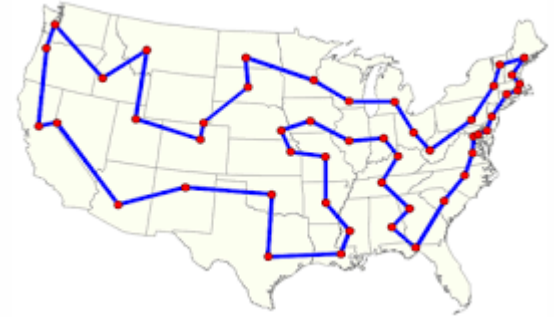
Nota para aprovação = $(12 - \text{Média Anual})$



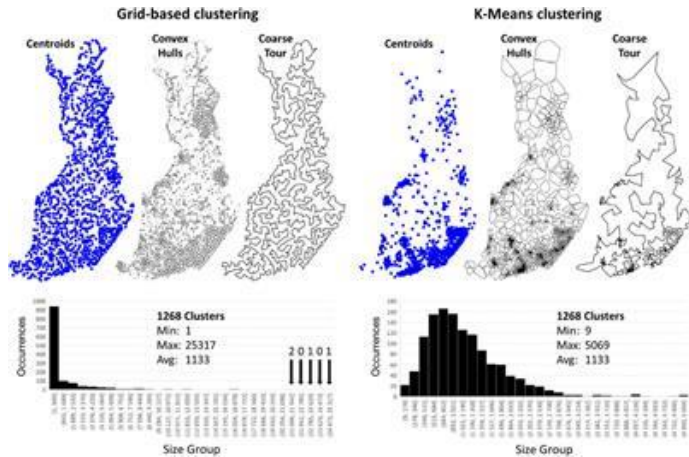
QUE OS JOGOS COMECEM!



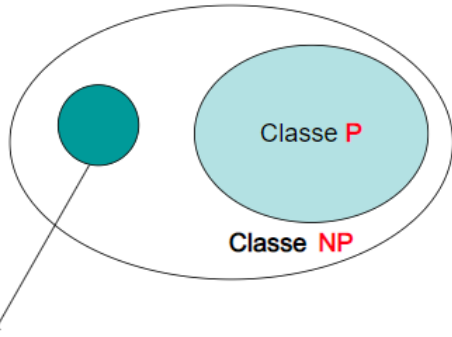
PYTHON – Problema do Caxeiro Viajante – Travelling Salesman Problem - TSP



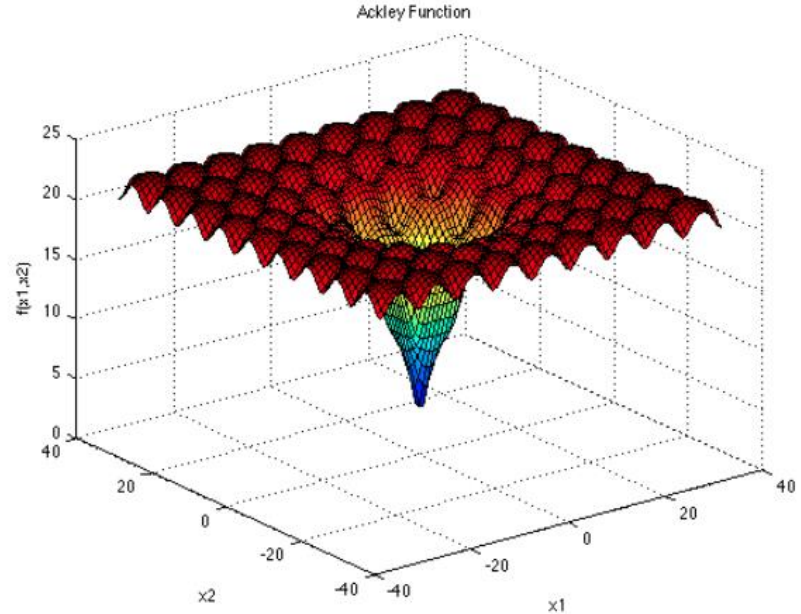
PYTHON – Problema do Caxeiro Viajante – Travelling Salesman Problem - TSP



PYTHON – Problema do Caxeiro Viajante – Travelling Salesman Problem - TSP

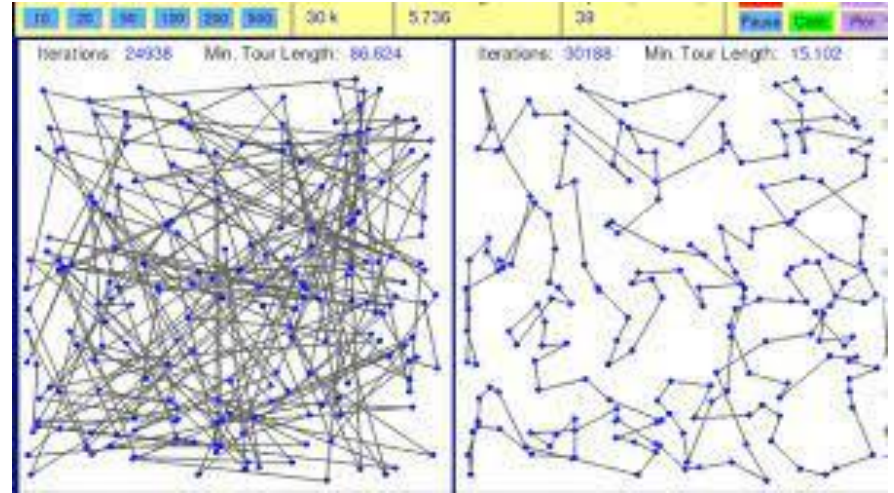


Classe dos problemas NP-completos = os mais “difíceis” da classe NP
Resolvendo um deles com um algoritmo polinomial $\rightarrow P=NP$



$$f(\mathbf{x}) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(cx_i) \right) + a + \exp(1)$$

PYTHON – Problema do Caixeiro Viajante – Travelling Salesman Problem - TSP



E Vamos para o PYTHON!!!

Um pouco de história

A linguagem Python foi desenvolvida em 1991 pelo holandês Guido van Rossum.

O nome Python foi devido ao costume do departamento onde foi criado de dar o nome em homenagem a algum programa de televisão.

Sendo assim, o criador resolveu remeter à um de seus programas favoritos: Monty Python's Flying Circus.

Apenas depois a analogia com a cobra píton foi estabelecida e é usada no símbolo da linguagem, que são duas cobras entrelaçadas.

Atualmente a propriedade intelectual da linguagem é mantida pela Python Software Foundation. <https://www.python.org/psf/>

Trata-se de uma corporação sem fins lucrativos com objetivos principais de promover, proteger e evoluir a linguagem



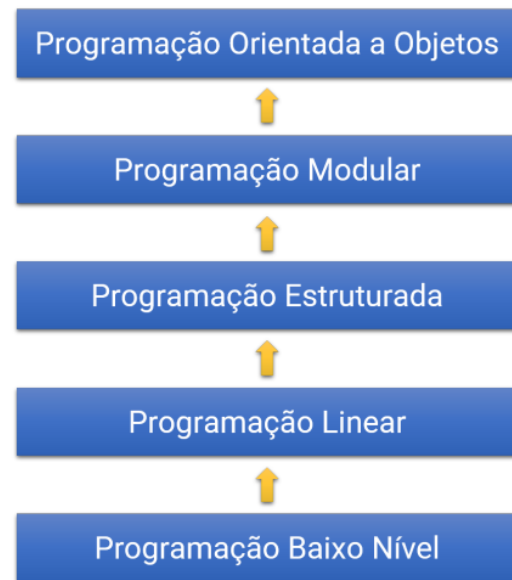
Um pouco de história

A linguagem Python é classificada como linguagem de propósito geral, que possui uma sintaxe elegante e tipagem dinâmica.

Pode ser facilmente transformada para uma aplicação por meio da importação de bibliotecas.

Também é uma linguagem orientada a objetos, ou seja, de alto nível. A hierarquia dos níveis de linguagens de programação pode ser observada na Figura ao lado

As linguagens de alto nível podem ser classificadas em dois tipos: as interpretadas e as compiladas.



O primeiro tipo necessita de um interpretador que irá transformar o código fonte em uma saída.

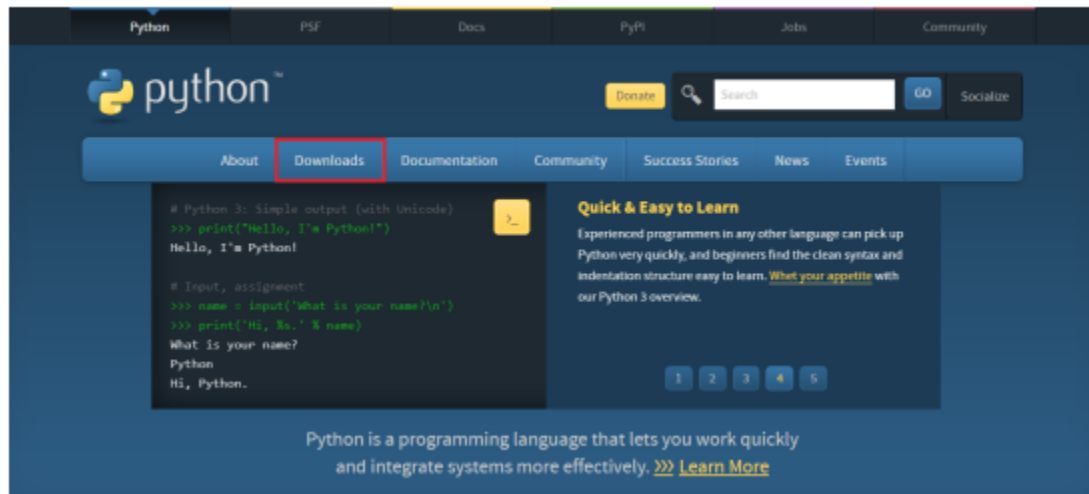
Já o segundo, necessita um compilador para transformar o código fonte em arquivos de objeto, binários ou executáveis e de um executor para ler tais arquivos. Geralmente, os códigos compilados possuem melhor desempenho, em termos de tempo e processamento do que os interpretados

Instalação

- Instalação

- + Como mencionado anteriormente, Linux e MacOS já possuem o Python em seus terminais. Assim, é necessário instalar apenas no Windows. Nos outros sistemas operacionais, talvez seja necessário apenas atualizar.

- + Para a instalação deve-se abrir o site oficial do Python: <https://www.python.org/> e ir em Downloads



Instalação

Selecione a versão e o instalador adequado para o sistema

Looking for Python 2.7? See below for specific releases

Looking for a specific release?
Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 3.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes
Python 3.8.0	Dec. 14, 2018	Download	Release Notes

[View older releases](#)

Licenses

All Python releases are [Open Source](#). Historically, most, but not all, Python releases have also been GPL-compatible. The [Licenses page](#) details GPL-compatibility and Terms and Conditions.

Sources

For most Unix systems, you must download and compile the source code. The same source code archive can also be used to build the Windows and Mac versions, and is the starting point for ports to all

Alternative Implementations

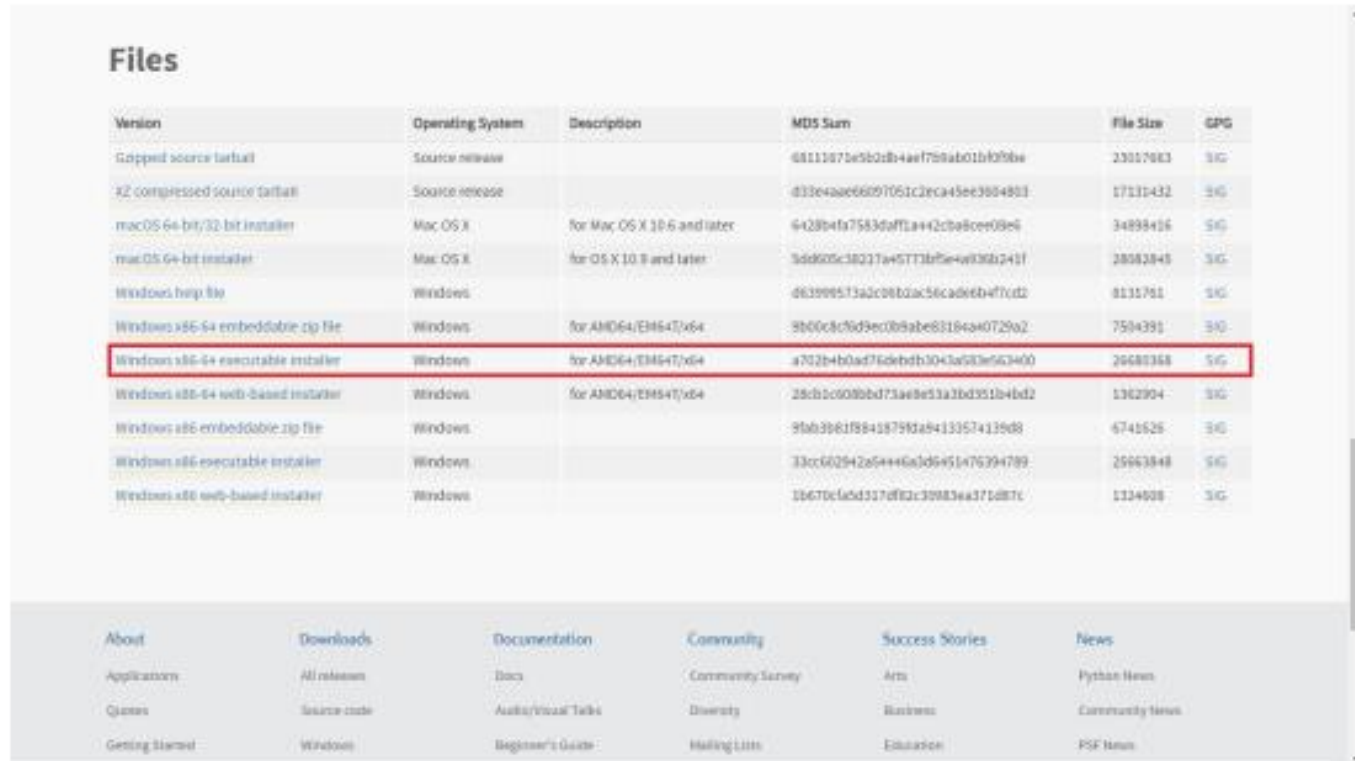
This site hosts the "traditional" implementation of Python (nicknamed CPython). A number of alternative implementations are available as well

History

Python was created in the early 1990s by Guido van Rossum at Stichting Mathematisch Centrum in the Netherlands as a successor of a language called ABC. Guido remains Python's principal author, although

Instalação

- Selecionar a versão e o instalador adequado para o sistema



Files

Version	Operating System	Description	MD5 Sum	File Size	GPB
Gimp source tarball	Source release		68111075e5b2db4ae7b9ab01b939ba	23017683	5G
x2 compressed source tarball	Source release		d33e4aae66097051c2eca45ee3804803	17331432	5G
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583da71a442c9a8ce08e6	3489436	5G
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	3dd925c38217a45773bfe4e038b243f	38883848	5G
Windows help file	Windows		d63988573a2c98b2ac56cad6b477cd2	8131761	5G
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9b00c8c7d9ec0b9abe83186a40729a2	7584391	5G
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0ad76d6db3043a93e963400	2668368	5G
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28db5c6080bd73aeb53a2bd351b4bd2	1362904	5G
Windows x86 embeddable zip file	Windows		9fab3861f8841879fda94133574139d8	6746526	5G
Windows x86 executable installer	Windows		33cc602942a5446ad9451476394789	26683848	5G
Windows x86 web-based installer	Windows		1b670c5ed317df82c3083ea371d87c	1324808	5G

[About](#)[Downloads](#)[Documentation](#)[Community](#)[Success Stories](#)[News](#)

[Applications](#)[All releases](#)[Docs](#)[Community Survey](#)[Arts](#)[Python News](#)

[Queries](#)[Source code](#)[Audio/Visual Talks](#)[Diversity](#)[Business](#)[Community News](#)

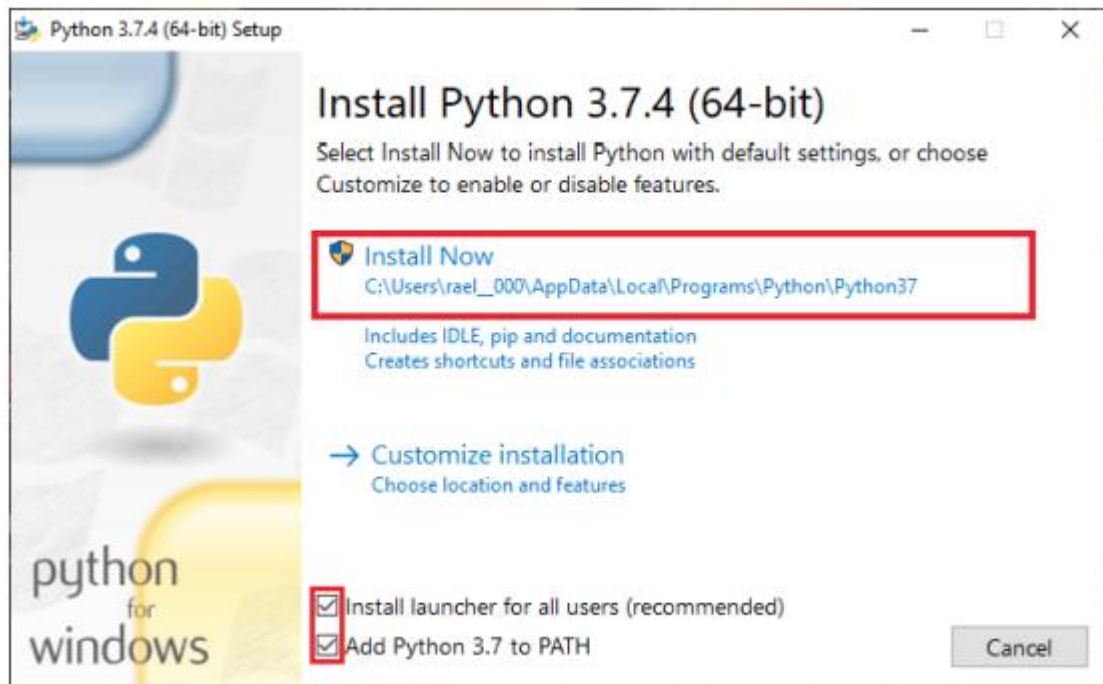
[Getting Started](#)[Windows](#)[Beginner's Guide](#)[Hacking Linux](#)[Education](#)[PSF News](#)

- • • • • +
- • • • •

• Por fim, basta abrir o instalador e clicar em Install Now.

É recomendado marcar as duas caixas de seleção para que também seja possível acessar o Python pelos terminais do Windows.

Instalação



IDE's

- Para o aprendizado e desenvolvimento de softwares em linguagens de programação existem programas com funcionalidades e ferramentas integradas, que auxiliam os programadores. Esses programas são Ambientes de Desenvolvimento Integrado, do inglês Integrated Development Enviroment (IDE).

Para o desenvolvimento em Python, a IDE PyCharm, do desenvolvedor JetBrains, é bem conceituada e, por isso, recomendada. O download pode ser feito no site oficial da JetBrains, <https://www.jetbrains.com/pycharm/>.

Há a versão paga Professional, com mais utilidades, e a versão gratuita para a comunidade estudantil e científica: Community. A IDE ideal é aquela que atende aos gostos e necessidades do usuário.

Outros ambientes mais amplos como Atom ou Sublime Text também são muito populares ou até mesmo um editor de texto para uma tela mais limpa como o Notepad++.

Instalador de Pacotes - pip

O pip é o gerenciador de pacotes padrão do Python. Ele permite a instalação e o gerenciamento de pacotes adicionais que não fazem parte da biblioteca padrão.

Basicamente, um pacote é um diretório que contém módulos, estes, por sua vez, são objetos que servem como uma unidade organizacional de código Python.

O pip é um programa que é utilizado via linha comando. Para usá-lo, a seguinte estrutura de comando é utilizada:

Instalador de Pacotes - pip

Para instalar um pacote é possível usar um dos comandos a seguir:

```
$ pip <argumentos>
```

Para instalar um pacote é possível

```
$ pip install pacote                # Instala a ultima versao
$ pip install pacote==1.0.4        # Instala uma versao
    especifica
$ pip install 'pacote>=1.0.4'      # Instala a partir de
    uma versao minima
```

E para desinstalar um pacote:

```
$ pip uninstall pacote
```


Shell X Script

Para instalar um pacote é possível usar um dos comandos a seguir:

As linguagens de programação, descritas acima como tradutoras entre usuários e máquinas funcionam a base de comandos.

Esses comandos são sequências de palavras reservadas e estruturas de códigos que seguem regras e traduzem o que o usuário quer para instruções de máquina.

Por exemplo, para fazer com que a máquina imprima "Hello World!" na tela, devemos passar para a linguagem algo como "ImprimaNaTela ("Hello World!")".

As regras e sintaxe irão variar de linguagem para linguagem.

Shell X Script

Quando as ações do programa se tornam mais complexas, o código responsável por elas também se torna mais complexo.

Uma característica importante das linguagens de programação é que os códigos, ou seja, as instruções que são passadas, são executados de forma sequencial.

Ou seja, a linha 1 é executada primeiro, em seguida a 2 e assim por diante, seguindo um fluxo linear. A sequencialidade implica que ações como:

```
1 a = 2;  
2 b = a + 1;
```

Irão ser executadas propriamente, enquanto que outras como:

```
1 b = a + 1;  
2 a = 2;
```

São impossíveis de ser executadas, porque a primeira linha depende da execução da segunda.

Shell

O termo Shell, sozinho, é popularmente usado para tradutores entre o sistema operacional e o usuário, conceito muito similar a própria definição de linguagens de programação. No caso do Shell, é comum que eles sejam linhas de comando, isto é, são prompts que executam os comandos usados pelo programador imediatamente, diferente dos códigos, que são organizados pelo programadores e depois executados. Exemplos de Shell são o Bash do Linux e a Command Window do Matlab.

Pode-se acessar o Shell do Python, chamado de Python Interativo ou de Python Shell, usando o comando `python` no terminal. Nota-se que, de fato, os comandos são executados imediatamente quando se pressiona a tecla enter.

No caso do Python Shell, não é possível estruturar códigos e procedimentos mais complexos. Ele é melhor utilizado para testes de procedimentos simples e instruções rápidas.

Script

Como visto, o Python Interativo não é capaz de armazenar sequências de comandos ou rotinas para serem executados posteriormente.

Para isso, é preciso usar códigos, conhecidos como roteiros ou scripts, no caso do Python, chamado de Python Script. Esses arquivos contém instruções para serem executadas.

É importante afirmar novamente que os scripts são executados de formas sequenciais. Eles podem ser criados até mesmo pelo bloco de notas ou qualquer editor de texto e editados livremente. Para isso, basta salvá-los com a extensão .py. meu_primeiro_programa.py

```
1 print('Hello World')
```

Erros de Depuração

Há vários tipos de programadores, com hábitos, lógicas, aptidões e que trabalham com aplicações diferentes. Mas de uma maneira geral, um programador comete vários erros, isto é, dificilmente um programa complexo irá funcionar de primeira.

O trabalho de um programador também envolve a habilidade de reler o código, identificar os erros e consertá-los, repetitivamente. Os erros costumam ser de duas naturezas diferentes: erros de sintaxe e erros de lógica. Os erros de sintaxe dizem respeito a digitação errada, ou indentação incorreta ou marcações incorretas. Esses erros causam falhas na execução dos códigos e são, geralmente, indicados.

É difícil preveni-los, quase impossível em programas muito longos, desta forma, o programador deve conferir as linhas de código em que foram apontados os erros, e como a linguagem é Python, deve-se caprichar na indentação.

Erros de Depuração

Já os erros de lógica acontecem quando o programa roda, mas não executa o que o programador gostaria que fizesse. Neste caso, deve-se revisar a lógica proposta e a lógica implementada. Em alguns casos, o erro pode ser de sintaxe, por exemplo, um procedimento que deveria estar subordinado a um loop não estar indentado, assim, ele irá ser executado apenas uma vez e não repetidas.

Ademais, o programador deve desenvolver boas práticas de programação. A própria linguagem Python, pelos seus princípios, já favorece um código mais organizado e visualmente mais limpo, mas ainda sim, boas práticas devem ser desenvolvidas, como comentários no código e modularização.

DICA : Comentários são indicados pelo caractere cardinal "#", tudo o que estiver após ele em uma linha de código será ignorado pelo interpretador.

Estrutura de Dados

Em algoritmos de computação é necessário armazenar informações diversas. Essas informações são traduzidas para dados de diferentes tipos e armazenados na memória.

Cada bloco de memória que armazena um dado recebe uma etiqueta, um nome, e essas estruturas são chamadas de variáveis do programa.

Isto é, nomes de controle que existirão no código para armazenar dados. A este ramo da programação é dado o nome de estrutura de dados.

Variáveis

Para o armazenamento, as informações devem ser comportadas em longas cadeias de dígitos binários, chamados de bits. Isto é, longas sequências de zeros e uns.

Esses dados podem ser interpretados de diferentes tipos, dependendo do seu uso e das operações a serem realizadas com eles. Seu armazenamento se dá em variáveis, que são espaços reservados na memória para fácil acesso do usuário.

Todas as variáveis de Python são objetos, mas esse conceito será abordado futuramente, por hora, as classes serão tratadas como se fossem tipos.

CURIOSIDADE: O termo *bit* é uma sigla para se referir a dígito binário, do inglês, *Binary digIT*.

Variáveis - Nomeclatura

A nomenclatura de uma variável, em Python, pode conter letras, números ou underscore "_", iniciando necessariamente com letras ou underscore.

Como a maioria, a linguagem é Case Sensitive, isto é, diferencia caracteres minúsculos de maiúsculos. Ainda assim, algumas palavras não podem ser usadas para nomear variáveis, pois são palavras reservadas, ou seja, palavras que indicam outros procedimentos para o interpretador. Algumas das palavras reservadas estão na tabela abaixo:

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

Variáveis - Tipos

Inteiro

O primeiro tipo é o inteiro. Nesse tipo os dados são interpretados a fim de determinar o sinal e o valor, assumindo valores negativos, positivos e o zero, exatamente como no conjunto dos números inteiros.

Em algumas linguagens há o tipo de inteiro unsigned, isto é, sem sinal, no qual só é possível armazenar valores não negativos. Com isso, o maior valor possível de se representar é o dobro+1 do tipo inteiro padrão.

Variáveis - Tipos

Ponto Flutuante

O tipo ponto flutuante é responsável por representar o conjunto dos números reais.

Pode-se também representar inteiros, mas não tantos quanto o tipo inteiro, pois a capacidade de armazenamento deste tipo está direcionada para poder suportar valores decimais.

Neste tipo, é possível representar valores como 2.5, -0.3333, etc

Lógico

Já o tipo lógico armazena os valores booleanos, ou seja, verdadeiro ou falso, 0 ou 1

Variáveis - Tipos

Ponto Flutuante

O tipo ponto flutuante é responsável por representar o conjunto dos números reais.

Pode-se também representar inteiros, mas não tantos quanto o tipo inteiro, pois a capacidade de armazenamento deste tipo está direcionada para poder suportar valores decimais.

Neste tipo, é possível representar valores como 2.5, -0.3333, etc

Lógico

Já o tipo lógico armazena os valores booleanos, ou seja, verdadeiro ou falso, 0 ou 1

Variáveis - Tipos

Caractere

O tipo caractere é responsável por armazenar valores de letras e outros caracteres. Geralmente, os caracteres são codificados em valores seguindo um padrão universal, conhecido como Tabela ASCII

Variáveis - Tipos

Tipos do Python

Analogamente aos tipos primitivos, o Python possui as classes `int` para inteiros, `float` para pontos flutuantes, `bool` para booleanos e `str` para strings, isto é, sequências de caracteres.

Não há um tipo simples para trabalhar com caracteres, como o tipo `char` de outras linguagens. No Python os caracteres são trabalhados por arranjos, em uma estrutura composta chamada de `string`.

Adicionalmente, há a classe `complex` para representar números complexos. Python é uma linguagem de tipagem dinâmica, isto é, dependendo do valor atribuído a uma variável, seu tipo é definido dinamicamente, podendo ser alterado durante o código. O comando `type(var)` retorna a classe de `var`.

Variáveis - Tipos

Tipos do Python

```
>>> Frase = 'Esta e uma frase'
>>> type(Frase)
<class 'str'>
>>> a = 4
>>> type(a)
<class 'int'>
>>> x = 2.5
>>> type(x)
<class 'float'>
>>> Q = True
>>> type(Q)
<class 'bool'>
>>> s = 1 + 2j
>>> type(s)
<class 'complex'>
```

Variáveis - Tipos

Tipos do Python

DICA: A presença do caractere ponto já indica que o usuário deseja que o valor seja armazenado em float, mesmo que o valor seja inteiro. Exemplo: se digitado $x = 4.$ x será do tipo float.

DICA: O valor de uma variável pode ser impresso digitando apenas o nome dela.
Exemplo:
 $x = 4$
 x
 4

Variáveis - Tipos

Tipos do Python

Os dados podem ser convertidos para outros tipos usando-se o nome do tipo desejado e o valor como argumento.

```
>>> c = str(19)
>>> c
'19'
>>> x = int(c)
>>> x
19
```

Variáveis - Tipos

Tipos do Python

Mesmo os tipos sendo implicitamente definidos, o programador pode definí-los usando o nome do tipo e o valor como argumento, ou seja, fazendo uma conversão:

```
>>> x = float(3)
>>> a = str(x)
>>> x = x + 1
>>> a
'3.0'
>>> print('Valor = ' + a)
Valor = 3.0
>>> print('Valor = ', x)
Valor = 4.0
```

Variáveis - Tipos

Tipos do Python

DICA: Em Python, as aspas duplas também podem ser usadas para definir strings, mas geralmente são usadas aspas simples como boas práticas de programação.

DICA: Note que é possível usar uma para escrever a outra como caractere: ' "' ou "' ".

DICA: Deve-se ter atenção para essa sintaxe, porque apesar de para o Python aspas simples ou duplas terem o mesmo efeito, em outras linguagens elas podem ser fundamentalmente diferentes. Em C, por exemplo, as duplas definem arranjos de caracteres, enquanto que as simples definem caracteres únicos. Em PHP, as duplas definem tipos dinâmicos e as simples tipos estáticos.

(*) PHP é uma linguagem versátil e poderosa para o desenvolvimento de aplicativos web, permitindo a criação de sites dinâmicos e interativos que podem se comunicar com bancos de dados e outros serviços web.

Variáveis - Tipos

Tipos do Python

Valores numéricos são geralmente atribuídos na notação de base decimal, mas é possível atribuir valores na notação de binários, de base oito e de hexadecimais com os prefixos 0b, 0o e 0x, respectivamente.

```
>>> d = 31 #base decimal
>>> d
31
>>> b = 0b11111 #base binaria
>>> b
31
>>> o = 0o37 #base octal
>>> o
31
>>> h = 0x1F #base hexadecimal
>>> h
31
```

Variáveis - Tipos

Tipos do Python

DICA : Valores em bases específicas podem ser dados usando-se o comando `int('valor', base = X)`, sendo que o valor precisa estar obrigatoriamente em string.

Exemplo:

```
>>> x = int('11', base=3)
```

```
>>> x
```

Como as estruturas de dados abordadas aqui são classes, elas possuem métodos. A orientação a objetos será abordada futuramente, mas por hora, os métodos serão tratados como funções e procedimentos inerentes a toda variável.

Variáveis - Tipos

Tipos do Python

Por exemplo, uma string que contém uma frase pode ser separada pelas suas palavras pelo método `split()`. O uso dos métodos se dá da seguinte forma: `Variável.método()`.

```
>>> Frase = 'Uma frase'
>>> Frase
'Uma frase'
>>> Frase.split()
['Uma', 'frase']
>>> a, b = Frase.split()
>>> a
'Uma'
>>> b
'frase'
```

Variáveis - Tipos

Tipos do Python

Para visualizar todos os atributos e métodos da classe deve-se utilizar o comando `dir(classe)` ou `dir(variavel)`.

Python também possui outras classes, estruturas compostas. São elas listas, tuplas, e dicionários, além das strings. Vereemo-las em Estrutura de Dados.

Variáveis - Tipos

Exercícios

1 Em um determinado programa são necessárias variáveis de diferentes tipos para armazenar diferentes informações. Em cada caso a seguir informe qual tipo de dado primitivo seria mais adequado para o armazenamento. Informe também qual o tipo do Python correspondente:

- a) Idade de uma pessoa;
- b) Se a pessoa é maior de idade ou não;
- c) Altura de uma pessoa;
- d) Peso de uma pessoa;
- e) Primeira letra do nome de uma pessoa;
- f) Última letra do nome de uma pessoa.

Variáveis - Tipos

Exercícios

4 Em cada item a seguir diga se pode ser um nome de variável. Em caso negativo, informe o motivo:

- a) x;
- b) aux;
- c) 1aluno;
- d) aluno1;
- e) aluno_1;
- f) aluno 1;
- g) _aluno1;
- h) int;
- i) Int;
- j) int_idade;
- k) class

Variáveis - Tipos

Exercícios

2 Considere as variáveis: $a = 1$; $b = 2$; $c = '1'$; $d = '2'$.

Utilizando o operador "+" observe a diferença entre somar $a + b$ e somar $c + d$.

Qual a diferença e por que isso ocorre?

3 Ainda considerando as variáveis da questão anterior, o que poderia se fazer para que $a + b$ tenha o mesmo resultado de $c + d$ ou vice versa?

Comandos e funções

Entendido as estruturas de dados, é necessário compreender os comandos e funções para melhor utilizá-los.

Conteúdos mais complexos, bem como funções definidas pelo usuário e leitura/escrita de arquivos serão vistos posteriormente.

Os comandos são palavras chaves que executam certas ações e as funções são procedimentos que devolvem dados dependendo de seus argumentos.

Os conceitos são parecidos e as vezes confundidos. No Python, ainda, devido a orientação de objetos, alguns procedimentos, chamados de funções, são ainda métodos de objetos, mas essa distinção não será crucial para esta abordagem inicial.

Comandos e funções

As funções são similares a funções matemáticas, isto é, dependendo do argumento, o resultado é diferente.

O uso de funções em algum momento no script ou no Python interativo recebe o nome de chamada.

Quando uma função é chamada, geralmente recebe argumentos, que são passados como parâmetros, dentro de parênteses.

As respostas da função, quando há, são chamados de retornos. Desta forma temos que:

```
retorno1,retorno2 = funcao(argumento1,argumento2)
```

Comandos e funções

Por exemplo, ao usar a função de raiz quadrada `sqrt` com argumento 25, é retornado o valor 5, ou seja, para:

```
>>> x = sqrt(25)
```

A variável `x` receberá o valor 5.

O próprio usuário pode criar funções para serem utilizadas no código. Esse processo é denominado definição de funções.

A definição de funções será abordada mais a frente, porque, para isso, é necessário o conhecimento de controle de fluxo, isto é, estruturas condicionais, de repetição e aninhamento de estruturas.

Comandos e funções

• Importação

A linguagem Python é conhecida pelas diversas áreas de aplicações. Por vezes, tais áreas necessitam de comandos ou funcionalidades específicas que não estão, por padrão, no Python, elas são advindas de bibliotecas e precisam ser importadas, caso o usuário deseje utilizá-las. Para isso é utilizado o comando `import`.

Por exemplo, a função `sqrt` para cálculo de raízes quadradas está na biblioteca `math`. Portanto, deverá ser impresso um erro ao utilizá-la sem importar a biblioteca.

```
>>> sqrt(9)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'sqrt' is not defined
```

Para corrigir o erro, antes é necessário usar o comando *`import math`*.

Comandos e funções

• Importação

Ainda assim, ao utilizar as funcionalidades advindas da biblioteca deve-se usar o nome da biblioteca. No caso:

```
>>> import math
>>> sqrt(9)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'sqrt' is not defined
>>> math.sqrt(9)
3.0
```

Comandos e funções

• Importação

+ • Importação Otimizada

O comando import importa a biblioteca com todas as suas funcionalidades. As vezes, apenas algumas funcionalidades são de interesse, como no exemplo acima, apenas a sqrt. Há a opção de importar apenas o que se deseja para economizar memória e para não precisar especificar a biblioteca da qual foi importado. Para isso, em conjunto com o comando import, utiliza-se o comando from, de forma a ficar como: from import , ... Dessa forma, os métodos são importados diretamente para a pasta, por isso não é necessário explicitar a biblioteca de origem.

```
>>>from math import cos, pi
>>>cos(pi)
-1.0
```


Comandos e funções

• Importação

+ • Importação Otimizada

DICA: Dentro do site oficial do Python, na aba PyPI, é possível acessar o index de pacotes do Python, isto é, todas as bibliotecas e projetos possíveis de se importar. <https://pypi.org/>. É possível criar uma própria biblioteca e disponibilizar na comunidade para livre acesso.

IDE: Ctrl + espaço na IDE PyCharm mostra todas as opções dentro da biblioteca.

IDE: Para gerenciar os pacotes e bibliotecas instalados pela IDE PyCharm, deve-se ir em *Settings* ou *Preferences*, selecionar o projeto e selecionar *Project Interpreter*. Lá, será possível visualizar as bibliotecas instaladas, adicionar ou remover bibliotecas.

Comandos e funções

Importação

Outras Bibliotecas

Algumas bibliotecas são built-in, isto é, já são instaladas por padrão junto com o Python, como a biblioteca math e random. A importação delas é feita naturalmente. No entanto, ao tentar importar bibliotecas que não são built-in, deve-se antes instalá-las na máquina.

Algumas IDEs já fazem o gerenciamento e possuem atalhos para facilitar este processo, podendo importá-las também naturalmente. Mas, se for o caso do usuário não usar IDEs, ou usar o IDLE, por exemplo, deve-se fazer o download do pacote. Para isso, é recomendado usar o pip. Para instalar uma nova biblioteca, navegue até o diretório padrão de instalação do Python, abra o terminal e digite o comando:

```
$ pip install <bib>
```

Comandos e funções

Importação

Outras Bibliotecas

NumPy

NumPy, de Numeric Python, é uma famosa e importante biblioteca de Python, conhecida também como uma biblioteca científica.

É útil pela sua capacidade de processamento vetorial multidimensional homogêneo, estendendo o núcleo básico do Python.

CURIOSIDADE : Por convenção, a biblioteca NumPy é importada com o nome de np.

```
>>> import numpy as np
```

Comandos e funções

Importação

Outras Bibliotecas

NumPy

NumPy, de Numeric Python, é uma famosa e importante biblioteca de Python, conhecida também como uma biblioteca científica.

É útil pela sua capacidade de processamento vetorial multidimensional homogêneo, estendendo o núcleo básico do Python.

Vetores e matrizes podem ser criados pelo comando `array`, usando-se vírgulas e colchetes para identificá-los. Para selecionar elementos de um vetor ou matriz, deve-se usar também colchetes, muito similar ao uso e criação de listas:



Comandos e funções

NumPy

Vetores e matrizes podem ser criados pelo comando array, usando-se vírgulas e colchetes para identificá-los.

Para selecionar elementos de um vetor ou matriz, deve-se usar também colchetes, muito similar ao uso e criação de listas:

```
>>> u = np.array([1, 0, 0])
>>> M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> M
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> u
array([1, 0, 0])
>>> type(M), type(u)
(<class 'numpy.ndarray'>, <class 'numpy.ndarray'>)
>>> u[2]
0
>>> M[1, 2]
6
```

Comandos e funções

NumPy

Em Python, o ndarray (N-dimensional array) é um tipo de objeto que representa um array multidimensional. É uma estrutura de dados muito útil para armazenar e manipular grandes conjuntos de dados numéricos, como imagens, áudio, sinais de sensores, entre outros.

O ndarray é uma parte importante da biblioteca NumPy, que é usada para realizar cálculos numéricos em Python. O ndarray permite que você execute operações matemáticas em grandes conjuntos de dados de uma maneira eficiente e fácil de usar. O ndarray é caracterizado por alguns atributos, como shape (forma), dtype (tipo de dados) e strides (passos), que permitem que você acesse e manipule seus elementos de maneira eficiente.

Comandos e funções

NumPy

Por exemplo, você pode criar um ndarray usando a função `numpy.array` da seguinte maneira:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Este código cria um ndarray de 3 por 3 com os valores de 1 a 9. Você pode acessar os elementos deste ndarray da seguinte maneira:

```
a[0, 0] # retorna 1
a[1, 1] # retorna 5
a[2, 2] # retorna 9
```

Comandos e funções

NumPy

Em resumo, o ndarray é uma estrutura de dados fundamental da biblioteca NumPy que permite manipular eficientemente arrays multidimensionais em Python.

Pode-se especificar o tipo de dado com o parâmetro dtype, e há algumas funções especiais de criação de arranjos:

```
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
>>> np.eye(3, dtype=np.int64)
array([[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]], dtype=int64)
>>> np.zeros((2, 3))
array([[0., 0., 0.],
       [0., 0., 0.]])
>>> np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
>>> np.full((2, 3), 5)
array([[5, 5, 5],
       [5, 5, 5]])
>>> np.random.random((2, 3))
array([[0.36596055, 0.55954819, 0.94322377],
       [0.53376681, 0.00523872, 0.05504161]])
```


Comandos e funções

NumPy

Em Python, `int64` é um tipo de dados de inteiro que pode ser usado para armazenar números inteiros com sinal em um espaço de 64 bits.

O `int64` é um tipo de dados numérico pertencente à biblioteca NumPy, que é amplamente usada para realizar cálculos numéricos em Python.

O `int64` representa um número inteiro que pode variar entre `-9223372036854775808` e `9223372036854775807`, incluindo os extremos.

Esse tipo de dados é útil em situações em que é necessário trabalhar com grandes números inteiros e não há necessidade de usar números de ponto flutuante.

Comandos e funções

Valores podem ser acrescentados em um arranjo pelo comando `append`. É um método usado para adicionar um elemento ao final de uma lista, que é uma das estruturas de dados built-in da linguagem Python.

O método **`append`** (acrescentar) é usado para modificar a lista original, adicionando o novo elemento ao final da lista existente.

```
>>> x = np.random.random((1, 2))
>>> x
array([[0.84375509, 0.04608968]])
>>> x = np.append(x, [1, 2])
>>> x
array([0.84375509, 0.04608968, 1., 2.])
```

Comandos e funções

DICA : Um arranjo vazio pode ser criado pela expressão:

`np.array((0))`

Operações

matemáticas

básicas podem ser

feitas entre arrays

ou entre array, e

número usando-se

os operadores

normalmente

```
>>> x
array([0.84375509, 0.04608968, 1.          , 2.          ])
>>>
>>> x + 10
array([10.84375509, 10.04608968, 11.          , 12.          ])
>>> x - 10
array([-9.15624491, -9.95391032, -9.          , -8.          ])
>>> x * 5
array([ 4.21877543,  0.23044842,  5.          , 10.          ])
>>> x / 2
array([0.42187754, 0.02304484, 0.5          , 1.          ])
>>> y = np.array([1, 2, 3, 4])
>>> x + y
array([1.84375509, 2.04608968, 4.          , 6.          ])
>>> x / y
array([0.84375509, 0.02304484, 0.33333333, 0.5          ])
```

Comandos e funções

Para a multiplicação vetorial que representa o produto escalar entre dois vetores, há o método `dot()`.

O produto escalar entre dois vetores é definido como a soma dos produtos dos elementos correspondentes dos dois vetores.

```
>>> x
array([0.84375509, 0.04608968, 1.          , 2.          ])
>>> y
array([1, 2, 3, 4])
>>> x.dot(y)
11.935934454939305
>>> x.mean()
0.9724611925863914
>>> x.max()
2.0
>>> x.min()
0.04608968459373963
```

Sejam dois vetores v e w com dimensões iguais n , o produto escalar entre eles é representado por $v \cdot w$ e é calculado da seguinte maneira: $v \cdot w = v_1 * w_1 + v_2 * w_2 + \dots + v_n * w_n$. O resultado do produto escalar entre dois vetores é um número que representa a projeção de um vetor sobre o outro. O produto escalar é utilizado em diversas áreas da matemática e da física, como na geometria, álgebra linear, cálculo vetorial, análise de sinais, entre outras. Para extrair o valor máximo, mínimo e a média há os métodos `max()`, `min()` e `mean()`, respectivamente.

Comandos e funções

•Matplotlib

Este é um pacote para plotar figuras de diversos estilos, como gráficos com linhas variadas, histogramas, gráficos de barras, dentre outros.

É, geralmente, usado em conjunto com o NumPy, deixando o Python um pouco mais parecido com o Matlab.

Assim como a literatura convencionou a importação de NumPy como np, também convencionou a importação do módulo pyplot de matplotlib como plt.

Comandos e funções

Matplotlib

Tendo os arranjos de x e y, pode-se plotar o gráfico usando:

```
>>> plt.plot(x, y)
>>> plt.show()
```

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import math
4 import colorama
5
6 """
7 Estilos de linhas:
8 https://matplotlib.org/3.1.1/api/\_as\_gen/matplotlib
9 .pyplot.plot.html#matplotlib.pyplot.plot
10 """
11 def sine(x):
12     return (np.sin(x))
13
14 def plot(x, y):
15     plt.plot(x,y,'g-')
16     plt.grid(True)
17
18     # Limite dos eixos
19     # plt.axis([0, 10*math.pi, -1, 1.01])
20
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
20
21 # Nome dos eixos
22 plt.ylabel('Eixo y')
23 plt.xlabel('Eixo x')
24
25 # Acentos devem ser usados com o código TeX
26 # plt.title(r'$\ddot{o}\acute{e}\grave{e}\hat{O}$
    \breve{i}\bar{A}\tilde{n}\vec{q}$', fontsize=20)
27
28 # Título do gráfico
29 plt.title(r'$Gr\acute{a}fico$', fontsize = 15)
30
31
32 def cardiod():
33     t = np.arange(0, 2*math.pi, 0.01)
34     r = 1 + np.cos(t)
35     x = r * np.cos(t)
36     y = r * np.sin(t)
37     plot(x,y)
```


Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
38 plt.show()
39
40
41 def butterfly():
42     t = np.arange(0, 1000*math.pi, 0.01)
43     x = np.sin(t) * (np.exp(np.cos(t)) - 2*np.cos(4*t) -
44                     np.power(np.sin(t/12), 5))
45     y = np.cos(t) * (np.exp(np.cos(t)) - 2*np.cos(4*t) -
46                     np.power(np.sin(t/12), 5))
47     plot(x,y)
48     plt.show()
49
50 def tesla():
51     x1 = np.arange(0, 2.0*math.pi, 0.01)
52     x2 = np.arange(-2.0*math.pi/3.0, 4.0*math.pi/3.0, 0.01)
53     x3 = np.arange(-4.0*math.pi/3.0, 2.0*math.pi/3.0, 0.01)
54     y1 = np.sin(x1)
55     y2 = np.sin(x2 + 2.0*math.pi/3.0)
56     y3 = np.sin(x3 + 4.0*math.pi/3.0)
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
56 plot(x1,y1)
57 plot(x2,y2)
58 plot(x3,y3)
59 plt.show()
60
61 def lissajous_curve():
62     a = 4.0
63     b = 5.0
64     kx = 3.0
65     ky = 2.0
66     t = np.arange(0, 2.0*math.pi, 0.01)
67     x = a*np.cos(kx*t)
68     y = b*np.sin(ky*t)
69     plot(x,y)
70     plt.show()
71
72 def begin():
73     print('Curves: \n 1 - Cardioid \n 2 - Butterfly Curve \n
74           3 - Tesla \n 4 - Lissajous Curve')
75     curve = int(input('Insert curve number: '))
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
56 plot(x1,y1)
57 plot(x2,y2)
58 plot(x3,y3)
59 plt.show()
60
61 def lissajous_curve():
62     a = 4.0
63     b = 5.0
64     kx = 3.0
65     ky = 2.0
66     t = np.arange(0, 2.0*math.pi, 0.01)
67     x = a*np.cos(kx*t)
68     y = b*np.sin(ky*t)
69     plot(x,y)
70     plt.show()
71
72 def begin():
73     print('Curves: \n 1 - Cardioid \n 2 - Butterfly Curve \n
74           3 - Tesla \n 4 - Lissajous Curve')
75     curve = int(input('Insert curve number: '))
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
75
76 if (curve == 1):
77     cardioid()
78 elif (curve == 2):
79     butterfly()
80 elif (curve == 3):
81     tesla()
82 elif (curve == 4):
83     lissajous_curve()
84 else:
85     print(colorama.Fore.RED + colorama.Back.WHITE +
86           '\nInvalid input, try again')
87     print(colorama.Style.RESET_ALL)
88     begin()
89
90 if __name__ == '__main__':
91     # x = np.arange(0, 10*math.pi, 0.01)
92     # y = sine(x)
93     # y = np.arange(0)
94     # y = math.sin(x)
95     colorama.init()
96     begin()
```

Variáveis e entradas de dados – parte 2

Exercício 3.1 Complete a tabela a seguir, marcando inteiro ou ponto flutuante dependendo do número apresentado.

Número	Tipo numérico
5	☐ inteiro ☐ ponto flutuante
5.0	☐ inteiro ☐ ponto flutuante
4.3	☐ inteiro ☐ ponto flutuante
-2	☐ inteiro ☐ ponto flutuante
100	☐ inteiro ☐ ponto flutuante
1.333	☐ inteiro ☐ ponto flutuante

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Muitas vezes, queremos armazenar um conteúdo simples: verdadeiro ou falso em uma variável. Nesse caso, utilizaremos um tipo de variável chamado tipo lógico ou booleano.

Em Python, escreveremos True para verdadeiro e False para falso. Observe que o T e o F são escritos com letras maiúsculas.

Exemplo de variáveis do tipo lógico

```
resultado = True
```

```
aprovado = False
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Para realizarmos comparações lógicas, utilizaremos operadores relacionais.

A lista de operadores relacionais suportados em Python é apresentada na tabela.

Operador	Operação	Símbolo matemático
<code>==</code>	igualdade	$=$
<code>></code>	maior que	$>$
<code><</code>	menor que	$<$
<code>!=</code>	diferente	$\neq$
<code>>=</code>	maior ou igual	$\geq$
<code><=</code>	menor ou igual	$\leq$

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

O resultado de uma comparação é um valor do tipo lógico, ou seja, True (verdadeiro) ou False (falso).

Utilizaremos o verbo “avaliar” para indicar a resolução de uma expressão.

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Exemplo de uso de operadores relacionais

```
>>> a = 1 # a recebe 1
```

```
>>> b = 5 # b recebe 5
```

```
>>> c = 2 # c recebe 2
```

```
>>> d = 1 # d recebe 1
```

```
>>> a == b # a é igual a b ? False
```

```
>>> b > a # b é maior que a? True
```

```
>>> a < b # a é menor que b? True
```

```
>>> a == d # a é igual a d? True
```

```
>>> b >= a # b é maior ou igual a a? True
```

```
>>> c <= b # c é menor ou igual a b? True
```

```
>>> d != a # d é diferente de a? False
```

```
>>> d != b # d é diferente de b? True
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Esses operadores são utilizados como na matemática.

Especial atenção deve ser dada aos operadores $\geq$ e $\leq$. O resultado desses operadores é realmente maior ou igual e menor ou igual, ou seja, $5 \geq 5$ é verdadeiro, assim como $5 \leq 5$.

Observe que, na listagem anterior, utilizamos o símbolo de cerquilha (#) para escrever comentários na linha de comando.

Todo texto à direita do cerquilha é ignorado pelo interpretador Python, ou seja, você pode escrever o que quiser. Veja novamente a listagem e observe como é mais fácil entender cada linha quando a comentamos

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Comentários não são obrigatórios, ***mas são muito importantes.***

Você pode e deve utilizá-los em seus programas para facilitar o entendimento e oferecer uma anotação para você mesmo.

É comum lermos um programa alguns meses depois de escrito e termos dificuldade de lembrar o que realmente queríamos fazer.

Você também não precisa comentar todas as linhas de seus programas ou escrever o óbvio.

Uma dica é identificar os programas com seu nome, a data em que começou a ser escrito e mesmo a listagem ou capítulo do livro onde você o encontrou.

Variáveis de tipo lógico também podem ser utilizadas para armazenar o resultado de expressões e comparações.

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Exemplo do uso de operadores relacionais com variáveis do tipo lógico

nota = 8

média = 7

aprovado = nota > média print(aprovado)

Se uma expressão contém operações aritméticas, estas devem ser calculadas antes que os operadores relacionais sejam avaliados.

Quando avaliamos uma expressão, substituímos o nome das variáveis por seu conteúdo e só então verificamos o resultado da comparação

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores relacionais

Complete a tabela abaixo, respondendo True ou False. Considere $a = 4$, $b = 10$, $c = 5.0$, $d = 1$ e $f = 5$.

Expressão	Resultado
$a == c$	☐ True ☐ False
$a < b$	☐ True ☐ False
$d > b$	☐ True ☐ False
$c != f$	☐ True ☐ False
$a == b$	☐ True ☐ False
$c < d$	☐ True ☐ False
$b > a$	☐ True ☐ False
$c >= f$	☐ True ☐ False
$f >= c$	☐ True ☐ False
$c <= c$	☐ True ☐ False
$c <= f$	☐ True ☐ False

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Para agrupar operações com lógica booleana, utilizaremos operadores lógicos.

Python suporta três operadores básicos:

not (não),

and (e),

or (ou).

Esses operadores podem ser traduzidos como
não ($\neg$ negação),
e ($\wedge$ conjunção) e
ou ($\vee$ disjunção)

Operador Python	Operação
not	não
and	e
or	ou

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Cada operador obedece a um conjunto simples de regras, expresso pela tabela verdade desse operador.

A tabela verdade demonstra o resultado de uma operação com um ou dois valores lógicos ou operandos.

Quando o operador utiliza apenas um operando, dizemos que é um operador unário.

Ao utilizar dois operandos, é chamado operador binário.

O operador de negação (not) é um operador unário. or (ou) e and (e) são operadores binários, precisando, assim, de dois operandos.

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Operador not

O operador not (não) é o mais simples, pois precisa apenas de um operador.

A operação de negação também é chamada de inversão, pois um valor verdadeiro negado se torna falso e vice-versa.

A tabela verdade do operador not (não) é apresentada na tabela ao lado.

V_1	$\text{not } V_1$
V	F
F	V

```
>>> not True
False
>>> not False
True
```


Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Operador and

O operador and (e) tem sua tabela verdade representada na tabela.

O operador and (e) resulta verdadeiro apenas quando seus dois operadores forem verdadeiros

V_1	V_2	$V_1 \text{ and } V_2$
V	V	V
V	F	F
F	V	F
F	F	F

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Operador and

>>> True and True True

>>> True and False False

>>> False and True False

>>> False and False False

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Operadores lógicos

Operador or

A tabela verdade do operador or (ou) é apresentada na tabela.

A regra fundamental do operador or (ou) é que ele resulta em falso apenas se seus dois operadores também forem falsos.

Se apenas um de seus operadores for verdadeiro, ou se os dois forem, o resultado da operação será verdadeiro.

V_1	V_2	$V_1 \text{ or } V_2$
V	V	V
V	F	V
F	V	V
F	F	F

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Os operadores lógicos podem ser combinados em expressões lógicas mais complexas.

Quando uma expressão tiver mais de um operador lógico, avalia-se o operador not (não) primeiramente, seguido do operador and (e) e, finalmente, or (ou).

Vejamos a seguir a ordem de avaliação da expressão, onde a operação sendo avaliada é sublinhada; e o resultado, mostrado na linha seguinte.

True or False and not True

True or False and False

True or False

True

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Os operadores relacionais também podem ser utilizados em expressões com operadores lógicos.

salário > 1000 and idade > 18

Nesses casos, os operadores relacionais devem ser avaliados primeiramente. Façamos salário = 100 e idade = 20. Teremos:

salário > 1000 and idade > 18

100 > 1000 and 20 > 18

False and True

False

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

A grande vantagem de escrever esse tipo de expressão é representar condições que podem ser avaliadas com valores diferentes. Por exemplo:

Imagine que $\text{salário} > 1000$ and $\text{idade} > 18$ seja uma condição para um empréstimo de compra de um carro novo.

Quando $\text{salário} = 100$ e $\text{idade} = 20$, sabemos que o resultado da expressão é falso, e podemos interpretar que, nesse caso, a pessoa não receberia o empréstimo.

Avaliemos a mesma expressão com $\text{salário} = 2000$ e $\text{idade} = 30$

$\text{salário} > 1000$ and $\text{idade} > 18$

$2000 > 1000$ and $30 > 18$

True and True

True

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Exercício 3.4 Escreva uma expressão para determinar se uma pessoa deve ou não pagar imposto. Considere que pagam imposto pessoas cujo salário é maior que R\$ 1.200,00.

Exercício 3.5 Calcule o resultado da expressão $A > B$ and C or D , utilizando os valores da tabela a seguir.

A	B	C	D	Resultado
1	2	True	False	
10	3	False	False	
5	1	True	True	

Variáveis e entradas de dados – parte 2

Variáveis do tipo Lógico

Exercício 3.6 Escreva uma expressão que será utilizada para decidir se um aluno foi ou não aprovado. Para ser aprovado, todas as médias do aluno devem ser maiores que 7. Considere que o aluno cursa apenas três matérias, e que a nota de cada uma está armazenada nas seguintes variáveis: matéria1, matéria2 e matéria3.

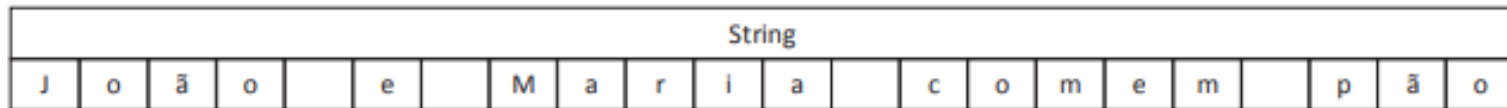
Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Variáveis do tipo string armazenam cadeias de caracteres como nomes e textos em geral. Chamamos cadeia de caracteres uma sequência de símbolos como letras, números, sinais de pontuação etc.

Exemplo: João e Maria comem pão. Nesse caso, João é uma sequência com as letras J, o, ã, o.

Para simplificar o texto, utilizaremos o nome string para mencionar cadeias de caracteres. Podemos imaginar uma string como uma sequência de blocos, onde cada letra, número ou espaço em branco ocupa uma posição, como mostra a figura



Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Para possibilitar a separação entre o texto do seu programa e o conteúdo de uma string, utilizaremos aspas (") para delimitar o início e o fim da sequência de caracteres.

Voltando ao exemplo anterior, escreveremos "João e Maria comem pão".

Veja que nesse caso não há qualquer problema em utilizarmos espaços.

Na verdade, o computador ignora praticamente tudo que escrevemos entre aspas, mas veremos mais tarde que não é bem assim.

As variáveis do tipo string são utilizadas para armazenar sequências de caracteres, normalmente utilizadas em textos ou mensagens. O tipo string é muito útil e bastante utilizado para exibir mensagens ou mesmo para gerar outros arquivos.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Uma string em Python tem um tamanho associado, assim como um conteúdo que pode ser acessado caractere a caractere. O tamanho de uma string pode ser obtido utilizando-se a função len.

Essa função retorna o número de caracteres na string. Dizemos que uma função retorna um valor quando podemos substituir o texto da função por seu resultado. A função len retorna um valor do tipo inteiro, representando a quantidade de caracteres contidos na string.

Se a string é vazia (representada simplesmente por "", ou seja, duas aspas sem nada entre elas, nem mesmo espaços em branco), seu tamanho é igual a zero. Façamos alguns testes, como mostra a listagem a seguir.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Listagem

A função `len()` lenght

```
>>> print (len("A")) 1
```

```
>>> print (len("AB")) 2
```

```
>>> print (len("")) 0
```

```
>>> print (len("O rato roeu a roupa")) 19
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Como dito anteriormente, outra característica de strings é poder acessar seu conteúdo caractere a caractere.

Sabendo que uma string tem um determinado tamanho, podemos acessar seus caracteres utilizando um número inteiro para representar sua posição.

Esse número é chamado de índice, e começamos a contar de zero.

Isso quer dizer que o primeiro caractere da string é de posição ou índice 0. Observe a figura:

String									
0	1	2	3	4	5	6	7	8	← Índice
A	B	C	D	E	F	G	H	I	← Conteúdo

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Para acessar os caracteres de uma string, devemos informar o índice ou posição do caractere entre colchetes ([]).

Como o primeiro caractere de uma string é o de índice 0, podemos acessar valores de 0 até o tamanho da string menos 1.

Logo, se a string contiver 9 caracteres, poderemos acessar os caracteres de 0 a 8.

Veja o resultado de alguns testes com strings na listagem a seguir.

Se tentarmos acessar um índice maior que a quantidade de caracteres da string, o interpretador emitirá uma mensagem de erro.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Listagem – Manipulação de strings no interpretador

```
>>> a = "ABCDEF"
```

```
>>> print(a[0]) A
```

```
>>> print(a[1]) B
```

```
>>> print(a[5]) F
```

```
>>> print(a[6])
```

Traceback (most recent call last):

File "", line 1, in Index

Error: string index out of range

```
>>> print(len(a)) 6
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Operações com strings

As variáveis de tipo string suportam operações como fatiamento, concatenação e composição.

Por fatiamento, podemos entender a capacidade de utilizar apenas uma parte de uma string, ou uma fatia.

A concatenação nada mais é que poder juntar duas ou mais strings em uma nova string maior.

A composição é muito utilizada em mensagens que enviamos à tela e consiste em utilizar strings como modelos onde podemos inserir outras strings.

Veremos cada uma dessas operações nas seções a seguir.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Operações com strings - Concatenação

O conteúdo de variáveis string podem ser somados, ou melhor, concatenados.

Para concatenar duas strings, utilizamos o operador de adição (+). Assim, "AB" + "C" é igual a "ABC".

Um caso especial de concatenação é a repetição de uma string várias vezes.

Para isso, utilizamos o operador de multiplicação (*): "A" * 3 é igual a "AAA". Vejamos alguns exemplos na listagem a seguir

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Operações com strings - Concatenação

O conteúdo de variáveis string podem ser somados, ou melhor, concatenados.

Para concatenar duas strings, utilizamos o operador de adição (+). Assim, "AB" + "C" é igual a "ABC".

Um caso especial de concatenação é a repetição de uma string várias vezes.

Para isso, utilizamos o operador de multiplicação (*): "A" * 3 é igual a "AAA". Vejamos alguns exemplos na listagem a seguir

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Operações com strings - Concatenação

Exemplo de concatenação

```
>>> s = "ABC" >>> print (s + "C") ABCC
```

```
>>> print (s + "D" * 4) ABCDDDD
```

```
>>> print ("X" + "-"*10 + "X") X-----X
```

```
>>> print (s+"x4 = "+s*4) ABCx4 = ABCABCABCABC
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Juntar várias strings para construir uma mensagem nem sempre é prático. Por exemplo, exibir que "João tem X anos", onde X é uma variável numérica.

Usando a composição de strings do Python, podemos escrever de forma simples e clara:

```
"João tem %d anos" % X
```

Onde o símbolo de % foi utilizado para indicar a composição da string anterior com o conteúdo da variável X.

O %d dentro da primeira string é o que chamamos de marcador de posição. O marcador indica que naquela posição estaremos colocando um valor inteiro, daí o %d.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Python suporta diversas operações com marcadores. Veremos mais sobre marcadores em outras partes dessa apostila.

A tabela apresenta os principais tipos de marcadores.

Veja que eles são diferentes, de acordo com o tipo de variável que vamos utilizar

Marcador	Tipo
%d	Números inteiros
%s	Strings
%f	Números decimais

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Imagine que precisamos apresentar um número como 001 ou 002, mas que também pode ser algo como 050 ou 561. Nesse caso, estamos querendo apresentar um número com três posições, completando com zeros à esquerda se o número for menor.

Podemos realizar essa operação utilizando "%03d" % X.

Observe que adicionamos 03 entre o % e o d.

Se você precisar apenas que o número ocupe três posições, mas não desejar zeros à esquerda, basta retirar o zero e utilizar "%3d" % X.

Isso é muito importante quando estamos gravando dados em um arquivo ou simplesmente exibindo informações na tela.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Exemplo de composição com marcadores

```
>>> idade = 22
```

```
>>> print("[%d]" % idade)
[22]
```

```
>>> print("[%03d]" % idade)
[022]
```

```
>>> print("[%3d]" % idade)
[ 22]
```

```
>>> print("[% -3d]" % idade)
[22]
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Quando formatamos números decimais, podemos utilizar dois valores entre o símbolo de % e a letra f.

O primeiro indica o tamanho total em caracteres a reservar;
O segundo, o número de casas decimais.

Assim, %5.2f diz que estaremos imprimindo um número decimal utilizando cinco posições, sendo que duas são para a parte decimal.

Isso é muito interessante para exibir o resultado de cálculos ou representar dinheiro.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Isso é muito interessante para exibir o resultado de cálculos ou representar dinheiro.

Por exemplo, para exibir R\$ 5, você pode utilizar

```
"R$%f" % 5
```

mas o resultado não é bem o que esperamos, pois normalmente utilizamos apenas dois dígitos após a vírgula quando falamos de dinheiro.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Vejamos alguns exemplos:

Exemplos de composição com números decimais

```
>>> print("%5f" % 5) 5.000000
```

```
>>> print("%5.2f" % 5) 5.00
```

```
>>> print("%10.5f" % 5) 5.00000
```

O primeiro indica o tamanho total em caracteres a reservar;
O segundo, o número de casas decimais.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

O poder da composição realmente aparece quando precisamos combinar vários valores em uma nova string.

Imagine que João tem 22 anos e apenas R\$ 51,34 no bolso.

Para exibir essa mensagem, podemos utilizar:

```
"%s tem %d anos e apenas R$%5.2f no bolso" % ("João", 22, 51.34)
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Python suporta diversas operações com marcadores.

Veremos mais sobre marcadores em outras partes desta apostila.

Quando temos mais de um marcador na string, somos obrigados a escrever os valores a substituir entre parênteses.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Vejamos exemplos com outros tipos, e utilizando mais de uma variável na composição,

```
>>> nome = "João"
```

```
>>> idade = 22
```

```
>>> grana = 51.34
```

```
>>> print("%s tem %d anos e R$%f no bolso." % (nome, idade, grana))
```

```
João tem 22 anos e R$51.340000 no bolso.
```

```
>>> print("%12s tem %3d anos e R$%5.2f no bolso." % (nome, idade, grana))
```

```
João tem 22 anos e R$51.34 no bolso.
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

```
>>> print("%12s tem %03d anos e R$%5.2f no bolso." % (nome, idade, grana))  
João tem 022 anos e R$51.34 no bolso.
```

```
>>> print("%-12s tem %-3d anos e R$%-5.2f no bolso." % (nome, idade, grana))  
João tem 22 anos e R$51.34 no bolso.
```

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Fatiamento

O fatiamento em Python é muito poderoso. Imagine nossa string anterior

```
S = ABCDEFGHI
```

Podemos fatiá-la de forma a escrever apenas seus dois primeiros caracteres AB utilizando como índice [0:2].

O fatiamento funciona com a utilização de dois pontos no índice da string.

O número à esquerda dos dois pontos indica a posição de início da fatia;
O à direita, do fim.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Fatiamento

O número à esquerda dos dois pontos indica a posição de início da fatia;
O à direita, do fim.

No entanto, é preciso ter atenção ao final, pois no exemplo anterior utilizamos 2; e o C, que é o caractere na posição 2; não foi incluído.

Dizemos que isso acontece porque o final da fatia não é incluído na mesma, sendo deixado de fora.

Entenda `[0:2]` como a fatia de caracteres da posição 0 até a posição 2, sem incluí-la, ou o intervalo fechado em 0 e aberto em 2

Variaveis e entradas de dados – parte 2

Exemplo de fatiamento

```
>>> s="ABCDEFGH I"
```

```
>>> print (s[0:2])
```

```
AB
```

```
>>> print (s[1:2])
```

```
B
```

```
>>> print (s[2:4])
```

```
CD
```

```
>>> print (s[0:5])
```

```
ABCDE
```

```
>>> print (s[1:8])
```

```
BCDEFGH
```

Variáveis e entradas de dados – parte 2

Fatiamento

Podemos também omitir o número da esquerda ou o da direita para representar do início ou até o final.

Assim, [:2] indica do início até o segundo caractere (sem incluí-lo), e [1:] indica do caractere de posição 1 até o final da string.

Observe que, nesse caso, nem precisamos saber quantos caracteres a string contém.

Se omitirmos o início e o fim da fatia, estaremos fazendo apenas uma cópia de todos os caracteres da string para uma nova string.

Podemos também utilizar valores negativos para indicar posições a partir da direita.

Assim -1 é o último caractere; -2, o penúltimo; e assim por diante. Vejamos alguns exemplos:

Variaveis e entradas de dados – parte 2

Exemplo de fatiamento com omissão de valores e com índices negativos

```
>>> s="ABCDEFGH I"
```

```
>>> print (s[:2])
```

```
AB
```

```
>>> print (s[1:])
```

```
BCDEFGH I
```

```
>>> print (s[0:-2]) ABCDEFG
```

```
>>> print (s[:])
```

```
ABCDEFGH I
```

Variaveis e entradas de dados – parte 2

Exemplo de fatiamento com omissão de valores e com índices negativos

```
>>> s="ABCDEFGH I"
```

```
>>> print (s[-1:])
```

```
I
```

```
>>> print (s[-5:7])
```

```
EFG
```

```
>>> print (s[-2:-1])
```

```
H
```

Variáveis e entradas de dados – parte 2

Sequências e tempo

Um programa é executado linha por linha pelo computador, executando as operações descritas no programa uma após a outra.

Quando trabalhamos com variáveis, devemos nos lembrar de que o conteúdo de uma variável pode mudar com o tempo.

Isso porque a cada vez que alteramos o valor de uma variável, o valor anterior é substituído pelo novo. Observe o programa da listagem a seguir.

A variável dívida foi utilizada para registrar quanto alguém estava devendo; e a variável compra, o valor de novas despesas dessa pessoa. Como somos justos, a pessoa começou sem dívidas em **1**

Variáveis e entradas de dados – parte 2

Sequências e tempo

Exemplo de sequência e tempo

dívida = 0 ①

compra = 100 ②

dívida = dívida + compra ③

compra = 200 ④

dívida = dívida + compra ⑤

Variáveis e entradas de dados – parte 2

Sequências e tempo

Exemplo de sequência e tempo

compra = 300 ⑥

dívida = dívida + compra ⑦

compra = 0 ⑧

print(dívida) ⑨

Variáveis e entradas de dados – parte 2

Sequências e tempo

Em ❷, temos a primeira compra no valor de R\$ 100.

No entanto, o valor da dívida continua sendo 0, pois ainda não alteramos seu valor de forma a adicionar a compra realizada. Isso é feito em ❸. Observe que estamos atualizando o valor da dívida com o valor atual mais a compra.

Em ❹, registramos uma nova compra no valor de R\$ 200. Nesse ponto, compra tem seu valor substituído por R\$ 200, causando a perda do valor anterior R\$ 100.

Como já somamos o valor anterior na variável dívida, essa perda não representará um problema

Variáveis e entradas de dados – parte 2

Sequências e tempo

⑤ é idêntica a ③, mas o resultado é bem diferente.

Nesse momento, compra vale R\$ 200; e dívida, R\$ 100.

Em ⑥, alteramos o valor de compra novamente.

Dessa vez, a compra foi de R\$ 300. ⑦ é idêntica a ③ e ⑤, mas seu resultado é diferente, pois nesse momento temos compra valendo R\$ 300; e dívida igual a R\$ 300 (100 + 200), sendo atualizada para R\$ 600 (300 + 300).

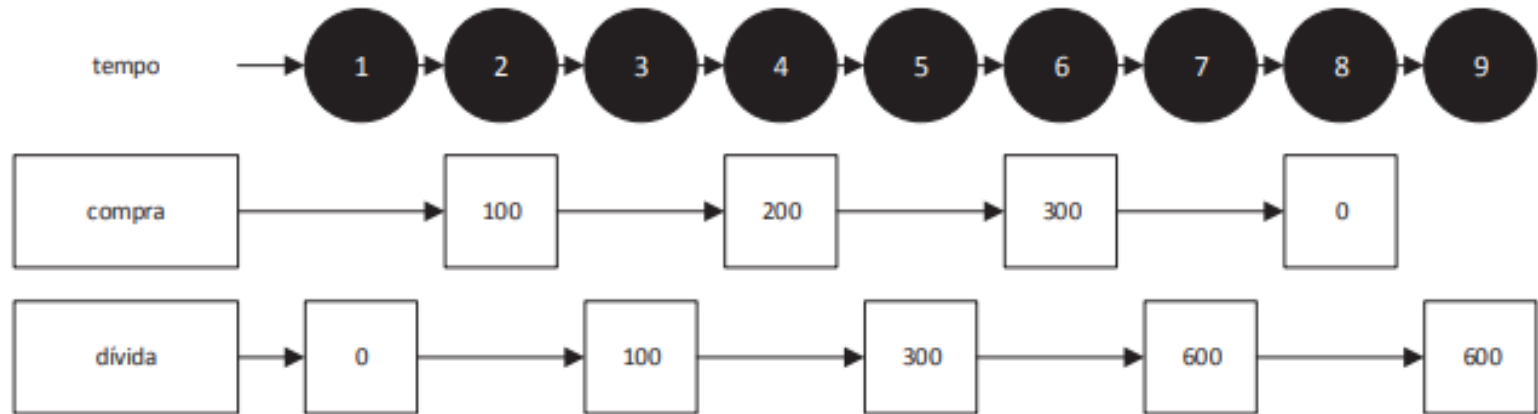
Em ⑧, simplesmente dizemos que a compra foi 0, representando que a pessoa não comprou mais nada.

⑨ exibe o conteúdo da variável dívida na tela (600)

Variáveis e entradas de dados – parte 2

Sequências e tempo

A figura mostra a evolução do conteúdo de nossas duas variáveis em função do tempo, representado pelo número da linha:



Variáveis e entradas de dados – parte 2

Rastreamento

Uma das principais diferenças entre ler um texto e um programa é justamente seguir as mudanças de valores de cada variável conforme o programa é executado.

Entender que o valor das variáveis pode mudar durante a execução do programa não é tão natural, mas é fundamental para a programação de computadores.

Um programa não pode ser lido como um texto, mas cuidadosamente analisado linha a linha (**galera, trabalha sempre com F9!!!**).

Ao escrever seus programas, verifique linha a linha os efeitos e mudanças causados no valor de cada variável.

Variáveis e entradas de dados – parte 2

Rastreamento

Para programar corretamente, você deve ser capaz de entender o que cada linha do programa significa e os efeitos que produz.

Essa atividade, chamada de rastreamento, é muito importante para entender novos programas e para encontrar erros nos programas que você escreverá.

Para rastrear um programa, utilize lápis, borracha e uma folha de papel.

Escreva o nome de suas variáveis na folha de papel, como se fossem títulos de colunas, deixando espaço para ser preenchido embaixo desses nomes.

Leia uma linha do programa de cada vez e escreva o valor atribuído a cada variável na outra folha, na mesma coluna em que escreveu o nome da variável.

Variáveis e entradas de dados – parte 2

Rastreamento

Se o valor da variável mudar, escreva o novo valor e risque o anterior, um embaixo do outro, formando uma coluna.

Tela	dívida	Compra
600	0	100
	100	200
	300	300
	600	0

Ao exibir algo na tela, escreva também na outra folha, como se ela fosse a sua tela (você pode desenhar a tela como se fosse uma variável, mas se lembre de deixar um pouco mais de espaço).

Um exemplo de como ficaria o resultado do rastreamento do programa da listagem anterior é apresentado na figura.

Variáveis e entradas de dados – parte 2

Rastreamento

O rastreamento vai ajudá-lo a entender melhor as mudanças de valores de suas variáveis e a acompanhar a execução do programa, como mais tarde será feito pelo computador.

É um processo detalhado que precisa de atenção. Não tente simplificá-lo ou começar a rastrear no meio de um programa.

Você deve rastrear linha a linha, do início ao fim do programa.

Se você encontrar um erro, pode parar o rastreamento e corrigi-lo, mas lembre-se de recomeçar do início sempre que alterar o programa ou os valores sendo rastreados.

Variáveis e entradas de dados – parte 2

Rastreamento

O rastreamento vai ajudá-lo a entender melhor as mudanças de valores de suas variáveis

Dominar o rastreamento de um programa é essencial para programar e ajuda muito a entender como os programas realmente funcionam.

Lembre-se de que programar é detalhar, e que simplesmente ler o texto de um programa não é suficiente. Você deve rastreá-lo para entendê-lo.

Embora pareça óbvio, esse é um dos erros mais comuns quando se começa a programar.

Se um programa não funciona ou se você não entendeu exatamente o que ele faz, o rastreamento é a melhor ferramenta.

Variáveis e entradas de dados – parte 2

Entrada de dados

Até agora nossos programas trabalharam apenas com valores conhecidos, escritos no próprio programa.

No entanto, o melhor da programação é poder escrever a solução de um problema e aplicá-la várias vezes.

Para isso, precisamos melhorar nossos programas de forma a permitir que novos valores sejam fornecidos durante sua execução, de modo que poderemos executá-los com valores diferentes sem alterar os programas em si.

Chamamos de entrada de dados o momento em que seu programa recebe dados ou valores por um dispositivo de entrada de dados (como o teclado do computador) ou de um arquivo em disco.

Variáveis e entradas de dados – parte 2

Entrada de dados

A função `input()` é utilizada para solicitar dados do usuário.

Ela recebe um parâmetro, que é a mensagem a ser exibida, e retorna o valor digitado pelo usuário. Vejamos um exemplo na listagem:

Entrada de dados

```
x = input("Digite um número: ")  
print(x)
```

Saída na tela, tendo o 5 como exemplo de número digitado pelo usuário

```
Digite um número: 5  
5
```

Variáveis e entradas de dados – parte 2

Entrada de dados

Listagem Exemplo de entrada de dados:

```
nome = input("Digite seu nome:") ❶
```

```
print("Você digitou %s" % nome)
```

```
print("Olá, %s!" % nome)
```

Em ❶, solicitamos a entrada de dados, no caso, o nome do usuário. A mensagem “Digite seu nome:” é exibida, e o programa para até que o usuário digite ENTER. Só então o resto do programa é executado.

Variáveis e entradas de dados – parte 2

Entrada de dados

Vejamos a saída de dados quando digitamos João como nome na listagem:

```
nome = input("Digite seu nome:") ❶
```

```
print("Você digitou %s" % nome)
```

```
print("Olá, %s!" % nome)
```

Tela

Digite seu nome: João

Você digitou João

Olá, João!

Execute o programa outras vezes digitando, por exemplo, 123 como nome. Observe que o programa não se importa com os valores digitados pelo usuário. Essa verificação deve ser feita por seu programa. Veremos como fazer isso em outro capítulo, como validação de dados.

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

A função `input()` sempre retorna valores do tipo `string`, ou seja, não importa se digitamos apenas números, o resultado sempre é `string`.

Para resolver esse pequeno problema, vamos utilizar a função `int()` para converter o valor retornado em um número inteiro, e a função `float()` para convertê-lo em número decimal ou de ponto flutuante.

Vejamos outro exemplo usando essas funções, no qual devemos calcular o valor de um bônus por tempo de serviço na listagem a seguir:

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Listagem - Cálculo de bônus por tempo de serviço

```
anos = int(input("Anos de serviço: "))
```

```
valor_por_ano = float(input("Valor por ano: "))
```

```
bônus = anos * valor_por_ano
```

```
print("Bônus de R$ %5.2f" % bonus)
```

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Vejamos o resultado se testarmos 10 anos e R\$ 25 por ano na tela da listagem.

Observe que escrevemos apenas 25, e não R\$ 25.

Isso porque 25 é um número, e R\$ 25 é uma string.

Por enquanto, não vamos misturar dois tipos de dados na mesma entrada de dados, para simplificar nossos programas.

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Listagem – Resultado do cálculo para 10 anos e R\$ 25 por ano

Anos de serviço: 10

Valor por ano: 25

Bônus de R\$ 250.00

```
anos = int(input("Anos de serviço: "))  
  
valor_por_ano = float(input("Valor por ano: "))  
  
bônus = anos * valor_por_ano  
  
print("Bônus de R$ %5.2f" % bônus)
```

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Execute o programa novamente com outros valores.

Experimente digitar uma letra em anos de serviço ou em valor por ano.

Você receberá uma mensagem de erro, pois a conversão de letras em números não é automática.

A próxima seção explica melhor o problema

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Exercícios:

Exercício 3.7 Faça um programa que peça dois números inteiros. Imprima a soma desses dois números na tela.

Exercício 3.8 Escreva um programa que leia um valor em metros e o exiba convertido em milímetros.

Exercício 3.9 Escreva um programa que leia a quantidade de dias, horas, minutos e segundos do usuário. Calcule o total em segundos.

Variáveis e entradas de dados – parte 2

Conversão da entrada de dados

Exercícios:

Exercício 3.10 Faça um programa que calcule o aumento de um salário. Ele deve solicitar o valor do salário e a porcentagem do aumento. Exiba o valor do aumento e do novo salário.

Exercício 3.11 Faça um programa que solicite o preço de uma mercadoria e o percentual de desconto. Exiba o valor do desconto e o preço a pagar.

Exercício 3.12 Escreva um programa que calcule o tempo de uma viagem de carro. Pergunte a distância a percorrer e a velocidade média esperada para a viagem.

Variaveis e entradas de dados – parte 2

Conversão da entrada de dados

Exercícios:

Exercício 3.13 Escreva um programa que converta uma temperatura digitada em °C em °F. A fórmula para essa conversão é:

$$F = \frac{9 \times C}{5} + 32$$

Exercício 3.14 Escreva um programa que pergunte a quantidade de km percorridos por um carro alugado pelo usuário, assim como a quantidade de dias pelos quais o carro foi alugado. Calcule o preço a pagar, sabendo que o carro custa R\$ 60 por dia e R\$ 0,15 por km rodado.

Exercício 3.15 Escreva um programa para calcular a redução do tempo de vida de um fumante. Pergunte a quantidade de cigarros fumados por dia e quantos anos ele já fumou. Considere que um fumante perde 10 minutos de vida a cada cigarro, calcule quantos dias de vida um fumante perderá. Exiba o total em dias.

Variáveis e entradas de dados – parte 2

Erros comuns

A entrada de dados é um ponto frágil em nossos programas.

Como não temos como prever o que o usuário vai digitar, temos que nos preparar para reconhecer os erros mais comuns.

Vejamos um programa que lê três valores na listagem ao lado:

Listagem Entrada de dados com conversão de tipos

```
nome = input("Digite seu nome: ")
```

```
idade = int(input("Digite sua idade: "))
```

```
saldo = float(input("Digite o saldo da sua conta  
bancária: "))
```

```
print(nome)
```

```
print(idade)
```

```
print(saldo)
```

Variáveis e entradas de dados – parte 2

A listagem ao lado mostra o resultado da execução do programa quando todos os valores são digitados corretamente.

Correto significa uma ausência de erros durante a função `input()` ou durante a conversão do valor retornado pela função

Listagem Exemplo de entrada de dados

Digite seu nome: João

Digite sua idade: 42

Digite o saldo da sua conta bancária: 15756.34

João

42

15756.34

Variáveis e entradas de dados – parte 2

- A listagem ao lado mostra outro exemplo de entrada de dados bem-sucedida.

Observe que, como utilizamos a função `float()` para converter o saldo, mesmo inserindo 34 o valor foi convertido para 34.0

Listagem – Exemplo de entrada de dados

Digite seu nome: Maria

Digite sua idade: 28

Digite o saldo da sua conta bancária: 34

Maria

28

34.0

Variáveis e entradas de dados – parte 2

- Já na listagem a seguir, temos um exemplo de erro durante a entrada de dados.

No caso, digitamos letras (abc) que não podem ser convertidas em um valor inteiro.

Observe que o erro interrompe nosso programa, exibindo a linha em que ocorreu e o nome do erro.

Nesse caso, o erro aconteceu na linha 2 e seu nome é

ValueError: invalid literal for int with base 10: 'abc'

Listagem – Erro de conversão

Digite seu nome: Minduim

Digite sua idade: abc

Traceback (most recent call last): File "input/input2.py", line 2, in

```
idade = int(input("Digite sua idade: "))
```

ValueError: invalid literal for int() with base 10: 'abc'

Condições

+ Nem sempre todas as linhas dos programas serão executadas.

Muitas vezes, será mais interessante decidir que partes do programa devem ser executadas com base no resultado de uma condição.

+ A base dessas decisões consistirá em expressões lógicas que permitam representar escolhas em programas

Executar ou não executar? Eis a Questão!

Atalhos no Spyder – 125 no total... Aqui alguns

<https://keycombiner.com/collections/spyder/>



Atalhos no Spyder

Aqui estão alguns dos atalhos mais comuns no Spyder, um ambiente de desenvolvimento integrado (IDE) para Python:

- Executar o código: Pressione "F5" ou "Ctrl+Enter"
- Comentar código: Selecione o código e pressione "Ctrl+1"
- Descomentar código: Selecione o código e pressione "Ctrl+1"
- Indentação automática: Selecione o código e pressione "Tab"
- Remover indentação automática: Selecione o código e pressione "Shift+Tab"
- Auto completar: Pressione "Ctrl+Space"
- Ir para a definição da função: Pressione "Ctrl+Click" ou "Ctrl+B"
- Ir para a declaração da variável: Pressione "Ctrl+Click" ou "Ctrl+B"
- Selecionar todas as ocorrências da palavra selecionada: Selecione a palavra e pressione "Ctrl+Shift+G"
- Excluir a linha atual: Pressione "Ctrl+Shift+D"
- Aumentar / diminuir a tela "Ctrl +" "Ctrl -"

Condições

• **if**

+ As condições servem para selecionar quando uma parte do programa deve ser ativada e quando deve ser simplesmente ignorada.

Em Python, a estrutura de decisão é o if. Seu formato é apresentado na listagem 4.1.

+ Listagem – Formato da estrutura de condicional if

if : <condição>

bloco verdadeiro

Condições

O if nada mais é que o nosso “se”.

Poderemos então entendê-lo em português da seguinte forma:

se a condição for verdadeira, faça alguma coisa.

Vejamos um exemplo: ler dois valores e imprimir o maior deles, apresentado na listagem.

Listagem – Condições

```
a = int(input("Primeiro valor: "))
```

```
b = int(input("Segundo valor: "))
```

Condições

```
if a > b: ❶
```

```
    print ("O primeiro número é o maior!") ❷
```

```
if b > a: ❸
```

```
    print ("O segundo número é o maior!") ❹
```

Em ❶, temos a condição $a > b$. Essa expressão será avaliada, e, se o seu resultado for verdadeiro, a linha ❷ será executada. Se for falso, a linha ❷ será ignorada.

O mesmo acontece para a condição $b > a$ da linha ❸. Se o seu resultado for verdadeiro, a linha ❹ será executada. Se for falso, ignorada.

Condições

A sequência de execução do programa é alterada de acordo com os valores digitados como o primeiro e segundo valores.

Digite o programa da listagem anterior e execute-o duas vezes.

Na primeira vez, digite um valor maior primeiro e um menor em segundo.

Na segunda vez, inverta esses valores e verifique se a mensagem na tela também mudou.

Quando o primeiro valor é maior que o segundo, temos as seguintes linhas sendo executadas: ①, ②, ③.

Quando o primeiro valor é menor que o segundo, temos outra sequência: ①, ③, ④.

É importante entender que a linha com a condição em si é executada mesmo se o resultado da expressão for falso.

Condições

As linhas ❶ e ❸ foram terminadas com o símbolo dois pontos (:).

Quando isso acontece, temos o anúncio de um bloco de linhas a seguir.

Em Python, um bloco é representado deslocando-se o início da linha para a direita.

O bloco continua até a primeira linha com deslocamento diferente.

Observe que começamos a escrever a linha ❷ alguns caracteres mais à direita da linha anterior ❶ que iniciou o bloco.

Como a linha ❸ foi escrita mais à esquerda, dizemos que o bloco da linha ❷ foi terminado.

Condições

TRÍVIA

Python é uma das poucas linguagens de programação que utiliza o deslocamento do texto à direita (recuo) para marcar o início e o fim de um bloco.

Outras linguagens contam com palavras especiais para isso, como BEGIN e END, em Pascal; ou as famosas chaves ({ e }), em C e Java.

Exercício 4.1 Analise o programa da listagem 4.2. Responda o que acontece se o primeiro e o segundo valores forem iguais? Explique.

Condições

Vejam os outros exemplos, onde solicitaremos a idade do carro do usuário e, em seguida, escreveremos novo se o carro tiver menos de três anos; ou velho, em caso contrário.

Listagem 4.3 – Carro novo ou velho, dependendo da idade

```
idade = int(input("Digite a idade do seu carro: "))
```

```
if idade <= 3:
```

```
    print("Seu carro é novo") ❶
```

```
if idade > 3:
```

```
    print("Seu carro é velho") ❷
```

Execute o programa da listagem 4.3 e verifique o que aparece na tela. Você pode executá-lo várias vezes com as seguintes idades: 1, 3 e 5. A primeira condição é `idade <= 3`. Essa condição decide se a linha com função `print` ❶ será ou não executada.

Condições

if

Como é uma condição simples, podemos entender que só exibiremos a mensagem do carro novo para as idades 0, 1, 2 e 3.

A segunda condição, $\text{idade} > 3$, é o inverso da primeira.

Se você observar de perto, não há um só número que torne ambas verdadeiras ao mesmo tempo.

A segunda decisão é responsável por decidir a impressão da mensagem do carro velho 2

Embora óbvio que um carro não poderia ter valores negativos como idade, o programa não trata desse problema. **Vamos alterá-lo mais adiante para verificar valores inválidos.**

Condições

Exercício 4.2 Escreva um programa que pergunte a velocidade do carro de um usuário. Caso ultrapasse 80 km/h, exiba uma mensagem dizendo que o usuário foi multado. Nesse caso, exiba o valor da multa, cobrando R\$ 5 por km acima de 80 km/h.

Um bloco de linhas em Python pode ter mais de uma linha, embora o último exemplo mostre apenas dois blocos com uma linha em cada.

Se você precisar de duas ou mais no mesmo bloco, escreva essas linhas na mesma direção ou na mesma coluna da primeira linha do bloco.

Isso basta para representá-lo.

Condições

if

Um problema comum é quando temos que pagar Imposto de Renda.

Normalmente, pagamos o Imposto de Renda por faixa de salário. Imagine que para salários menores que R\$ 1.000,00 não teríamos imposto a pagar, ou seja, alíquota 0%.

Para salários entre R\$ 1.000,00 e R\$ 3.000,00 pagaríamos 20%.

Acima desses valores, a alíquota seria de 35%. Esse problema se pareceria muito com o anterior, salvo se o imposto não fosse cobrado diferentemente para cada faixa, ou seja, quem ganha R\$ 4.000,00 tem os primeiros R\$ 1.000,00 isentos de imposto; com o montante entre R\$ 1.000,00 e R\$ 3.000,00 pagando 20%, e o restante pagando os 35%.

Condições

Listagem 4.4 – Cálculo do Imposto de Renda

```
salário = float(input("Digite o salário para cálculo do imposto: "))
```

```
base = salário ❶
```

```
imposto = 0
```

```
if base > 3000: ❷
```

```
    imposto = imposto + ((base - 3000) * 0.35) ❸
```

```
    base = 3000 ❹
```

```
if base > 1000: ❺
```

```
    imposto = imposto + ((base - 1000) * 0.20) ❻
```

```
print("Salário: R$%6.2f Imposto a pagar: R$%6.2f" % (salário, imposto))
```

Condições

O programa da listagem anterior é interessante.

Tente executá-lo algumas vezes e compare o valor impresso com o valor calculado por você.

Rastreie o programa e tente entender o que ele faz antes de ler o parágrafo seguinte.

Verifique o que acontece para salários de R\$ 500,00, R\$ 1.000,00 e R\$ 1,500,00.

Em **1** temos a variável base recebendo uma cópia de salário. Isso é necessário porque, quando atribuímos um novo valor para uma variável, o valor anterior é substituído (e perdido se não o guardarmos em outro lugar).

Como vamos utilizar o valor do salário digitado para exibi-lo na tela, não podemos perdê-lo; por isso, a necessidade de uma variável auxiliar chamada aqui de base

Condições

if

Em ❷ verificamos se a base é maior que R\$ 3.000,00.

Se verdadeiro, executamos as linhas ❸ e ❹. Em ❸, calculamos 35% do valor superior a R\$ 3.000,00.

O resultado é armazenado na variável imposto. Como essa variável contém o valor a pagar para essa quantia, atualizaremos o valor de base para R\$ 3.000,00 ❹, pois o que ultrapassa esse valor já foi tarifado.

Em ❺ verificamos se o valor de base é maior que R\$ 1.000,00, calculando 20% de imposto em ❻, caso verdadeiro. Vejamos o rastreamento para um salário de R\$ 500,00:

salário	base	imposto
500	500	0

Condições

if

Para um salário de R\$ 1.500,00:

salário	base	imposto
1500	1500	0
		100

Para um salário de R\$ 3.000,00

salário	base	imposto
3000	3000	0
		400

Condições

Para um salário de R\$ 5.000,00

salário	base	imposto
5000	5000	0
	3000	700
		1100

Exercício 4.3 Escreva um programa que leia três números e que imprima o maior e o menor.

Exercício 4.4 Escreva um programa que pergunte o salário do funcionário e calcule o valor do aumento. Para salários superiores a R\$ 1.250,00, calcule um aumento de 10%. Para os inferiores ou iguais, de 15%.

else

Condições

Quando há problemas, como a mensagem do carro velho, onde a segunda condição é simplesmente o inverso da primeira, podemos usar outra forma de if para simplificar os programas.

Essa forma é a cláusula else para especificar o que fazer caso o resultado da avaliação da condição seja falso, sem precisarmos de um novo if. Vejamos como ficaria o programa reescrito para usar else

```
idade = int(input("Digite a idade de seu carro: "))
```

```
if idade <= 3:
```

```
    print("Seu carro é novo")
```

```
else: ①
```

```
    print("Seu carro é velho") ②
```

Condições

Isso é necessário porque else inicia um bloco, da mesma forma que if. É importante notar que devemos escrever else na mesma coluna do if, ou seja, com o mesmo recuo.

Assim, o interpretador reconhece que else se refere a um determinado if. Você obterá um erro caso não alinhe essas duas estruturas na mesma coluna.

A vantagem de usar `else` é deixar os programas mais claros, uma vez que podemos expressar o que fazer caso a condição especificada em `if` seja falsa.

A linha **2** só é executada se a condição `idade <= 3` for falsa.

• • • • •
• • • • •
• + • -
+ •

else

Condições

Exercício 4.5 Execute o programa (Listagem 4.5) e experimente alguns valores. Verifique se os resultados foram os mesmos do programa anterior (Listagem 4.3).

Exercício 4.6 Escreva um programa que pergunte a distância que um passageiro deseja percorrer em km. Calcule o preço da passagem, cobrando R\$ 0,50 por km para viagens de até de 200 km, e R\$ 0,45 para viagens mais longas.

Condições

- **Estruturas aninhadas**

Nem sempre nossos programas serão tão simples. Muitas vezes, precisaremos aninhar vários if para obter o comportamento desejado do programa. Aninhar, nesse caso, é utilizar um if dentro de outro.

Vejamos o exemplo de calcular a conta de um telefone celular da empresa Tchau. Os planos da empresa Tchau são bem interessantes e oferecem preços diferenciados de acordo com a quantidade de minutos usados por mês.

Abaixo de 200 minutos, a empresa cobra R\$ 0,20 por minuto. Entre 200 e 400 minutos, o preço é de R\$ 0,18. Acima de 400 minutos, o preço por minuto é de R\$ 0,15. O programa da listagem seguinte resolve esse problema.

Condições

- **Estruturas aninhadas**

Listagem – Conta de telefone com três faixas de preço

```
minutos=int(input("Quantos minutos você utilizou este mês:"))
```

```
if minutos < 200: ❶
```

```
    preço = 0.20 ❷
```

```
else:
```

```
    if minutos < 400: ❸
```

```
        preço = 0.18 ❹
```

```
    else: ❺
```

```
        preço = 0.15 ❻
```

```
print("Você vai pagar este mês: R$%6.2f" % (minutos * preço))
```

Condições

- **Estruturas aninhadas**

Em ❶, temos a primeira condição: minutos < 200. Se a quantidade de minutos for menor que 200, atribuímos 0,20 ao preço em ❷. Até aqui, nada de novo. Observe que if de ❸ está dentro de else da linha anterior: dizemos que está aninhado dentro de else. A condição de ❹, minutos < 400, decide se vamos executar a linha de ❺ ou a de ❻. Observe que else de ❼ está alinhado com if de ❸. No final, calculamos e imprimimos o preço na tela. Lembre-se de que o alinhamento do texto é muito importante em Python.

Vejamos, por exemplo a situação em que cinco categorias são necessárias. Façamos um programa que leia a categoria de um produto e determine o preço pela tabela ao lado

Categoria	Preço
1	10,00
2	18,00
3	23,00
4	26,00
5	31,00

Condições

Estruturas aninhadas

À esquerda da listagem, Categoria X Preço, você encontrará os números de linha do programa, numeradas de 1 a 19.

Esses números servem **apenas** para ajudar o entendimento da explicação a seguir: **lembre-se de não digitá-los**

```
1 categoria = int(input("Digite a categoria do produto:"))
2 if categoria == 1:
3     preço = 10
4 else:
5     if categoria == 2:
6         preço = 18
7     else:
8         if categoria == 3:
9             preço = 23
10        else:
11            if categoria == 4:
12                preço = 26
13            else:
14                if categoria == 5:
15                    preço = 31
16                else:
17                    print("Categoria inválida, digite um valor entre 1 e 5!")
18                    preço = 0
19 print("O preço do produto é: R$%.2f" % preço)
```


Condições

Estruturas aninhadas

Observe que o alinhamento se tornou um grande problema, uma vez que tivemos que deslocar à direita a cada else.

No programa da listagem anterior, introduzimos o conceito de validação da entrada.

Dessa vez, se o usuário digitar um valor inválido, receberá uma mensagem de erro na tela.

Nada muito prático ou bonito. Vejamos a execução das linhas dependendo da categoria digitada na tabela ao lado:

Categoria	Linhas executadas
1	1,2,3,19
2	1,2,4,5,6,19
3	1,2,4,5,7,8,9,19
4	1,2,4,5,7,8,10,11,12,19
5	1,2,4,5,7,8,10,11,13,14,15,19
outras	1,2,4,5,7,8,10,11,13,14,16,17,18,19

Condições

Estruturas aninhadas

Quando lermos um programa com estruturas aninhadas, devemos prestar muita atenção para visualizar corretamente os blocos.

Observe como o alinhamento é importante!

Condições

Estruturas aninhadas

elif

Python apresenta uma solução muito interessante ao problema de múltiplos ifs aninhados. A cláusula elif substitui um par else if, mas sem criar outro nível de estrutura, evitando problemas de deslocamentos desnecessários à direita.

Vamos revisitar o problema da listagem anterior, dessa vez usando elif. Veja o resultado no programa:

```
categoria = int(input("Digite a categoria do produto:"))
if categoria == 1:
    preço = 10
elif categoria == 2:
    preço = 18
elif categoria == 3:
    preço = 23
elif categoria == 4:
    preço = 26
elif categoria == 5:
    preço = 31
else:
    print("Categoria inválida, digite um valor entre 1 e 5!")
    preço = 0
print("O preço do produto é: R$%6.2f" % preço)
```

Condições

Estruturas aninhadas

Exercício 4.8 Escreva um programa que leia dois números e que pergunte qual operação você deseja realizar. Você deve poder calcular a soma (+), subtração (-), multiplicação (*) e divisão (/). Exiba o resultado da operação solicitada.

Exercício 4.9 Escreva um programa para aprovar o empréstimo bancário para compra de uma casa. O programa deve perguntar o valor da casa a comprar, o salário e a quantidade de anos a pagar. O valor da prestação mensal não pode ser superior a 30% do salário. Calcule o valor da prestação como sendo o valor da casa a comprar dividido pelo número de meses a pagar.

Condições

Estruturas aninhadas

Exercício 4.10 Escreva um programa que calcule o preço a pagar pelo fornecimento de energia elétrica. Pergunte a quantidade de kWh consumida e o tipo de instalação: R para residências, I para indústrias e C para comércios. Calcule o preço a pagar de acordo com a tabela a seguir.

Preço por tipo e faixa de consumo		
Tipo	Faixa (kWh)	Preço
Residencial	Até 500	R\$ 0,40
	Acima de 500	R\$ 0,65
Comercial	Até 1000	R\$ 0,55
	Acima de 1000	R\$ 0,60
Industrial	Até 5000	R\$ 0,55
	Acima de 5000	R\$ 0,60

Repetições

Repetições representam a base de vários programas.

São utilizadas para executar a mesma parte de um programa várias vezes, normalmente dependendo de uma condição.

Por exemplo, para imprimir três números na tela, poderíamos escrever um programa como o apresentado na listagem:

Listagem 5.1 – Imprimindo de 1 a 3

```
print(1)
print(2)
print(3)
```

Repetições

Podemos imaginar que para imprimir três números, começando de 1 até o 3, devemos variar `print(x)`, onde `x` varia de 1 a 3. Vejamos outra solução para o problema na listagem.

Listagem – Imprimindo de 1 a 3 usando uma variável

```
x=1
print(x)
x=2
print(x)
x=3
print(x)
```

Outra solução seria incrementar o valor de `x` após cada `print`. Vejamos essa solução na próxima listagem:

Repetições

Podemos imaginar que para imprimir três números, começando de 1 até o 3, devemos variar `print(x)`, onde `x` varia de 1 a 3. Vejamos outra solução para o problema na listagem.

Listagem – Imprimindo de 1 a 3 usando uma variável

```
x=1
print(x)
x=2
print(x)
x=3
print(x)
```

Outra solução seria incrementar o valor de `x` após cada `print`. Vejamos essa solução na próxima listagem:

Repetições

- Listagem – Imprimindo de 1 a 3 incrementando

```
x=1
print(x)
x=x+1
print(x)
x=x+1
print(x)
```

Porém, se o objetivo fosse escrever 100 números, a solução não seria tão agradável, pois teríamos que escrever pelo menos 200 linhas!

A estrutura de repetição aparece para nos auxiliar a resolver esse tipo de problema!

Repetições

Uma das estruturas de repetição do Python é o **while**, que repete um bloco enquanto a condição for verdadeira.

Seu formato é apresentado na listagem a seguir, onde condição é uma expressão lógica, e bloco representa as linhas de programa a repetir enquanto o resultado da condição for verdadeiro.

Listagem 5.4 – Formato da estrutura de repetição com while

```
while :  
    bloco
```

Repetições

Para resolver o problema de escrever três números utilizando o while, escreveríamos um programa como o da listagem 5.5.

Listagem – Imprimindo de 1 a 3 com while

```
x=1 ①
while x<=3: ②
    print(x) ③
    x = x + 1 ④
```

A execução desse programa seria um pouco diferente do que vimos até agora.

Primeiramente, ① seria executada inicializando a variável x com o valor 1. A linha ② seria uma combinação de estrutura condicional com estrutura de repetição.

Podemos entender a condição do while da mesma forma que a condição de if. A diferença é que, se a condição for verdadeira, repetiremos as linhas ③ e ④ (bloco) enquanto a avaliação da condição for verdadeira.

Repetições

Em ③ teremos a impressão na tela propriamente dita, onde x é 1.

Em ④ temos que o valor de x é acrescentado de 1.

Como x vale 1, $x + 1$ valerá 2.

Esse novo valor é então atribuído a x .

A parte nova é que a execução não termina após ④ que é o fim do bloco, mas retorna para ②.

É esse retorno que faz a estrutura de repetição especial.

Repetições

Agora, x vale 2 e $x \leq 3$ continua verdadeiro (True), logo, o bloco será executado outra vez.

③ realizará a impressão do valor 2, e ④ atualizará o valor de x para $x + 1$; nesse caso, $2 + 1 = 3$. A execução volta novamente para a linha ②.

A condição em ② é avaliada, e como x vale 3, e $x \leq 3$ continua verdadeira, fazendo com que as linhas ③ e ④ sejam executadas, exibindo 3 e atualizando o valor de x para 4 ($3 + 1$).

Nesse ponto, ②, temos que x vale 4 e que a condição $x \leq 3$ resulta em Falso (False), terminando, assim, a repetição do bloco.

Repetições

Exercício 5.1 Modifique o programa para exibir os números de 1 a 100.

Exercício 5.2 Modifique o programa para exibir os números de 50 a 100.

Exercício 5.3 Faça um programa para escrever a contagem regressiva do lançamento de um foguete. O programa deve imprimir 10, 9, 8, ..., 1, 0 e Fogo! na tela.

Repetições

Contadores O poder das estruturas de repetições é muito interessante, principalmente quando utilizamos condições com mais de uma variável. Imagine um problema onde deveríamos imprimir os números inteiros entre 1 e um valor digitado pelo usuário.

Vamos modificar o programa da listagem 5.5 de forma que o último número a imprimir seja informado pelo usuário. O programa já modificado é apresentado na listagem 5.6.
Listagem 5,6 – Impressão de 1 até um número digitado pelo usuário

```
fim=int(input("Digite o último número a imprimir:")) ❶
```

```
x = 1
```

```
while x <= fim: ❷
```

```
    print(x) ❸
```

```
    x = x + 1 ❹
```

Repetições

Nesse caso, o programa imprimirá de 1 até o valor digitado em ❶.

Em ❷ utilizamos a variável fim para representar o limite de nossa repetição.

Agora vamos analisar o que realizamos com a variável x dentro da repetição.

Em ❸, o valor de x é simplesmente impresso.

Em ❹ atualizamos o valor de x com $x + 1$, ou seja, com o próximo valor inteiro. Quando realizamos esse tipo de operação dentro de uma repetição estamos contando. Logo diremos que x é um contador.

Um contador é uma variável utilizada para contar o número de ocorrências de um determinado evento; nesse caso, o número de repetições do while, que satisfaz às necessidades de nosso problema.

Repetições

Experimente esse programa com vários valores, primeiro digitando 5, depois 500 e, por fim, 0 (zero). Provavelmente, 5 e 500 produzirão os resultados esperados, ou seja, a impressão de 1 até 5, ou de 1 até 500.

Porém, quando digitamos zero, nada acontece, e o programa termina logo a seguir, sem impressão. Analisando nosso programa quando a variável fim vale 0, ou seja, quando digitamos 0 em ❶, temos que a condição em ❷ é $x \leq \text{fim}$.

Como x é 1 e fim é 0, temos que $1 \leq 0$ é falso desde a primeira execução, fazendo com que o bloco a repetir não seja executado, uma vez que sua condição de entrada é falsa.

O mesmo aconteceria na inserção de valores negativos

Repetições

Imagine que o problema agora seja um pouco diferente: imprimir apenas os números pares entre 0 e um número digitado pelo usuário, de forma bem similar ao problema anterior.

Poderíamos resolver o problema com um `if` para testar se `x` é par ou ímpar antes de imprimir. Vale lembrar que um número é par quando é 0 ou múltiplo de 2.

Quando é múltiplo de 2, temos que o resto da divisão desse número por 2 é 0, ou seja, o resultado é uma divisão exata, sem resto.

Em Python, podemos escrever esse teste com `x % 2 == 0` (resto da divisão de `x` por 2 é igual a zero), alterando o programa anterior para o da listagem a seguir:

Repetições

Listagem 5.7 – Impressão de números pares de 0 até um número digitado pelo usuário

```
fim=int(input("Digite o último número a imprimir:"))  
x = 0 ❶  
while x <= fim:  
    if x % 2 == 0: ❷  
        print(x) ❸  
    x = x + 1
```

Veja que, para começar a imprimir do 0, e não de 1, modificamos ❶.

Um detalhe importante é que ❸ é um bloco dentro de if ❷, sendo para isso deslocado à direita.

Execute o programa e verifique seu resultado

Repetições

Agora, finalmente, estamos resolvendo o problema, mas poderíamos resolvê-lo de forma ainda mais simples se adicionássemos 2 a x a cada repetição. Isso garantiria que x sempre fosse par. Vejamos o programa da listagem.

Listagem 5.8 – Impressão de números pares de 0 até um número digitado pelo usuário, sem if

```
fim=int(input("Digite o último número a imprimir:"))
x = 0
while x <= fim:
    print(x)
    x = x + 2
```

Esses dois exemplos mostram que existe mais de uma solução para o problema, que podemos escrever programas diferentes e obter a mesma solução. Essas soluções podem ser às vezes mais complicadas, às vezes mais simples, mas ainda assim corretas.

Repetições

Exercício 5.4 Modifique o programa anterior para imprimir de 1 até o número digitado pelo usuário, mas, dessa vez, apenas os números ímpares.

Exercício 5.5 Reescreva o programa anterior para escrever os 10 primeiros múltiplos de 3.

Repetições

• • • • • +
• • • • •
• + •
+ •

• Vejamos outro tipo de problema. Imagine ter que imprimir a tabuada de adição de um número digitado pelo usuário.

|
+ Essa tabuada deve ser impressa de 1 a 10, sendo n o número digitado pelo usuário. Teríamos, assim, n+1, n+2, ... n+10. Confira a solução na listagem

Listagem – Tabuada simples

```
n = int(input("Tabuada de:"))  
x = 1  
while x <= 10:  
    print(n+x)  
    x=x+1
```

• • •

Execute o programa anterior e experimente diversos valores

Repetições

Exercício 5.6 Altere o programa anterior para exibir os resultados no mesmo formato de uma tabuada: $2 \times 1 = 2$, $2 \times 2 = 4$, ...

Exercício 5.7 Modifique o programa anterior de forma que o usuário também digite o início e o fim da tabuada, em vez de começar com 1 e 10.

Exercício 5.8 Escreva um programa que leia dois números. Imprima o resultado da multiplicação do primeiro pelo segundo. Utilize apenas os operadores de soma e subtração para calcular o resultado. Lembre-se de que podemos entender a multiplicação de dois números como somas sucessivas de um deles. Assim, $4 \times 5 = 5 + 5 + 5 + 5 = 4 + 4 + 4 + 4 + 4$.

Repetições

- Contadores também podem ser úteis quando usados com condições dentro dos programas.
- + Vejamos um programa para corrigir um teste de múltipla escolha com três questões. A resposta da primeira é “b”; da segunda, “a”; e da terceira, “d”. O programa da listagem 5.10 conta um ponto a cada resposta correta.

Listagem – Contagem de questões corretas

```
pontos = 0
questão = 1
while questão <= 3:
    resposta = input("Resposta da questão %d: " % questão)
    if questão == 1 and resposta == "b":
        pontos = pontos + 1
    if questão == 2 and resposta == "a":
        pontos = pontos + 1
    if questão == 3 and resposta == "d":
        pontos = pontos + 1
    questão += 1
print("O aluno fez %d ponto(s)" % pontos)
```


Repetições

Execute o programa e digite todas as respostas corretas, depois tente com respostas diferentes.

Veja que estamos verificando apenas respostas simples de uma só letra e que consideramos apenas letras minúsculas.

Em Python, uma letra minúscula é diferente de uma maiúscula. Se você digitar “A” na segunda questão em vez de “a”, o programa não considerará essa resposta correta.

Uma solução para esse tipo de problema é utilizar o operador lógico or e verificar a resposta maiúscula e minúscula.

Por exemplo, `questão == 1 and (resposta == "b" or resposta == "B")`

Repetições

Exercício 5.10 Modifique o programa da listagem 5.10 para que aceite respostas com letras maiúsculas e minúsculas em todas as questões.

Embora essa verificação resolva o problema, veremos que, se digitarmos um espaço em branco antes ou depois da resposta, ela também será considerada errada.

Sempre que trabalharmos com strings, esse tipo de problema deve ser controlado. Veremos mais sobre o assunto a frente.

Repetições

Acumuladores

Nem só de contadores precisamos.

Em programas para calcular o total de uma soma, por exemplo, **precisaremos de acumuladores.**

A diferença entre um contador e um acumulador é que nos contadores o valor adicionado é constante e, ***nos acumuladores, variável.***

Vejamos um programa que calcule a soma de 10 números na listagem a seguir.

Repetições

- **Acumuladores**

Listagem 1 – Soma de 10 números

```
n = 1  
soma = 0
```

```
while n <= 10:  
    x = int(input("Digite o %d número:"%n))  
    soma = soma + x ①  
    n = n + 1 ②  
  
print("Soma: %d"%soma)
```

Repetições

Acumuladores

Podemos definir a média aritmética como a soma de vários números divididos pela quantidade de números somados.

Assim, se somarmos três números, 4, 5 e 6, teríamos a média aritmética como $(4+5+6) / 3$, onde 3 é a quantidade de números.

Se chamarmos o primeiro número de $n1$, o segundo de $n2$, e o terceiro de $n3$, teremos $(n1 + n2 + n3) / 3$.

Vejamos um programa que calcula a média de cinco números digitados pelo usuário na próxima listagem.

Repetições

Acumuladores

Se chamarmos o primeiro valor digitado de n1, o segundo de n2, e assim sucessivamente, teremos que:

$$\text{média} = (n1+n2+n3+n4+n5)/5$$

Em vez de utilizarmos cinco variáveis, vamos acumular os valores à medida que são lidos.

Listagem – Cálculo de média com acumulador

```
x = 1
```

```
soma = 0 ❶
```

```
while x <= 5:
```

```
    n = int(input("%d Digite o número:" % x))
```

```
    soma = soma + n ❷
```

```
    x = x + 1
```

```
print("Média: %5.2f" % (soma/5)) ❸
```

Repetições

Acumuladores

Nesse caso, temos x sendo um contador e n o valor digitado pelo usuário. A variável soma é criada em ① e inicializada com 0.

Diferentemente de x , que recebe 1 a cada passagem, a variável soma, em ②, é adicionada do valor digitado pelo usuário. Podemos dizer que o incremento de soma não é um valor constante, pois varia com o valor digitado pelo usuário.

Podemos também dizer que soma acumula os valores de n a cada repetição. Logo, diremos que a variável soma é um acumulador.

Acumuladores são muito interessantes quando não sabemos ou não conseguimos obter o total da soma pela simples multiplicação de dois números. No caso do cálculo da média, o valor de n pode ser diferente cada vez que o usuário digitar um valor.

Repetições

Acumuladores

Exercício 5.11 Escreva um programa que pergunte o depósito inicial e a taxa de juros de uma poupança. Exiba os valores mês a mês para os 24 primeiros meses. Escreva o total ganho com juros no período.

Exercício 5.12 Altere o programa anterior de forma a perguntar também o valor depositado mensalmente. Esse valor será depositado no início de cada mês, e você deve considerá-lo para o cálculo de juros do mês seguinte.

Exercício 5.13 Escreva um programa que pergunte o valor inicial de uma dívida e o juro mensal. Pergunte também o valor mensal que será pago. Imprima o número de meses para que a dívida seja paga, o total pago e o total de juros pago.

Repetições

Interrompendo a repetição

Embora muito útil, a estrutura while só verifica sua condição de parada no início de cada repetição. Dependendo do problema, a habilidade de terminar while dentro do bloco a repetir pode ser interessante.

A instrução break é utilizada para interromper a execução de while independentemente do valor atual de sua condição. Vejamos o exemplo da leitura de valores até que digitemos 0 (zero) no programa da listagem abaixo:

Listagem – Interrompendo a repetição

```
s=0
while True: ❶
    v=int(input("Digite um número a somar ou 0 para sair:"))
    if v==0:
        break ❷
    s = s+v ❸
print(s) ❹
```

Repetições

Interrompendo a repetição

Nesse exemplo, substituímos a condição do while por True em ❶.

Dessa forma, o while executará para sempre, pois o valor de sua condição de parada (True) é constante.

Em ❷ temos a instrução break sendo ativada dentro de um if, especificamente quando v é zero. Porém, enquanto v for diferente de zero, a repetição continuará a somar v a s em ❸.

Quando v for igual a zero (0), teremos ❷ sendo executada, terminando a repetição e transferindo a execução para ❹, que, então, exibe o valor de s na tela

Repetições

Interrompendo a repetição

Exercício 5.14 Escreva um programa que leia números inteiros do teclado. O programa deve ler os números até que o usuário digite 0 (zero). No final da execução, exiba a quantidade de números digitados, assim como a soma e a média aritmética.

Exercício 5.15 Escreva um programa para controlar uma pequena máquina registradora. Você deve solicitar ao usuário que digite o código do produto e a quantidade comprada. Utilize a tabela de códigos abaixo para obter o preço de cada produto:

Código	Preço
1	0,50
2	1,00
3	4,00
5	7,00
9	8,00

Seu programa deve exibir o total das compras depois que o usuário digitar 0. Qualquer outro código deve gerar a mensagem de erro “Código inválido”.

Repetições

Interrompendo a repetição

Vejamos como exemplo um programa que leia um valor e que imprima a quantidade de cédulas necessárias para pagar esse mesmo valor, apresentado na listagem a seguir

Exercício 5.16 Execute o programa (Listagem 5.14) para os seguintes valores: 501, 745, 384, 2, 7 e 1.

Exercício 5.17 O que acontece se digitarmos 0 (zero) no valor a pagar?

Exercício 5.18 Modifique o programa para também trabalhar com notas de R\$ 100.

Exercício 5.19 Modifique o programa para aceitar valores decimais, ou seja, também contar moedas de 0,01, 0,02, 0,05, 0,10 e 0,50.

Para simplificar, vamos trabalhar apenas com valores inteiros e com cédulas de R\$ 50, R\$ 20, R\$ 10, R\$ 5 e R\$ 1

Repetições

Interrompendo a repetição

Listagem – Contagem de cédulas valor=int(input("Digite o valor a pagar:"))

cédulas=0

atual=50

apagar=valor

while True:

 if atual<=apagar:

 apagar-=atual

 cédulas+=1

 else:

 print("%d cédula(s) de R\$%d" % (cédulas, atual))

 if apagar == 0:

 break

 if atual == 50:

 atual = 20

 elif atual == 20:

 atual = 10

 elif atual == 10:

 atual = 5

 elif atual == 5:

 atual = 1

 cédulas = 0

Repetições

Repetições aninhadas

Podemos combinar vários while de forma a obter resultados mais interessantes, como a repetição com incremento de duas variáveis.

Imagine imprimir as tabuadas de multiplicação de 1 a 10. Vejamos como fazer isso, lendo a listagem do programa 5.15.

Listagem – Impressão de tabuadas

```
tabuada=1
```

```
while tabuada <= 10: ❶
```

```
    número = 1 ❷
```

```
    while número <= 10: ❸
```

```
        print("%d x %d = %d" % (tabuada, número, tabuada * número))
```

```
        número+=1 ❹
```

```
    tabuada+=1 ❺
```

Repetições

Repetições aninhadas

Em ❶ temos nosso primeiro while, criado para repetir seu bloco enquanto o valor de tabuada for menor ou igual a 10.

Em ❷ temos a inicialização da variável número dentro do primeiro while. Isso é importante porque precisamos voltar a multiplicar por 1 a cada novo valor da variável tabuada.

Finalmente, em ❸ temos o segundo while com a condição de parada número ≤ 10 . Esse while executará suas repetições dentro do primeiro, ou seja, o ponto de execução passa de ❹ para ❸ enquanto a condição for verdadeira.

Repetições

Repetições aninhadas

Veja que em ④ utilizamos o operador += para representar número = número + 1.

Quando número valer 11, a condição em ③ resultará falsa, e a execução do programa continuará a partir da linha ⑤.

Em ⑤ incrementamos o valor de tabuada e voltamos a ①, onde será verificada a condição do primeiro while.

Como resulta verdadeiro, voltaremos a executar ②, reiniciando a variável número com o valor 1. ② é muito importante para que possamos novamente executar o segundo while, responsável por imprimir a tabuada na tela

Repetições

Repetições aninhadas

Vejam os o mesmo problema, mas sem utilizar repetições aninhadas, como apresenta a listagem:

Listagem – Impressão de tabuadas sem repetições aninhadas

```
tabuada=1
numero=1
while tabuada <= 10:
    print("%d x %d = %d" % (tabuada, numero, tabuada * numero))
    numero+=1
    if numero == 11:
        numero = 1
        tabuada+=1
```

Repetições

Repetições aninhadas

Exercício 5.21 Reescreva o programa da listagem 5.14 de forma a continuar executando até que o valor digitado seja 0. Utilize repetições aninhadas.

Exercício 5.22 Escreva um programa que exiba uma lista de opções (menu): adição, subtração, divisão, multiplicação e sair. Imprima a tabuada da operação escolhida. Repita até que a opção saída seja escolhida.

Exercício 5.23 Escreva um programa que leia um número e verifique se é ou não um número primo. Para fazer essa verificação, calcule o resto da divisão do número por 2 e depois por todos os números ímpares até o número lido. Se o resto de uma dessas divisões for igual a zero, o número não é primo. Observe que 0 e 1 não são primos e que 2 é o único número primo que é par.

Repetições aninhadas

Repetições

Exercício 5.24 Modifique o programa anterior de forma a ler um número n . Imprima os n primeiros números primos.

Exercício 5.25 Escreva um programa que calcule a raiz quadrada de um número. Utilize o método de Newton para obter um resultado aproximado. Sendo n o número a obter a raiz quadrada, considere a base $b=2$. Calcule p usando a fórmula $p=(b+(n/b))/2$. Agora, calcule o quadrado de p . A cada passo, faça $b=p$ e recalcule p usando a fórmula apresentada. Pare quando a diferença absoluta entre n e o quadrado de p for menor que 0,0001.

Exercício 5.26 Escreva um programa que calcule o resto da divisão inteira entre dois números. Utilize apenas as operações de soma e subtração para calcular o resultado.

Exercício 5.27 Escreva um programa que verifique se um número é palíndromo. Um número é palíndromo se continua o mesmo caso seus dígitos sejam invertidos. Exemplos: 454, 10501



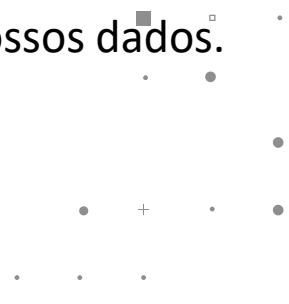
Listas

Listas são um tipo de variável que permite o armazenamento de vários valores, acessados por um índice.

Uma lista pode conter zero ou mais elementos de um mesmo tipo ou de tipos diversos, podendo inclusive conter outras listas.

O tamanho de uma lista é igual à quantidade de elementos que ela contém. Podemos imaginar uma lista como um edifício de apartamentos, onde o térreo é o andar zero, o primeiro andar é o andar 1 e assim por diante.

O índice é utilizado para especificarmos o “apartamento” onde guardaremos nossos dados. Em um prédio de seis andares, teremos números de andar variando entre 0 e 5.



Listas

Se chamarmos nosso prédio de P, teremos P[0] como o endereço do térreo, P[1] como endereço do primeiro andar, continuando assim até P[5].

Em Python, P seria o nome da lista; e o número entre colchetes, o índice. Listas são mais flexíveis que prédios e podem crescer ou diminuir com o tempo. Vejamos como criar uma lista em Python:

Listagem 6.1 – Uma lista vazia

```
L=[]
```

Essa linha cria uma lista chamada L com zero elemento, ou seja, uma lista vazia.

Os colchetes ([]) após o símbolo de igualdade servem para indicar que L é uma lista.

Listas

Vejaamos agora como criar uma lista Z, com 3 elementos

Listagem 6.2 – Uma lista com três elementos

Z=[15, 8, 9]

A lista Z foi criada com três elementos: 15, 8 e 9. Dizemos que o tamanho da lista Z é 3.

Como o primeiro elemento tem índice 0, temos que o último elemento é Z[2]. Veja o resultado de testes com Z:

Listagem 6.3 – Acesso a uma lista

Z=[15,8,9]

Z[0] 15

Z[1] 8

Z[2] 9

Listas

Vejaçs agora como criar uma lista Z, com 3 elementos

Listagem 6.4 – Modificação de uma lista

```
>>> Z=[15,8,9]
```

```
>>> Z[0]
```

```
15
```

```
>>> Z[0]=7
```

```
>>> Z[0]
```

```
7
```

```
>>> Z [7, 8, 9]
```

```
C:\>python
Python 3.11.1 (tags/v3.11.1:a7a450f, Dec 6 2022, 19:58:39) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> z = [15, 8, 9]
>>> z[0]
15
>>> z[0] = 7
>>> z[0]
7
>>> z
[7, 8, 9]
>>>
```

Listas

Quando criamos a lista Z, o primeiro elemento era o número 15.

Por isso, Z[0] era 15. Quando executamos Z[0]=7, alteramos o conteúdo do primeiro elemento para 7. Isso pode ser verificado quando pedimos para exibir Z, agora com 7, 8 e 9 como elementos.

Vejamos um exemplo onde um aluno tem cinco notas e no qual desejamos calcular sua média aritmética. Veja o programa na listagem

Listagem 6.5 – Cálculo da média

```
notas=[6,7,5,8,9] ❶
soma=0
x=0

while x < 5: ❷
    soma += notas[x] ❸
    x += 1 ❹
print("Média: %5.2f" % (soma/x))
```


Listas

Criamos a lista de notas em ❶.

Em ❷, criamos a estrutura de repetição para variar o valor de x e continuar enquanto este for menor que 5. Lembre-se de que uma lista de cinco elementos contém índices de 0 a 4. Por isso inicializamos x=0 na linha anterior.

Em ❸, adicionamos o valor de notas[0] à soma e depois notas[1], notas[2], notas[3] e notas[4], um elemento a cada repetição. Para isso, utilizamos o valor de x como índice e o incrementamos de 1 em ❹.

Uma grande vantagem desse programa foi que não precisamos declarar cinco variáveis para guardar as cinco notas. Todas as notas foram armazenadas na lista, utilizando um índice para identificar ou acessar cada valor.

Vejamos uma modificação desse exemplo, mas, dessa vez, vamos ler as notas uma a uma. O programa modificado é apresentado na listagem a seguir.

Listas

Listagem 6.6 – Cálculo da média com notas digitadas

```
notas=[0,0,0,0,0] ❶
```

```
soma=0
```

```
x=0
```

```
while x < 5:
```

```
    notas[x] = float(input("Nota %d:" % x)) ❷
```

```
    soma += notas[x]
```

```
    x += 1
```

```
X = 0 ❸
```

```
while x < 5: ❹
```

```
    print("Nota %d: %6.2f" % (x, notas[x]))
```

```
    x += 1
```

```
print("Média: %5.2f" % (soma/x))
```

Listas

Em ❶ criamos a lista de notas com cinco elementos, todos zero.

Em ❷, utilizamos a repetição para ler as notas do aluno e armazená-las na lista de notas.

Veja que adicionamos `nota[x]` à soma já na linha seguinte.

Terminada a primeira repetição, teremos a lista de notas preenchidas. Para imprimir a lista de notas, reinicializamos o valor da variável `x` para 0 ❸ e criamos outra estrutura de repetição

❹

Exercício 6.1 Modifique o programa da listagem 6.6 para ler 7 notas em vez de 5.

Listas

Trabalhando com índices

Vejamos outro exemplo: um programa que lê cinco números, armazena-os em uma lista e depois solicita que o usuário escolha um número a mostrar.

O objetivo é, por exemplo, ler 15, 12, 5, 7 e 9 e armazená-los na lista. Depois, se o usuário digitar 2, ele imprimirá o segundo número digitado, 3, o terceiro, e assim sucessivamente.

Observe que o índice do primeiro número é 0, e não 1: essa pequena conversão será feita no programa da próxima listagem:

Listas

Trabalhando com índices

#Listagem 6.7 – Apresentação de números

```
números=[0,0,0,0,0]
```

```
x=0
```

```
while x<5:
```

```
+   números[x]=int(input("Número %d:" % (x+1)))
```

```
    x+=1
```

```
while True:
```

```
    escolhido=int(input("Que posição você quer imprimir (0 para sair): "))
```

```
    if escolhido == 0:
```

```
        break
```

```
print("Você escolheu o número: %d" % (números[escolhido-1]))
```

Listas

Trabalhando com índices

Execute o programa da listagem 6.7 e experimente alguns valores. Observe que em **1** adicionamos 1 a x para que possamos imprimir Número 1...5 e não a partir de 0.

Isso é importante porque começar a contar de 0 não é natural para a maioria das pessoas.

Veja que mesmo imprimindo $x+1$ para o usuário, a atribuição é feita para `números[x]` porque nossas listas começam em 0.

Em **2**, fizemos a operação inversa. Quando o usuário escolhe o número a imprimir, ele faz uma escolha entre 1 e 5. Como 1 é o elemento 0, 2, o elemento 1, e assim por diante.

Diminuímos o valor da escolha de um para obtermos o índice de notas

6.2 Cópia e fatiamento de listas

Embora listas em Python sejam um recurso muito poderoso, todo poder traz responsabilidades.

Um dos efeitos colaterais de listas aparece quando tentamos fazer cópias.

Vejamos um teste no interpretador na listagem 6.8

Listas

Listagem 6.8 – Tentativa de copiar listas

```
>>> L = [1,2,3,4,5]
>>> V = L
>>> L
[1, 2, 3, 4, 5]
>>> V
[1, 2, 3, 4, 5]
>>> V[0] = 6
>>> V
[6, 2, 3, 4, 5]
>>> L
[6, 2, 3, 4, 5]
```

Listas

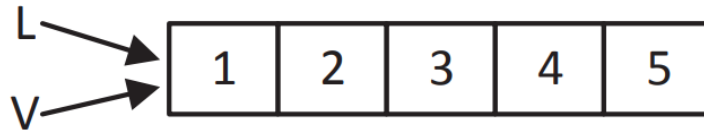
6.2 Cópia e fatiamento de listas

Veja que, ao modificarmos V, modificamos também o conteúdo de L.

Isso porque uma lista em Python é um objeto e, quando atribuímos um objeto a outro, estamos apenas copiando a mesma referência da lista, e não seus dados em si.

Nesse caso, V funciona como um apelido de L, ou seja, V e L são a mesma lista.

Vejamos o que acontece no gráfico da figura



Listas

6.2 Cópia e fatiamento de listas

Quando modificamos `V[0]`, estamos modificando o mesmo valor de `L[0]`, pois ambos são referências, ou apelidos para a mesma lista na memória.

Dependendo da aplicação, esse efeito pode ser desejado ou não.

Para criar uma cópia independente de uma lista, utilizaremos outra sintaxe.

Vejamos o resultado das operações da listagem 6.9.

#Listagem 6.9 – Cópia de listas

```
>>> L = [1,2,3,4,5]
```

```
>>> V = L[:]
```

```
>>> V[0] = 6
```

```
>>> L  
[1, 2, 3, 4, 5]
```

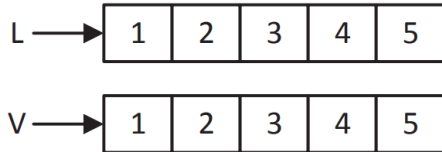
```
>>> V  
[6, 2, 3, 4, 5]
```

Listas

6.2 Cópia e fatiamento de listas

Ao escrevermos `L[:]`, estamos nos referindo a uma nova cópia de `L`.

Assim `L` e `V` se referem a áreas diferentes na memória, permitindo alterá-las de forma independente, como na figura:



Podemos também fatiar uma lista, da mesma forma que fizemos com strings anteriormente. Vejamos alguns exemplos no interpretador na listagem 6.10

#Listagem 6.10 – Fatiamento de listas

```
>>> L = [1,2,3,4,5]
```

```
>>> L[0:5]
```

```
[1, 2, 3, 4, 5]
```

```
>>> L[:5]
```

```
[1, 2, 3, 4, 5]
```

```
>>> L[:-1]
```

```
[1, 2, 3, 4]
```

```
>>> L[1:3]
```

```
[2, 3]
```

```
>>> L[1:4]
```

```
[2, 3, 4]
```

```
>>> L[3:]
```

```
[4, 5]
```

```
>>> L[:3]
```

```
[1, 2, 3]
```

```
>>> L[-1]
```

```
5
```

Listas

6.2 Cópia e fatiamento de listas

Veja que índices negativos também funcionam.

Um índice negativo **começa a contar do último elemento**, mas observe que começamos de **+1**.

Assim $L[0]$ representa o primeiro elemento;

$L[-1]$, o último;

$L[-2]$, o penúltimo,

e assim por diante

Listas

6.3 Tamanho de listas

Podemos usar a função `len()` com listas.

O valor retornado é igual ao número de elementos da lista.

Veja alguns testes na listagem 6.11.

#Listagem 6.11 – Tamanho de listas

```
>>> L = [12,9,5]
```

```
>>> len(L)
```

```
3
```

```
>>> V = []
```

```
>>> len(V)
```

```
0
```

A função `len` pode ser utilizada em repetições para controlar o limite dos índices,

Listas

6.4 Adição de elementos

Uma das principais vantagens de trabalharmos com listas é poder adicionar novos elementos durante a execução do programa.

Vejamos um teste no interpretador (Listagem 6.14). Para adicionar um elemento ao fim da lista, utilizaremos o método `append`.

Em Python, chamamos um método escrevendo o nome dele após o nome do objeto. Como listas são objetos, sendo `L` a lista, teremos `L.append(valor)`.

Métodos são recursos de orientação a objetos, suportados e muito usados em Python: Você pode imaginar um método como uma função do objeto. Quando invocado, ele já sabe a que objeto estamos nos referindo, pois o informamos à esquerda do ponto

Listas

6.4 Adição de elementos

#Listagem 6.14 – Adição de elementos à lista

```
>>> L = []
```

```
>>> L.append("a")
```

```
>>> L
```

```
['a']
```

```
>>> L.append("b")
```

```
>>> L
```

```
['a', 'b']
```

```
>>> L.append("c")
```

```
>>>
```

```
L ['a', 'b', 'c']
```

```
>>> len(L)
```

```
3
```

Listas

6.4 Adição de elementos

Vejam os um programa que lê números até que 0 seja digitado. Esse programa depois os imprimirá na mesma ordem em que foram digitados (Listagem 6.15)

#Listagem 6.15 – Adição de elementos à lista

```
L = []
```

```
while True:
```

```
    n = int(input("Digite um número (0 sai):"))
```

```
    if n == 0:
```

```
        break
```

```
    L.append(n)
```

```
X = 0
```

```
while x < len(L):
```

```
    print(L[x])
```

```
x=x+1
```

Listas

6.4 Adição de elementos

Esse simples programa é capaz de ler e imprimir um número inicialmente indeterminado de valores. Isso é possível porque adicionamos elementos à lista `L`, conforme necessário. Outra forma de adicionarmos elementos a uma lista é com adição de listas (Listagem 6.16).

Quando adicionamos apenas um elemento, tanto `L.append(1)` quanto `L+[1]` produzem o mesmo resultado.

Perceba que em `L+[1]` escrevemos o elemento a adicionar dentro de uma lista (`[1]`), e que, em `append`, apenas `1`.

Isso porque, quando adicionamos uma lista a outra, o interpretador executa um método chamado `extend` que adiciona os elementos de uma lista a outra. Vejamos alguns exemplos na listagem 6.17.

6.4 Adição de elementos

#Listagem 6.16 – Adição de listas

```
>>> L = []
>>> L = L+[1]
>>> L
[1]
>>> L += [2]
>>> L
[1, 2]
>>> L += [3,4,5]
>>> L
[1, 2, 3, 4, 5]
```

Listas

#Listagem 6.17 – Adição de elementos e listas

```
>>> L = ["a"]
>>> L.append("b")
>>> L
['a', 'b']
>>> L.extend(["c"])
>>> L
['a', 'b', 'c']
>>> L.append(["d","e"])
>>> L
['a', 'b', 'c', ['d', 'e']]
>>> L.extend(["f","g","h"])
>>> L
['a', 'b', 'c', ['d', 'e'], 'f', 'g', 'h']
```

6.4 Adição de elementos

O método `extend` sequer aceita parâmetros que não sejam listas.

Se você utilizar o método `append` com uma lista como parâmetro, **em vez de adicionar os elementos no fim da lista, `append` adicionará a lista inteira**, mas como apenas um novo elemento.

Teremos então listas dentro de listas (Listagem 6.18)

Listas

Listagem 6.18 – Adição de elementos e listas com `append`

```
>>> L = ["a"]
>>> L.append(["b"])
>>> L.append(["c","d"])
>>> len(L)
3
>>> L[1]
['b']
>>> L[2]
['c', 'd']
>>> len(L[2])
2
>>> L[2][1]
'd'
```

Listas

6.4 Adição de elementos

Esse conceito é interessante, pois permite a utilização de estruturas de dados mais complexas, como matrizes, árvores e registros.

Por enquanto, vamos utilizar esse recurso para armazenar múltiplos valores por elemento.

Exercício 6.2 Faça um programa que leia duas listas e que gere uma terceira com os elementos das duas primeiras.

Exercício 6.3 Faça um programa que percorra duas listas e gere uma terceira sem elementos repetidos. 6.5 Remoção de elementos

Listas

6.5 Remoção de elementos da lista

Como o tamanho da lista pode variar, permitindo a adição de novos elementos, podemos também retirar alguns elementos da lista, ou mesmo todos eles.

Para isso, utilizaremos a instrução `del` (Listagem 6.19).

#Listagem 6.19 – Remoção de elementos

```
>>> L = ["a","b","c"]
```

```
>>> del L[1]
```

```
>>> L
```

```
['a', 'c']
```

```
>>> del L[0]
```

```
>>> L
```

```
['c']
```

Listas

6.5 Remoção de elementos da lista

É importante notar que o elemento excluído não ocupa mais lugar na lista, **fazendo com que os índices sejam reorganizados**, ou melhor, que passem a ser calculados sem esse elemento.

Podemos também apagar fatias inteiras de uma só vez (Listagem 6.20).

#Listagem 6.20 – Remoção de fatias

```
>>> L = list(range(101))
```

```
>>> del L[1:99]
```

```
>>> L
```

```
[0, 99, 100]
```

Listas

6.6 Usando listas como filas

Uma lista pode ser utilizada como fila se obedecermos a certas **regras de inclusão e eliminação de elementos**.

Em uma fila, **a inclusão é sempre realizada no fim**, e as remoções **são feitas no início**. Dizemos que o primeiro a chegar é o primeiro a sair (FIFO - First In First Out).

É mais simples de entender se imaginarmos uma fila de banco. Quando a agência abre pela manhã, a fila está vazia. Quando os clientes começam a chegar, eles vão diretamente para o fim da fila.

Os caixas então começam a atender esses clientes por ordem de chegada, ou seja, o cliente que chegou primeiro será atendido primeiro. Uma vez que o cliente é atendido, ele sai da fila.

Listas

6.6 Usando listas como filas

Então, um novo cliente passa a ser o primeiro da fila e o próximo a ser atendido. Para escrevermos algo similar em Python, imaginaremos uma lista de clientes, representando a fila, onde o valor de cada elemento é igual à ordem de chegada do cliente.

Vamos imaginar uma lista inicial com 10 clientes. Se outro cliente chegar, realizaremos um `append()` para que ele seja inserido no fim da fila (`fila.append(último)`). Para retirarmos um cliente da fila e atendê-lo, poderíamos fazer `del fila[0]`, **porém, isso apagaria o cliente da fila.**

Se quisermos retirá-lo da fila e, ao mesmo tempo, obter o elemento retirado, podemos utilizar o **método pop** `fila.pop(0)`. O método `pop` retorna o valor do elemento e o exclui da fila. Passamos 0 como parâmetro para indicar que queremos excluir o primeiro elemento. Veja o programa completo na listagem 6.21

Listas

6.6 Usando listas como filas

#Listagem 6.21 – Simulação de uma fila de banco

último = 10

fila = list(range(1,último+1))

while True:

print("\nExistem %d clientes na fila" % len(fila))

print("Fila atual:", fila)

print("Digite F para adicionar um cliente ao fim da fila,")

print("ou A para realizar o atendimento. S para sair.")

operação = input("Operação (F, A ou S):")

if operação == "A":

if(len(fila))>0:

atendido = fila.pop(0)

print("Cliente %d atendido" % atendido)

else:

print("Fila vazia! Ninguém para atender.")

elif operação == "F":

último += 1 # Incrementa o ticket do novo cliente

fila.append(último)

elif operação == "S":

break

else:

print("Operação inválida! Digite apenas F, A ou S!")

Listas

6.7 Uso de listas como pilhas

Uma pilha tem uma política de acesso bem definida: novos elementos são adicionados ao topo. A retirada de elementos também é feita pelo topo.

Imagine uma pilha de pratos para lavar. Retiramos o prato que está no topo da pilha para lavar e, se mais alguns pratos chegarem, serão também adicionados ou empilhados ao topo, ou seja, um sobre o outro.

Na pilha, o último elemento a chegar é o primeiro a sair (LIFO - Last In First Out). Por enquanto, vamos nos concentrar nas pilhas de prato.

Veja o programa da listagem 6.22, que simula uma pia de cozinha cheia de pratos.

Listas

6.7 Uso de listas como pilhas

Você deve ter percebido que o exemplo da listagem 6.22 é muito parecido com o programa da listagem 6.21.

Isso porque a grande diferença entre pilhas e filas é o elemento que escolhemos para retirar.

Em uma fila, o primeiro elemento é retirado primeiro. Já em pilhas, retira-se a partir do último elemento.

A única mudança em Python é o valor que passamos para o método pop.

No caso de uma pilha, como retiramos o último elemento, passamos -1 a pop

6.7 Uso de listas como pilhas

Listas

#Listagem 6.22 – Pilha de pratos

prato = 5

pilha = list(range(1,prato+1))

while True:

print("\nExistem %d pratos na pilha" % len(pilha))

print("Pilha atual:", pilha)

print("Digite E para empilhar um novo prato,")

print("ou D para desempilhar. S para sair.")

operação = input("Operação (E, D ou S):")

if operação == "D": if(len(pilha))>0:

lavado = pilha.pop(-1)

print("Prato %d lavado" % lavado)

else:

print("Pilha vazia! Nada para lavar.")

elif operação == "E":

prato+=1 # Novo prato

pilha.append(prato)

elif operação == "S":

break

else:

print("Operação inválida! Digite apenas E, D ou S!")

Listas

6.7 Uso de listas como pilhas

Você pode também observar o uso de pilhas com seu browser de internet.

Abra algumas páginas, clicando em seus links. Observe como o histórico de navegação é modificado.

Cada novo link adiciona uma página a seu histórico de navegação. Se você clicar em voltar uma página, o browser utilizará a última página que entrou no histórico. Nesse caso, o histórico do browser funciona como uma pilha.

Na realidade, ele é um pouco mais complexo, pois permite que voltemos várias vezes e que depois possamos voltar no outro sentido se necessário (avançar).

Listas

6.7 Uso de listas como pilhas

Agora faça outro teste:

volte duas ou três vezes e depois visite um novo link. Dessa vez, o browser deve ter desativado a função de avanço, uma vez que o histórico mudou (você mudou de direção).

Tente entender esse processo como duas pilhas, uma à esquerda e outra à direita. Quando você escolhe voltar, desempilhamos um elemento da pilha à esquerda. Ao visitar um novo link, adicionamos um novo elemento a essa mesma pilha.

Para simular a opção de avançar, imagine que, ao retirarmos um elemento da pilha à esquerda, devemos adicioná-lo à da direita.

Se escolhermos um endereço novo, apagamos a pilha da direita

Listas

6.7 Uso de listas como pilhas

Essas mesmas operações podem ser simuladas com outras estruturas, como uma lista simples e uma variável contendo onde estamos no histórico de navegação.

A cada novo endereço, adicionaríamos um novo elemento à lista e atualizaríamos nossa posição.

Ao voltarmos, decrementaríamos a posição e a incrementaríamos no caso de avanço. Se mudássemos de direção, bastaria apagar os elementos entre a posição atual e o fim da lista antes de adicionar o novo elemento.

A importância de aprender a manipular listas como filas ou pilhas é entender algoritmos mais complexos no futuro.

Listas

6.7 Uso de listas como pilhas

Exercício 6.7 Faça um programa que leia uma expressão com parênteses. Usando pilhas, verifique se os parênteses foram abertos e fechados na ordem correta. Exemplo:

(())	OK
()()()()()	OK
()	Erro

Você pode adicionar elementos à pilha sempre que encontrar abre parênteses e desempilhá-la a cada fecha parênteses.

Ao desempilhar, verifique se o topo da pilha é um abre parênteses. Se a expressão estiver correta, sua pilha estará vazia no final

6.8 Pesquisa

Podemos pesquisar se um elemento está ou não em uma lista, verificando do primeiro ao último elemento se o valor procurado estiver presente.

Vejamos a listagem 6.23.

Listagem 6.23 – Pesquisa sequencial

```
L = [15,7,27,39]
p = int(input("Digite o valor a procurar:"))
achou = False ❶
x = 0
while x < len(L):
    if L[x] == p
        achou = True ❷
        break ❸
    x += 1
if achou: ❹
    print("%d achado na posição %d" % (p,x))
else:
    print("%d não encontrado" % p)
```


6.8 Pesquisa

A pesquisa simplesmente compara todos os elementos da lista com o valor procurado, interrompendo a repetição ao encontrar o primeiro elemento cujo valor é igual ao procurado.

É importante saber que podemos ou não encontrar o que procuramos, daí a utilização da variável `achou` em ❶.

Essa variável do tipo lógico (booleano) será utilizada para verificar se saímos da repetição por termos achado o que procurávamos ou simplesmente porque visitamos todos os elementos sem encontrar o valor procurado. Veja que `achou` é marcada como `True` em ❷, mas dentro de `if` com a condição de pesquisa, e antes do `break` em ❸.

Dessa forma, `achou` só será `True` se algum elemento for igual ao valor procurado. Em ❹, verificamos o valor de `achou` para decidir o que vamos imprimir.

6.8 Pesquisa

Exercício 6.8 Modifique o primeiro exemplo (Listagem 6.23) de forma a realizar a mesma tarefa, mas sem utilizar a variável `achou`. Dica: observe a condição de saída do `while`.

Exercício 6.9 Modifique o exemplo para pesquisar dois valores. Em vez de apenas `p`, leia outro valor `v` que também será procurado. Na impressão, indique qual dos dois valores foi achado primeiro.

Exercício 6.10 Modifique o programa do exercício 6.9 de forma a pesquisar `p` e `v` em toda a lista e informando o usuário a posição onde `p` e a posição onde `v` foram encontrados.

6.9 Usando for

Python apresenta uma estrutura de repetição especialmente projetada para percorrer listas.

A instrução **for** funciona de forma parecida a **while**, mas a cada repetição utiliza um elemento diferente da lista.

A cada repetição, o próximo elemento da lista é utilizado, o que se repete até o fim da lista. Vamos escrever um programa que utilize for para imprimir todos os elementos de uma lista (Listagem 6.24).

Listagem 6.24 – Impressão de todos os elementos da lista com for

```
L=[8,9,15]
```

```
for e in L: ①
```

```
    print(e) ②
```

6.9 Usando for

Quando começamos a executar o for em ❶, temos e igual ao primeiro elemento da lista, no caso, 8, ou seja, L[0]. Em ❷ imprimimos 8, e a execução do programa volta para ❶, onde e passa a valer 9, ou seja, L[1]. Na próxima repetição e valerá 15, ou seja, L[2].

Depois de imprimir o último número, a repetição é concluída, pois não temos mais elementos a substituir. Se tivéssemos que fazer a mesma tarefa com while, teríamos que escrever um programa como o da listagem 6.25.

Listagem 6.25 – Impressão de todos os elementos da lista com while

```
L=[8,9,15]
```

```
x=0
```

```
while x < len(L):
```

```
    e=L[x]
```

```
    print(e)
```

```
    x+=1
```

6.9 Usando for

Embora a instrução **for** facilite nosso trabalho, ela não substitui completamente **while**. Dependendo do problema, utilizaremos **for** ou **while**. Normalmente utilizaremos **for** quando quisermos processar os elementos de uma lista, um a um.

while é indicado para repetições nas quais não sabemos ainda quantas vezes vamos repetir ou onde manipulamos os índices de forma não sequencial. Vale lembrar que a instrução **break** também interrompe o **for**. Vejamos a pesquisa, escrita usando **for**, na listagem 6.26.

Listagem 6.26 – Pesquisa usando for

```
L = [7,9,10,12]
P = int(input("Digite um número a
pesquisar:"))
for e in L:
    if e == p:
        print("Elemento encontrado!")
        break ①
    else: ②
        print("Elemento não encontrado.")
```

6.9 Usando for

Utilizamos a instrução `break` para interromper a busca depois de encontrarmos o primeiro elemento em ❶.

Em ❷ utilizamos um `else`, parecido com o da instrução `if`, para imprimir a mensagem informando que o elemento não foi encontrado.

O `else` deve ser escrito na mesma coluna de `for` e só será executado se todos os elementos da lista forem visitados, ou seja, se não utilizarmos a instrução `break`, deixando `for` terminar normalmente

Exercício 6.11 Modifique o programa da listagem 6.15 usando `for`. Explique por que nem todos os `while` podem ser transformados em `for`.

6.10 Range

Podemos utilizar a função range para gerar listas simples. A função range não retorna uma lista propriamente dita, mas um gerador ou generator. Por enquanto, basta entender como podemos usá-la. Imagine um programa simples que imprime de 0 a 9 na tela (Listagem 6.27).

#Listagem 6.27 – Uso da função range

```
for v in range(10):  
    print(v)
```

Execute o programa e veja o resultado. A função range gerou números de 0 a 9 porque passamos 10 como parâmetro.

Ela normalmente gera valores a partir de 0, logo, ao especificarmos apenas 10, estamos apenas informando onde parar. Experimente mudar de 10 para 20. O programa deve imprimir de 0 a 19.

6.10 Range

Agora experimente 20000. A vantagem de utilizarmos a função range é gerar listas eficientemente, como mostrado no exemplo, sem precisar escrever os 20.000 valores no programa.

Com a mesma função range, podemos também indicar qual é o primeiro número a gerar. Para isso, utilizaremos dois parâmetros: início e fim (Listagem 6.28).

#Listagem 6.28 – Uso da função range com intervalos

```
for v in range(5, 8):  
    print(v)
```

Usando 5 como início e 8 como fim, vamos imprimir os números 5, 6 e 7.

A notação aqui para o fim é a mesma utilizada com fatias, ou seja, o fim é um intervalo aberto, isto é, não incluso na faixa de valores.

6.10 Range

Se acrescentarmos um terceiro parâmetro à função range, teremos como saltar entre os valores gerados, por exemplo, range(0,10,2) gera os pares entre 0 e 10, pois começa de 0 e adiciona 2 a cada elemento.

Vejamos um exemplo onde geramos os 10 primeiros múltiplos de 3 (Listagem 6.29).

#Listagem 6.29 – Uso da função range com saltos

```
for t in range(3,33,3):  
    print(t, end=" ")  
    print()
```

6.11 Enumerate

- Com a função enumerate podemos ampliar as funcionalidades de for facilmente. Vejamos como imprimir uma lista, onde teremos o índice entre colchetes e o valor à sua direita (Listagem 6.31).

#Listagem 6.31 – Impressão de índices sem usar a função enumerate

```
L=[5,9,13]
```

```
x=0
```

```
for e in L:
```

```
    print("[%d] %d" % (x,e))
```

```
    x+=1
```

Veja o mesmo programa, mas utilizando a função enumerate na listagem (Listagem 6.32).

#Listagem 6.32 – Impressão de índices usando a função enumerate

```
L=[5,9,13] for x, e in enumerate(L):
```

```
    print("[%d] %d" % (x,e))
```

6.11 Enumerate

- A função enumerate gera uma tupla em que o primeiro valor é o índice e o segundo é o elemento da lista sendo enumerada.

Ao utilizarmos `x, e` em `for`, indicamos que o primeiro valor da tupla deve ser colocado em `x`, e o segundo, em `e`.

Assim, na primeira iteração teremos a tupla `(0,5)`, onde `x=0` e `e=5`. Isso é possível porque Python permite o desempacotamento de valores de uma tupla, atribuindo um elemento da tupla a cada variável em `for`. O que temos a cada iteração de `for` é equivalente a `x, e = (0,5)`, em que o gerador `enumerate` retorna cada vez uma nova tupla.

Os próximos valores retornados são `(1,9)` e `(2,13)`, respectivamente. Experimente substituir `x,e` na listagem 6.30 por `z`. Antes de `print`, faça `x,e = z`. Adicione mais um `print` para exibir também o valor de `z`.

6.12 Operações com listas

- Podemos percorrer uma lista de forma a verificar o menor e o maior valor (Listagem 6.33)

#Listagem 6.33 – Verificação do maior valor

```
L=[1,7,2,4] máximo=L[0] ❶
```

```
for e in L:
```

```
    if e > máximo:
```

```
        máximo = e
```

```
print(máximo)
```

Em ❶, utilizamos um pequeno truque, inicializando o máximo com o valor do primeiro elemento.

Precisamos de um valor para máximo antes de utilizá-lo na comparação com if. Se usássemos 0, não teríamos problema, desde que nossa lista não tenha valores negativos.

6.12 Operações com listas

Exercício 6.12

- Altere o programa da listagem 6.33 de forma a imprimir o menor elemento da lista.

Exercício 6.13

A lista de temperaturas de Mons, na Bélgica, foi armazenada na lista $T = [-10, -8, 0, 1, 2, 5, -2, -4]$. Faça um programa que imprima a menor e a maior temperatura, assim como a temperatura média

• 6.13 Aplicações

- Vejamos uma situação na qual temos que selecionar os elementos de uma lista de forma a copiá-los para outras duas listas. Para simplificar o problema, imagine que os valores estejam inicialmente na lista V, mas que devam ser copiados para a P, se forem pares; ou para a I, se forem ímpares. Veja o programa que resolve esse problema, na listagem 6.34.

| #Listagem 6.34 – Cópia de elementos para outras listas

+ V=[9,8,7,12,0,13,21]

P=[]

I=[]

for e in V:

 if e % 2 == 0:

 P.append(e)

 else:

 I.append(e)

print("Pares: ", P)

print("Ímpares: ", I)

• 6.13 Aplicações

- Vejamos agora um programa que controla a utilização das salas de um cinema.

Imagine que a lista Salas = [10,2,1,3,0] contenha o número de lugares vagos nas salas 1, 2, 3, 4 e 5, respectivamente.

Esse programa lerá o número da sala e a quantidade de lugares solicitados.

Ele deve informar se é possível vender o número de lugares solicitados, ou seja, se ainda há lugares livres.

Caso seja possível vender os bilhetes, atualizará o número de lugares livres.

A saída ocorre quando se digita 0 no número da sala (Listagem 6.35)

• 6.13 Aplicações

• #Listagem 6.35 – Controle da utilização de salas de um cinema

```
+ lugares_vagos = [10,2,1,3,0]
while True:
    sala = int(input("Sala (0 sai): "))
    if sala == 0:
        print("Fim")
        break
    if sala > len(lugares_vagos) or sala<1:
        print("Sala inválida")
    elif lugares_vagos[sala-1]==0:
        print("Desculpe, sala lotada!")
    else:
        lugares = int(input("Quantos lugares você deseja (%d vagos):"
        % lugares_vagos[sala-1]))
```

#continuação vermelho está repetido...

else:

```
        lugares =int(input("Quantos lugares você
deseja (%d vagos):"
        % lugares_vagos[sala-1]))

        if lugares > lugares_vagos[sala-1]:
            print("Esse número de lugares não está
disponível.")
        elif lugares < 0:
            print("Número inválido")
        else:
            lugares_vagos[sala-1]-=lugares
            print("%d lugares vendidos" % lugares)
        print("Utilização das salas")
        for x,l in enumerate(lugares_vagos):
            print("Sala %d - %d lugar(es) vaziao(s)" % (x+1, l))
```


• 6.14 Listas com strings +

- O programa da listagem 6.37 lê e imprime uma lista de compras até que seja digitado fim.

+ •
#Listagem 6.37 – Lendo e imprimindo uma lista de compras

```
compras=[]  
| while True:  
+     produto = input("Produto:")  
    if produto == "fim":  
        break  
    compras.append(produto)  
for p in compras:  
    print(p)
```

• 6.15 Listas dentro de listas

- Um fator interessante é que podemos acessar as strings dentro da lista, letra por letra, usando um segundo índice. Vejamos as listagens 6.38 e 6.39

#Listagem 6.38 – Listas com strings, acessando letras

```
>>> S=["maçãs", "peras", "kiwis"]
```

```
>>> print(S[0][0])
```

m

```
>>> print(S[0][1])
```

a

```
>>> print(S[1][1])
```

e

```
>>> print(S[2][2])
```

w

#Listagem 6.39 – Impressão de uma lista de strings, letra a letra

```
L=["maçãs", "peras", "kiwis"]
```

```
for s in L:
```

```
    for letra in s:
```

```
        print(letra)
```

• 6.15 Listas dentro de listas

- Isso nos leva a outra vantagem das listas em Python: listas dentro de listas.
- + Como bônus, temos também que os elementos de uma lista não precisam ser do mesmo tipo.

Vejamos um exemplo onde teríamos uma lista de compras na listagem 6.40.

O primeiro elemento seria o nome do produto; o segundo, a quantidade; e o terceiro, seu preço

#Listagem 6.40 – Listas com elementos de tipos diferentes

```
produto1 = [ "maçã", 10, 0.30]
```

```
produto2 = [ "pera", 5, 0.75]
```

```
produto3 = [ "kiwi", 4, 0.98]
```

Assim, produto1, produto2, produto3 seriam três listas com três elementos cada uma.

Observe que o primeiro elemento é do tipo string; o segundo, do tipo inteiro; e o terceiro, do tipo ponto flutuante (float)!

• • • • •

 $+$ $\bullet$

+

```
produto2 = [ "pera", 5, 0.75]
```

```
produto3 = [ "kiwi", 4, 0.98]
```

```
compras = [ produto1, produto2, produto3]
```

```
print(compras)
```

#Listagem 6.42 – Impressão das compras

```
produto1 = [ "maçã", 10, 0.30]
```

```
produto2 = [ "pera", 5, 0.75]
```

```
produto3 = [ "kiwi", 4, 0.98]
```

```
compras = [ produto1, produto2, produto3]
```

```
for e in compras:
```

```
print("Produto: %s" % e[0])
```

```
print("Quantidade: %d" % e[1])
```

```
print("Preco: %5.2f" % e[2])
```

• 6.15 Listas dentro de listas

• Da mesma forma, poderíamos ter acessado o preço do segundo elemento com `compras[1][2]`. Vejamos agora um programa completo, capaz de perguntar nome do produto, quantidade e preço e, no final, imprimir uma lista de compras completa

#Listagem 6.43 – Criação e impressão da lista de compras

```
compras = [ ]
```

```
while True:
```

```
    produto = input("Produto: ")
```

```
    if produto == "fim":
```

```
        break
```

```
    quantidade = int(input("Quantidade: "))
```

```
    preço = float(input("Preço: "))
```

```
    compras.append([produto, quantidade, preço])
```

```
soma = 0.0
```

```
for e in compras:
```

```
    print("%20s x%5d %5.2f %6.2f" % (e[0], e[1], e[2], e[1] * e[2]))
```

```
    soma += e[1] * e[2]
```

```
print("Total: %7.2f" % soma)
```

• 6.16 Ordenação

- Até agora, os elementos de nossas listas apresentam a mesma ordem em que foram digitados, sem qualquer ordenação. Para ordenar uma lista, realizaremos uma operação semelhante à da pesquisa, mas trocando a ordem dos elementos quando necessário.

Um algoritmo muito simples de ordenação é o “Bubble Sort”, ou método de bolhas, fácil de entender e aprender. Por ser lento, você não deve utilizá-lo com listas grandes. A ordenação pelo método de bolhas consiste em comparar dois elementos a cada vez.

Se o valor do primeiro elemento for maior que o do segundo, eles trocarão de posição. Essa operação é então repetida para o próximo elemento até o fim da lista. O método de bolhas exige que percorramos a lista várias vezes. Por isso, utilizaremos um marcador para saber se chegamos ao fim da lista trocando ou não algum elemento.

Esse método tem outra propriedade, que é posicionar o maior elemento na última posição da lista, ou seja, sua posição correta. Isso permite eliminar um elemento do fim da lista a cada passagem completa. Vejamos o programa da listagem 6.44.

• 6.16 Ordenação

• #Listagem 6.44 – Ordenação pelo método de bolhas

• L=[7,4,3,12,8]

+ fim = 5 ①

while fim > 1: ②

trocou=False ③

x=0 ④

while x < (fim-1): ⑤

if L[x] > L[x+1]: ⑥

trocou = True ⑦

temp = L[x] ⑧

L[x] = L[x+1]

L[x+1] = temp

x += 1

if not trocou: ⑨

break

fim -= 1 ⑩

for e in L:

print(e)

• 6.16 Ordenação

•
• Execute o programa e tente entender como ele funciona.

+ Em ❶, utilizamos a variável fim para marcar a quantidade de elementos da lista. Precisamos marcar o fim ou a posição do último elemento porque no “Bubble Sort” não precisamos verificar o último elemento após uma passagem completa.

| Em ❷, verificamos se $\text{fim} > 1$, pois como comparamos o elemento atual($L[x]$) com o seguinte($L[x+1]$), precisamos de, no mínimo, dois elementos.

+ Utilizamos a variável trocou em ❸ para indicar que não realizamos nenhuma troca. O valor de trocou será utilizado mais tarde para verificar se já temos a lista ordenada ou não. A variável x ❹ será utilizada como índice, começando da posição 0.

Nossa condição do segundo while ❺ é especial, pois temos que garantir um próximo elemento para comparar com o atual. Isso faz com que a condição de saída desse while seja x while seja $x < (\text{fim} - 1)$, ou melhor, que x seja anterior ao último elemento. Em ❻ temos a comparação em si, onde x é nossa posição atual, e o próximo elemento é o de índice $x+1$.

• 6.16 Ordenação

- Como estamos ordenando em ordem crescente, desejamos que o próximo elemento sempre seja maior que o atual, dessa forma garantindo a ordenação da lista.

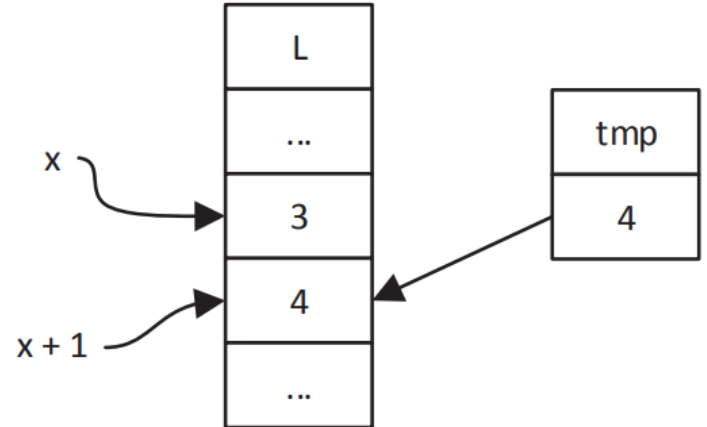
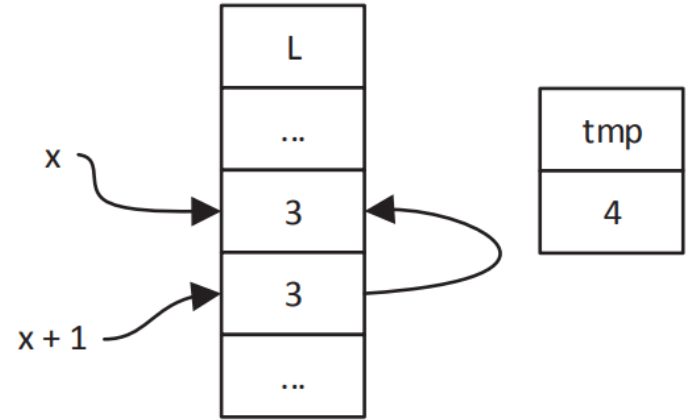
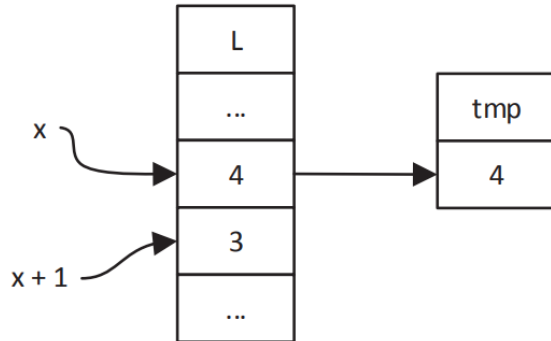
Como comparamos apenas dois elementos de cada vez, temos que visitar a lista várias vezes, até que todos os elementos estejam em suas posições. Se a condição de ⑥ for verdadeira, esses elementos estão fora de ordem. Nesse caso, devemos inverter ou trocar a posição dos dois elementos.

Em ⑦ marcamos que efetuamos uma troca e, em ⑧, utilizamos uma variável auxiliar ou temporária para trocar os dois valores de posição.

6.16 Ordenação

A troca de valores entre duas variáveis é mostrada nas figuras 6.3, 6.4 e 6.5.

Como cada variável só guarda um valor de cada vez, utilizaremos uma terceira variável temporária (temp) para armazenar o valor de uma delas durante a troca



6.16 Ordenação

Você pode entender melhor o processo de troca de valores utilizando um problema do dia a dia: imagine que você tenha duas xícaras, uma com leite e outra com café. O problema consiste em passar o leite para a xícara que contém o café, e o café para a xícara que contém o leite, ou seja, você deve trocar o conteúdo das xícaras.

Nesse caso, resolveríamos o problema utilizando uma terceira xícara para facilitar a operação. Fizemos o mesmo durante a ordenação, onde chamamos nossa terceira variável de temp. Quando a repetição iniciada em ⑤ termina, o maior elemento está posicionado na última posição da lista, ou seja, em sua posição correta. Em ⑨, verificamos se algo foi trocado na repetição anterior.

Se não trocamos nada de lugar, nossa lista está ordenada, e não precisamos executar a repetição outra vez, por isso o break. Caso contrário, como a última posição já está com o elemento correto, diminuiremos o valor de fim para que não precisemos mais verificá-lo.

6.16 Ordenação

+ Você pode entender melhor o processo de troca de valores utilizando um problema do dia a dia. Vamos rastrear o algoritmo e observar como tudo isso funciona. Observe o rastreamento completo na tabela 6.1. A coluna linha indica a linha do programa sendo executada: não confunda com os números dentro dos círculos brancos da listagem

Tabela 6.1 – Rastreamento da ordenação com Bubble Sort

[illegible][illegible]

Match Case – Estruturas de escolha

Vamos aprender como usar o Python para criar uma instrução switch-case.

Esse tipo de estrutura é muito comum em outras linguagens também e são utilizadas como uma outra solução para trabalharmos com um menu....

Elas são chamadas estruturas de seleção múltipla

A estrutura de Seleção Múltipla disponibiliza outro meio para realizar desvios em um algoritmo

É indicada para testar o valor de uma variável em relação a diversos valores pré-estabelecidos

Match Case – Estruturas de escolha

Vamos aprender como usar o Python para criar uma instrução switch-case.

Antes da versão 3.10 do Python, o Python não tinha uma instrução switch-case oficial. Para conseguir isso, você tinha várias opções diferentes, como instruções if-else e dicionários.

Então agora vc utilizará instruções de caso de troca do Python no Python 3.10+, mostrar quais são alguns casos de uso para Python Switch-Case Statements.

Building a simple match-case statement in Python

```
response_code = 400
```

```
match response_code:
```

```
    case 200:
        print("Everything's peachy!")
    case 300:
        print("You're going somewhere else!")
    case 400:
        print("Uh oh. Are you lost?")
    case 500:
        print("Yikes. Something went wrong!")
    case _:
        print("WHOPSSSS!")
```

Match Case – Estruturas de escolha

Para isso o Python disponibiliza a estrutura match/case

Quando o algoritmo chega ao comando match, ele verifica o valor de uma variável e procura o case que combina com ele.

#Exemplo de estrutura match case no Python, equivalente ao switch em varias linguagens

```
opcao = int(input('Escolha a opção desejada: de 1 a 5: '))
```

```
match opcao:
```

```
    case 1:
```

```
        print('Vc optou pela alternativa 1')
```

```
    case 2:
```

```
        print('Vc optou pela alternativa 2')
```

```
    case 3:
```

```
        print('Vc optou pela alternativa 3')
```

```
    case 4:
```

```
        print('Vc optou pela alternativa 4')
```

```
    case 5:
```

```
        print('Vc optou pela alternativa 5')
```

```
    case _:
```

```
        print('opção inválida')
```

Match Case – Estruturas de escolha

Para isso o Python disponibiliza a estrutura match/case

Quando o algoritmo chega ao comando match, ele verifica o valor de uma variável e procura o case que combina com ele.

#Exemplo de estrutura match case

```
opcao = int(input('Escolha a opção desejada: '))
```

```
match opcao:
```

```
    case 1:
```

```
        print('Vc optou pela alternativa 1')
```

```
    case 2:
```

```
        print('Vc optou pela alternativa 2')
```

```
    case opção >=3:
```

```
        print('Vc optou pela alternativa 3')
```

```
    case _:
```

```
        print('opção inválida')
```

ERRO!!!

Match Case – Estruturas de escolha

• O que são instruções Switch-Case?

• Uma instrução switch ou case é uma instrução que avalia uma expressão em relação a um case e, em seguida, executa algum código. É um método pelo qual as linguagens de programação podem controlar o fluxo do programa, com base em uma condição (ou condições) atendidas, as estruturas condicionais como vimos anteriormente (if-then-else-elif).

• Por muitos anos, os Pythonistas têm procurado declarações switch-case. Eles são comuns na maioria das outras linguagens de programação e muitos acharam que era uma omissão estranha não incluí-los na linguagem. Isso exigia que os programadores Python usassem métodos diferentes e não padronizados, como instruções if-else, para emular a instrução switch.

• Em 2020, via PEP 634 (PEP 634 – Structural Pattern Matching: Specification), Python introduziu o conceito de Structural Pattern Matching em Python. Isso representa as instruções switch-case finalmente chegando ao Python de maneira oficial!

• Além disso, o Python realmente se estende além de uma simples instrução de caso e fornece controle de correspondência flexível.

Match Case – Estruturas de escolha

• PEP 634

PEP 634 – Structural Pattern Matching: Specification

Author: Brandt Bucher <brandt at python.org>, Guido van Rossum <guido at python.org>

BDFL-Delegate:

Discussions-To: [Python-Dev list](#)

Status: Accepted

Type: Standards Track

Created: 12-Sep-2020

Python-Version: 3.10

Post-History: 22-Oct-2020, 08-Feb-2021

Replaces: [622](#)

Resolution: [Python-Committers message](#)

► **Table of Contents**

Match Case – Estruturas de escolha

- Quando utilizar o match/case ao invés de if/elif?

- Se estiver realizando múltiplas verificações de valor em uma mesma variável, talvez seja mais vantajoso utilizar o escolha-caso

- Quais as vantagens de usar o escolha-caso?
 - Clareza e organização

Estrutura Match Case - Exercícios

1. Crie um algoritmo e o respectivo código para mostrar ao usuário, primeiramente, um menu de opções e o numero da plataforma, do trem que ele deve tomar, conforme tabela abaixo, e uma mensagem de erro quando o destino não existir:

Destino	Número da Plataforma
São Paulo	Plataforma 1
Rio de Janeiro	Plataforma 2
Curitiba	Plataforma 3
Brasilia	Plataforma 4
Florianópolis	Plataforma 5

Estrutura Match Case - Exercícios

2. Crie um algoritmo e o respectivo código para mostrar ao usuário no restaurante, uma carta de vinho com 5 opções de código, para que ele faça sua escolha, e retorne o nome do vinho e o preço da garrafa

Código	Vinho	Preço
1	Casa Valduga Gran	R\$ 160,00
2	Casa Silva Gran Terrois los Carménère	R\$ 130,00
3	Chocalan Origen Viognier	R\$ 180,00
4	Miolo Demi-sec Reserva especial 2005	R\$ 90,00
5	Casa Silva Terrois Cabernet Savignon	R\$ 100,00

Estrutura Match Case - Exercicios

3. Crie uma calculadora de 4 operações simples.

Ela ira solicitar dois números, depois mostrar um menu com as 4 operações e executa-las.

Por exemplo:

P = 4

S = 2

O resultado devera aparecer no seguinte formato:

A divisão de 4.0 e 2.0 é 2.0

Lembre-se que não é possível dividir por zero.....

Vamos Codar!!!

1. Crie um algoritmo que exibe os números inteiros de 1 a 1000.

2. Escreva um algoritmo que solicite 5 números e exiba o quadruplo de cada número digitado.

Vamos Codar!!!

3. Escreva um algoritmo que solicite a idade de 20 pessoas e informe a quantidade de pessoas que não podem entrar em um pub (idade inferior a 18 anos)

4. Escreva um algoritmo que solicite 10 números e informe quantos números entre 1 e 1000 foram digitados.

Vamos Codar!!!

5. Escreva um algoritmo que solicite 10 números e informe o somatório de todos os números pares que foram digitados.

6. Solicite vários números ao usuário (até que ele digite zero) e e na sequencia informe a média dos números digitados.

Vamos Codar!!!

7. Solicite dois números diferentes ao usuário. Se os números forem iguais, solicite novamente) e informe a diferença entre o maior e o menor número.

8. Escreva um algoritmo que solicite 20 números e informe qual foi o menor número digitado.

Vamos Codar!!!

9. Faça um algoritmo que solicite N números e calcule a soma para os números pares e para os ímpares

10. Faça um algoritmo que solicite N números e calcule a média para os números pares e para os ímpares

Vamos Codar!!!

11. Crie um algoritmo para calcular o fatorial de um numero digitado. O fatorial é uma multiplicação do numero N por seus antecessores até 1

$$n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$$

Vamos Codar!!!

12. Crie um algoritmo que resolva o valor de S para a série:

$$S = 1 + \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{N}$$

Vamos Codar!!!

13. Quantos primos tem nos 100.000 primeiros numero inteiros não negativos?

Crie um algoritmo para determinar se o numero é primo ou não.

Depois apresente todos os primos desde 1 até o primo mais próximo de N inclusive em formato de lista e a quantidade deles.

Exceções: try - except - else - finally

- Exceção é um erro que ocorre durante a execução de um programa e as exceções podem e devem ser tratadas pelo programador evitando que o programa termine de modo anormal.

Para familiarização com o conceito de exceção existem 2 exemplos básicos

#Exemplo 1 - Divisão por zero

```
divisor = int(input('Digite o divisor: '))
```

```
quoc = 10 / divisor
```

Caso o valor digitado seja zero na execução desse código ocorrerá uma exceção traduzida em uma mensagem como a seguinte:

Traceback (most recente call last):

File ...

Quoc = 10 / divisor

ZeroDivisionError: Division by zero

Exceções: try - except - else - finally

- Exceção é um erro que ocorre durante a execução de um programa e as exceções podem e devem ser tratadas pelo programador evitando que o programa termine de modo anormal.

Para familiarização com o conceito de exceção existem 2 exemplos básicos

#Exemplo 2 – Acesso a um elemento que não existe dentro de uma lista

```
lista = [1, 2, 3]  
print(lista[4])
```

Na tentativa de execução desse código ocorre uma exceção traduzida em uma mensagem como a seguinte:

Traceback (most recente call last):

File ...

```
print(lista[4])
```

IndexError: list index out of range

Exceções: try - except - else - finally

Algumas exceções comuns:

NameError: exceção gerada quando o programa não consegue encontrar um nome local ou global. O nome que não pôde ser encontrado é incluído na mensagem de erro.

TypeError: exceção gerada quando é passado um objeto de um tipo diferente do tipo que a função espera como argumento. Mais detalhes sobre o tipo errado são fornecidos na mensagem de erro.

ValueError: essa exceção ocorre quando um argumento de uma função tem o tipo certo, mas um valor inadequado.

Exceções: try - except - else - finally

Algumas exceções comuns:

`NotImplementedError`: essa exceção é gerada quando um objeto deveria dar suporte a uma operação, mas não foi implementado ainda. Você não deve usar este erro quando determinada função não dá suporte ao tipo de argumento de entrada. Nessas situações, disparar uma exceção `TypeError` é mais apropriado.

`ZeroDivisionError`: exceção gerada quando você fornece um zero como segundo argumento para uma divisão ou módulo.

`FileNotFoundError`: essa exceção é gerada quando o arquivo ou diretório que o programa solicitou não existe.

Exceções: try - except - else - finally

- Na execução de cada um desses exemplos, a ultima linha da mensagem indica o tipo de exceção, ZeroDivisionError e IndexError, seguido de um texto explicativo.

No caso as exceções são predefinidas e podem ser disparadas pelo interpretador ou por funções predefinidas de Python

Tratamento de Exceções:

Ao ocorrer uma exceção, podemos trata-la evitando que o programa termine sua execução de modo anormal. Basicamente, o tratamento é feito pelas clausulas *try* e *except*

Os comandos contidos nas clausulas *try*, *except*, *else* e *finally* aqui serão denominados *bloco try*, *bloco except*, *bloco else* e *bloco finally*.

Exceções: try - except - else - finally

- Os comandos contidos nas cláusulas *try*, *except*, *else* e *finally* aqui serão denominados *bloco try*, *bloco except*, *bloco else* e *bloco finally*. Obedecem ao seguinte formato:

try:

<comandos1> -

comandos que podem gerar uma exceção

except ([tipoExcecao, ...]) [as argumento]: -

<comandos2>

Exceções que se pretende capturar

Argumento que terá a informação sobre a causa de uma exceção

else:

<comandos3>

Comandos a serem executados caso ocorra uma exceção

finally:

<comandos4>

Comandos a serem executados caso não ocorra exceção. Devem estar após todos os blocos except

Comandos a serem executados ocorrendo ou não uma exceção.

Exceções: try - except - else - finally

Comando	Explicação
Comandos1	Comandos que podem gerar uma exceção
TipoExceção1, ...	Exceções que se pretende capturar
Argumento	Argumento que terá a informação sobre a causa de uma exceção
Comandos2	Comandos a serem executados caso ocorra uma exceção
Comandos3	Comandos a serem executados caso não ocorra exceção. Devem estar após todos os blocos except
Comandos4	Comandos a serem executados ocorrendo ou não uma exceção.

Exceções: try - except - else - finally

O funcionamento é o seguinte:

Os comandos contidos entre try e except são executados.

Se não ocorrer exceção, o bloco try é concluído e o bloco except, ignorado, e será executado o bloco else, caso exista.

Se ocorrer uma exceção, os comandos do bloco try após aquele que gerou a exceção serão ignorados e o bloco except será executado.

Exceções: try - except - else - finally

#exemplo – usar clausulas try e except com except sem exceções específicas

```
lista = ['janeiro', 'fevereiro', 'março']
```

```
ind = int(input('Digite o numero do mês a ser exibido (de 1 a 3): '))
```

```
ind = ind - 1
```

```
try:
```

```
    print(lista[ind])
```

```
    print('não ocorreu exceção')
```

```
except:
```

```
    print('ocorreu uma exceção')
```

```
    print('e o programa continua ...')
```

Exceções: try - except - else - finally

#exemplo – usar clausulas try e except com except sem exceções específicas

```
num1 = 10
```

```
num2 = 0
```

```
try:
```

```
    resultado = num1 / num2
```

```
except ZeroDivisionError:
```

```
    print('Erro: divisão por zero')
```

```
else:
```

```
    print(resultado)
```

```
finally:
```

```
    print('Fim do programa – Lega!!! Entendi')
```


Função no Python

- Usamos função para automatizarmos uma ROTINA, uma REPETIÇÃO que é feita.
- É um bloco de código que executa uma tarefa específica.
- Em Python, você pode definir suas próprias funções para executar tarefas específicas.
- Isso pode tornar seu código mais organizado, mais legível e mais fácil de manter.
- Vamos ver um exemplo simples de como definir uma função em Python.



```
+ def soma (a, b):  
    resultado = a + b  
    return resultado
```



```
s = soma (1, 2)  
print(s)
```

Função no Python

Nesse exemplo, estamos definindo uma função chamada **soma** que recebe dois argumentos **a** e **b**.

Dentro do corpo da função, estamos adicionando **a** e **b** e armazenando o resultado na variável **resultado**.

Em seguida, estamos retornando o resultado usando a instrução **return**.

Isso significa que, quando a função é chamada, ela retorna o valor da variável **resultado**.

Para chamar uma função, você simplesmente usa seu nome, seguido dos argumentos entre parênteses.

Veja o exemplo ao lado:

Função no Python

Outro exemplo de definicao de função no Python

def **area_triangulo** (base, altura):

 area = (base * altura)/2

base = float(input('coloque o valor da base do triangulo: '))

altura = float(input('coloque o valor da altura do triangulo: '))

mult = **area_triangulo**(base, altura)

print('a area do triangulo é: ', mult)

Função no Python

Outro exemplo ainda de definicao de função no Python sem colocar parâmetros

```
print('-'*30)
```

```
print('      Vinheria Agnello')
```

```
print('-'*30)
```

```
print('')
```

```
print('-'*30)
```

```
print('      Promoção de vinhos - queima de estoque')
```

```
print('-'*30)
```

```
def linha_separacao():
```

```
    return print('-'*30)
```

```
linha_separacao()
```

```
print('      Promoção de vinhos - queima de estoque')
```

```
linha_separacao()
```

```
print('      Promoção de vinhos - Queima de Estoque')
```

```
linha_separacao()
```

Ofício nº 006/2023

São Paulo, 14 de fevereiro de 2023

Professor Doutor **EDER CASSETTARI**
Prof Associado USP/Esalq - Data Science & Analytics
Av. Pádua Dias, 235 – Trindade
CEP 13418-900 – Piracicaba - SP

Senhor Professor Dr Eder,

A *I2AI* - Associação sem fins lucrativos, que busca conectar pessoas, empresas, conhecimento e tecnologia para fomentar a adoção responsável e sustentável da Inteligência Artificial em todo o mundo, em parceria com a *CRDF Global*, líder global na promoção de segurança, saúde e progresso econômico, mitigando riscos e aprimorando oportunidades através da ciência, inovação e colaboração, convida Vossa Senhoria a participar da Conferência "*International Security and Responsible Research, Development and Use of Artificial Intelligence in Brazil*".

Este evento, acontecerá na cidade de São Paulo nos dias 9 a 11 de maio de 2023 e será realizado presencialmente. Ele reunirá cerca de 50 líderes de diferentes áreas relacionadas à Inteligência Artificial, como *CEO*, *C-Level*s, Militares, Pesquisadores, entre outros. A *CRDF Global* fornecerá transporte, hospedagem e diárias para os participantes selecionados que não residem em São Paulo, além de oferecer refeições durante a conferência. Já a *I2AI* ficará responsável pela divulgação e recrutamento de participantes interessados do Brasil.

Ao longo dos três dias de conferência, serão debatidos temas relevantes e atuais, tais como: Futuro da IA, IA e Guerra e Ética em uma sociedade movida pela IA. O idioma oficial do evento será o português, com interpretação simultânea para o inglês. A *CRDF Global* solicita um currículo resumido dos candidatos para aprovação.

Para mais informações sobre as instituições, acesse os sites da *CRDF Global* (<https://crdfglobal.org/>) e da *I2AI* (<https://www.i2ai.org/>). A *I2AI* espera contar com a presença de Vossa Senhoria nesta importante discussão sobre a Inteligência Artificial.

Por fim, solicito que confirme a sua candidatura até o dia 14 de fevereiro, pelo e-mail contato@i2ai.org, nos encaminhando **nome completo, correio eletrônico e uma minibiografia do candidato**. Outras informações, favor contatar Viviane Ramone, CMO *I2AI*, pelo e-mail: viviane.ramone@i2ai.org.

Respeitosamente,

 Documento assinado digitalmente
ONÉDIO SIQUEIRA SEABRA JUNIOR
Data: 15/02/2023 21:50:41-0300
Verifique em <https://verificador.it.br>

ONÉDIO SIQUEIRA SEABRA JUNIOR
PRESIDENTE *I2AI*

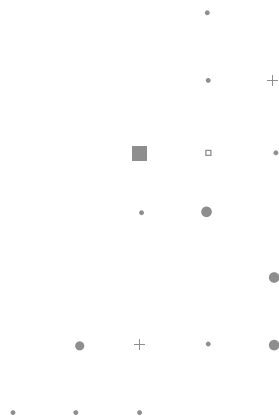


OBRIGADO

FIAP

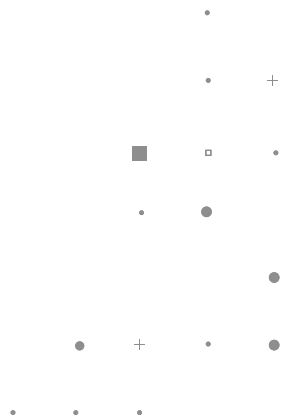
Copyright © 2022 | Professor Allen Fernando

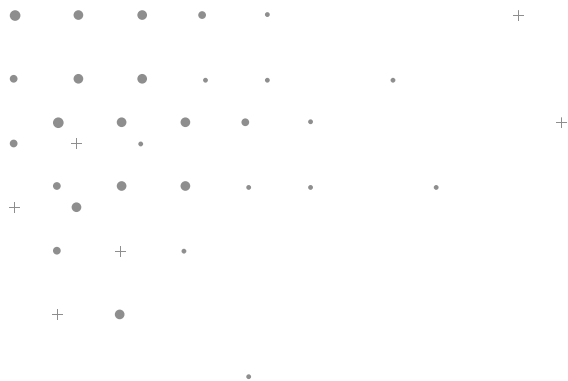
Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.



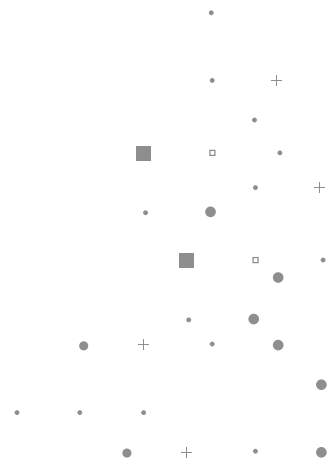


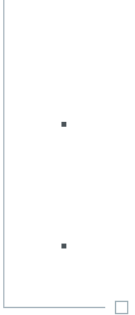
FIAP



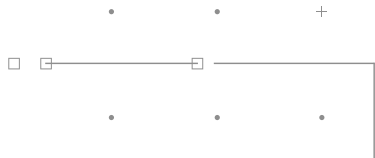


FIAP

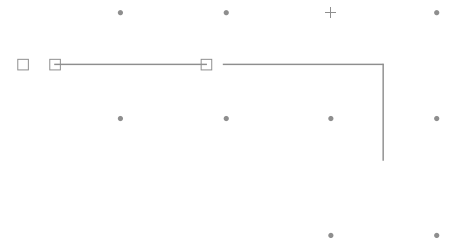
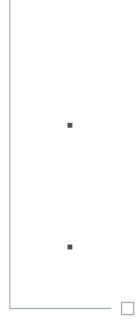




Computational Thinking with Python



Revisão 1º Semestre



AGENDA

1

Apresentação do professor

2

Objetivo da Disciplina e Bibliografia

3

Plano Semanal de Aulas

4

Metodologia

5

Avaliação

EMENTA

Os **Algoritmos** estão inseridos em quase todas as áreas da nossa vida. Podemos encontrá-los em computadores, notebooks, tablets, smartphones, TVs inteligentes, robôs e muitos outros equipamentos.

Entender como **construir soluções eficientes** para diversos problemas computacionais em uma das principais linguagens da atualidade: **Python**.

Dominar a base necessária para programar em outras linguagens como: Java, C#, C e muitas outras.

Criar programas e aplicativos que interajam com os usuários e os auxiliem no seu dia a dia.

Desenvolver conhecimento com o **aprendizado orientado a dados**.

OBJETIVOS

Apresentar e aplicar os conceitos de Lógica de Programação utilizando a Linguagem de Programação Python

HABILIDADES

Durante a disciplina espera-se que o aluno desenvolva as seguintes habilidades:

- Elaborar algoritmos.
- Codificar programas utilizando técnica de lógica de programação com a linguagem Python.

COMPETÊNCIAS

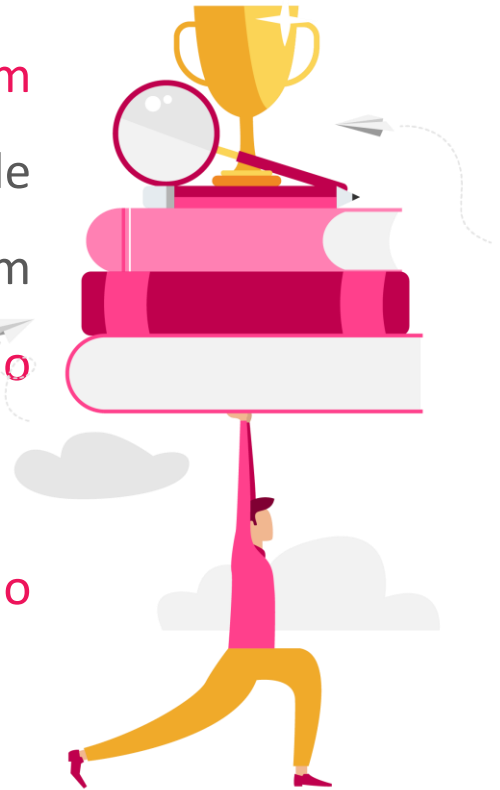
Ao término da disciplina os alunos deverão ser capazes de:

- 1 - Implementar algoritmos de programação.
- 2 - Utilizar linguagem de programação em ambiente de desenvolvimento.
- 3 - Entender os princípios de Codificação

Metodologia

A metodologia usada utiliza aulas teóricas e práticas em laboratório mescladas com uma grande quantidade de exercícios em cada unidade do conteúdo programático, além de jogos, projetos em grupo e trabalhos de avaliação continuada.

Além disso, a disciplina participa de um projeto integrado com outras disciplinas, a ser desenvolvido ao longo do ano.



CONTEÚDO

1º SEMESTRE

- ~~1. Conceito de algoritmos~~
- ~~2. Fluxograma | Pseudocódigo | Linguagem de programação~~
- ~~3. Tipos de dados~~
- ~~4. Inteiros | Reais | Texto | Lógico~~
- ~~5. Casting: int() | float() | str() | bool()~~
- ~~6. Variáveis e operadores Aritméticos~~
- ~~7. Comandos de entrada, saída e processamento de dados~~
- ~~8. print() | input()~~
- ~~9. Estruturas de Decisão e Operadores~~

CONTEÚDO

1º SEMESTRE

10. Operadores Relacionais e Lógicos

~~11. Decisão simples (if) | decisão composta (if else) | decisão encadeada (if elif else)~~

~~12. Estrutura de Seleção~~

~~13. Escolha: match case~~

~~14. Estruturas de repetição~~

~~15. Condicional: while~~

~~16. Contadora: for~~

~~17. Tratamento de erros~~

~~18. Comando: try except else finally~~

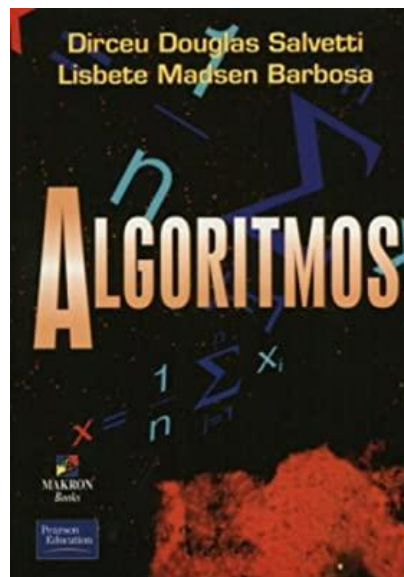
2° SEMESTRE

- ~~1. Subalgoritmos~~
- ~~2. Funções | Procedimentos~~
- ~~3. Parâmetros: Reais e formais | default | args~~
4. Estrutura de dados unidimensional
- ~~5. Estruturas de dados do tipo: vetor (dados homogêneos) | lista (list) | tupla (tuple)~~
6. Estrutura de dados homogênea bidimensional tipo matriz (dados homogêneos)
7. Busca e ordenação em vetores / listas

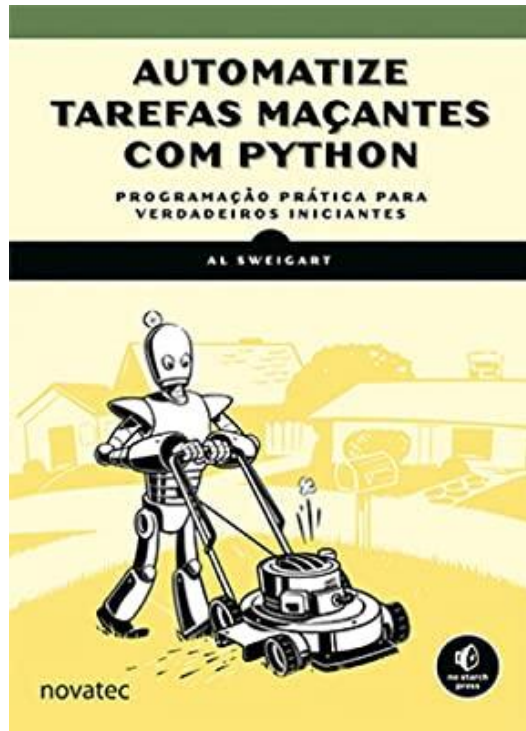
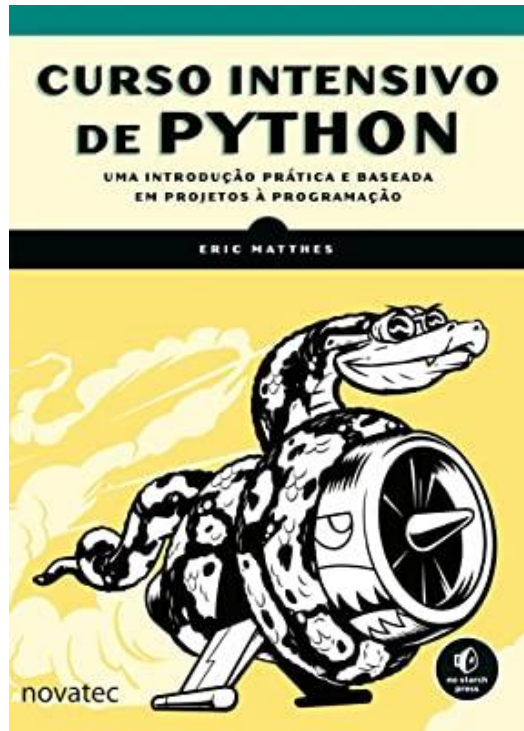
2° SEMESTRE

- 8. Conjuntos (set)
- 9. Dicionários
- 10. Tabelas de memória (listas x dicionários)
- 11. Arquivos
- 12. Manipulação de arquivos texto
- 13. Manipulação de arquivos CSV

BIBLIOGRAFIA BÁSICA



BIBLIOGRAFIA COMPLEMENTAR



HORÁRIO DE AULA

SEGUNDA FEIRA TEM AULA DO EDER (PRESENCIAL)

8h00 ÀS 09h40 e 10h00 ÀS 11h40 – manhã – 1ESPI

19h20 ÀS 21h00 e 21h15 ÀS 22h55 – noite -1ESPV - 1ESPW - 1ESPY



AVALIAÇÃO

As notas semestrais na FIAP são compostas:

- A média final para aprovação deve ser maior ou igual a 6,0 pontos. Essa nota será resultante do novo sistema de avaliação da FIAP, assim composto:
- Média 1º semestre: Challenge Sprint (2 atividades) e Checkpoints (2 a 3), representando 40% da nota semestral; uma Global Solution, representando 60% da nota do semestre. Essa nota representará 40% da média anual.
- Média 2º semestre: Challenge Sprint (2 atividades) e Checkpoints (2 a 3), representando 40% da nota semestral; uma Global Solution, representando 60% da nota do semestre. Essa nota representará 60% da média anual.

CÁLCULO DE MÉDIA ANUAL

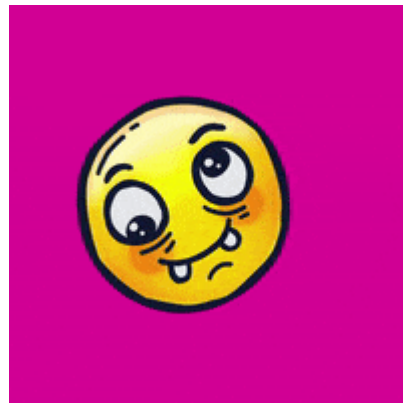
A média anual é ponderada, ou seja, os semestres possuem pesos diferentes:

$$MA = (MS1 \times 0.4 + MS2 \times 0.6)$$

CRITÉRIOS DE APROVAÇÃO

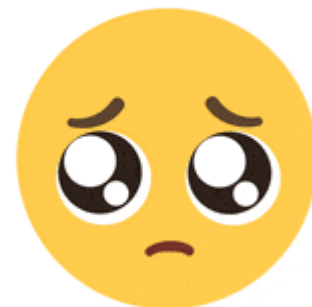


Média Anual	Situação
0 a 3.9	Reprovado
4.0 a 5.9	Exame
6.0 a 10	Aprovado



CASO O ALUNO FIQUE DE EXAME:

Nota para aprovação = $(12 - \text{Média Anual})$



QUE OS JOGOS COMECEM!

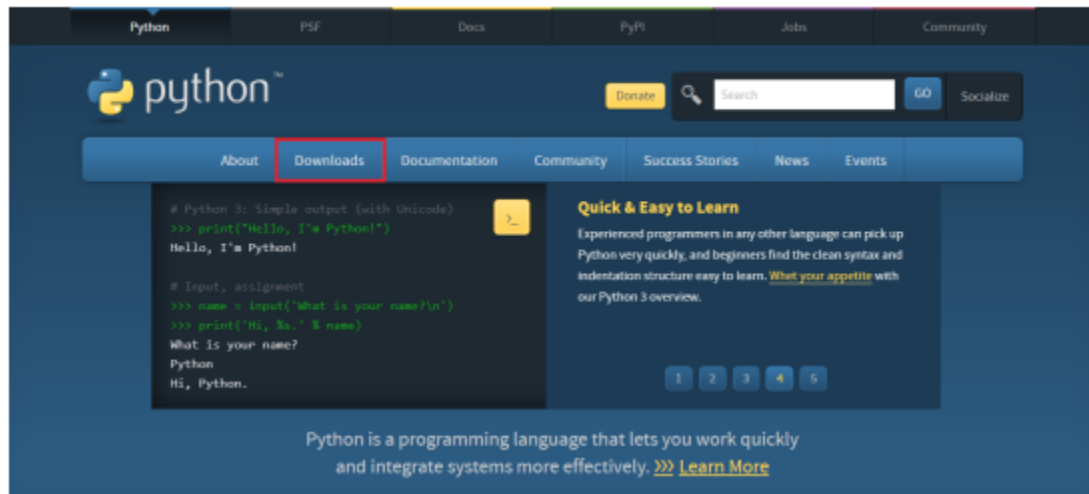


Instalação

- Instalação

- + Como mencionado anteriormente, Linux e MacOS já possuem o Python em seus terminais. Assim, é necessário instalar apenas no Windows. Nos outros sistemas operacionais, talvez seja necessário apenas atualizar.

- + Para a instalação deve-se abrir o site oficial do Python: <https://www.python.org/> e ir em Downloads



Instalação

Selecione a versão e o instalador adequado para o sistema

Looking for Python 2.7? See below for specific releases

Looking for a specific release?
Python releases by version number:

Release version	Release date		Click for more
Python 3.7.4	July 8, 2019	Download	Release Notes
Python 3.6.9	July 2, 2019	Download	Release Notes
Python 3.7.3	March 25, 2019	Download	Release Notes
Python 3.4.10	March 18, 2019	Download	Release Notes
Python 3.5.7	March 18, 2019	Download	Release Notes
Python 2.7.16	March 4, 2019	Download	Release Notes
Python 3.7.2	Dec. 24, 2018	Download	Release Notes
Python 3.6.8	Dec. 14, 2018	Download	Release Notes

[View older releases](#)

Licenses

All Python releases are [Open Source](#). Historically, most, but not all, Python releases have also been GPL-compatible. The [Licenses page](#) details GPL-compatibility and Terms and Conditions.

Sources

For most Unix systems, you must download and compile the source code. The same source code archive can also be used to build the Windows and Mac versions, and is the starting point for ports to all

Alternative Implementations

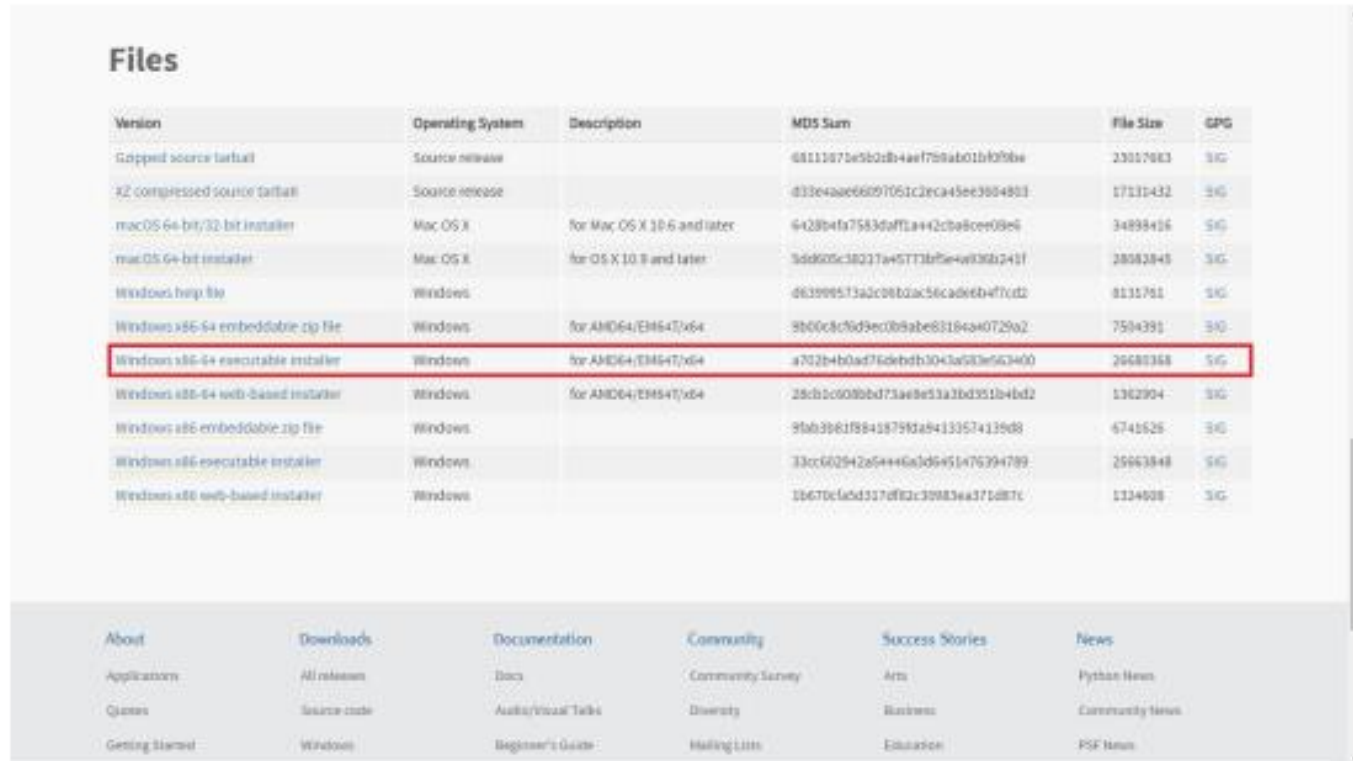
This site hosts the "traditional" implementation of Python (nicknamed CPython). A number of alternative implementations are available as well

History

Python was created in the early 1990s by Guido van Rossum at Stichting Mathematisch Centrum in the Netherlands as a successor of a language called ABC. Guido remains Python's principal author, although

Instalação

- Selecionar a versão e o instalador adequado para o sistema



Files

Version	Operating System	Description	MD5 Sum	File Size	GPC
Gimp source tarball	Source release		68111075e5b2db4ae7b9ab01b939ba	23017683	5G
x2 compressed source tarball	Source release		d33e4aae66097051c2eca45ee3804803	17331432	5G
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583daf1a442c9a8ce08e6	34898436	5G
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	3dd925c38217a45773bfe4e038b243f	38083848	5G
Windows help file	Windows		d6390e573a2c98b2ac56cad6b477cd2	8131761	5G
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64/x64	9b00c8c7d9ec0b9abe83186a40729a2	7504391	5G
Windows x86-64 executable installer	Windows	for AMD64/EM64/x64	a702b4b0ad76c6dbb3043a93e963400	2668368	5G
Windows x86-64 web-based installer	Windows	for AMD64/EM64/x64	28db5c6080bd73ae8e53a2bd351b4bd2	1362904	5G
Windows x86 embeddable zip file	Windows		9fab3861f8841879fda94133574139d8	6746526	5G
Windows x86 executable installer	Windows		33cc602942a54446ad64951476394789	2668388	5G
Windows x86 web-based installer	Windows		1b670c5ed317df82c3083ea371d87c	1324908	5G

[About](#)[Downloads](#)[Documentation](#)[Community](#)[Success Stories](#)[News](#)

Applications

Quizzes

Getting Started

All releases

Source code

Windows

Docs

Audio/Visual Talks

Beginner's Guide

Community Survey

Diversity

Mailing Lists

Arts

Business

Education

Python News

Community News

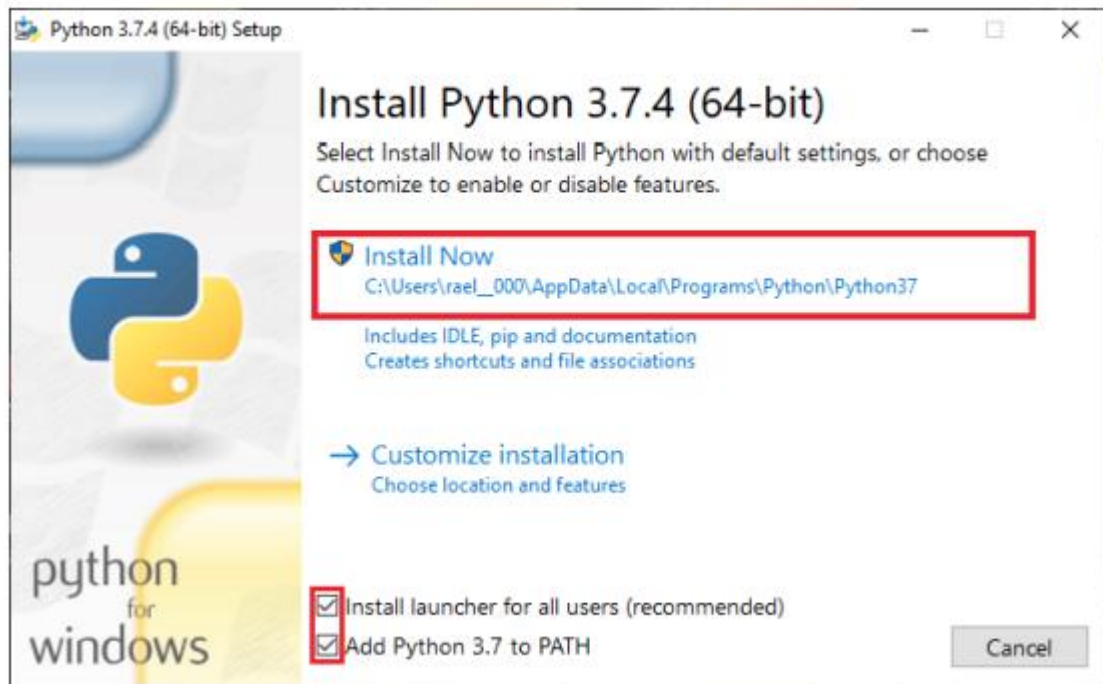
PSF News

- • • • • +
- • • • •

• Por fim, basta abrir o instalador e clicar em Install Now.

É recomendado marcar as duas caixas de seleção para que também seja possível acessar o Python pelos terminais do Windows.

Instalação



IDE's

- Para o aprendizado e desenvolvimento de softwares em linguagens de programação existem programas com funcionalidades e ferramentas integradas, que auxiliam os programadores. Esses programas são Ambientes de Desenvolvimento Integrado, do inglês Integrated Development Enviroment (IDE).

Para o desenvolvimento em Python, a IDE PyCharm, do desenvolvedor JetBrains, é bem conceituada e, por isso, recomendada. O download pode ser feito no site oficial da JetBrains, <https://www.jetbrains.com/pycharm/>.

Há a versão paga Professional, com mais utilidades, e a versão gratuita para a comunidade estudantil e científica: Community. A IDE ideal é aquela que atende aos gostos e necessidades do usuário.

Outros ambientes mais amplos como Atom ou Sublime Text também são muito populares ou até mesmo um editor de texto para uma tela mais limpa como o Notepad++.

Instalador de Pacotes - pip

O pip é o gerenciador de pacotes padrão do Python. Ele permite a instalação e o gerenciamento de pacotes adicionais que não fazem parte da biblioteca padrão.

Basicamente, um pacote é um diretório que contém módulos, estes, por sua vez, são objetos que servem como uma unidade organizacional de código Python.

O pip é um programa que é utilizado via linha comando. Para usá-lo, a seguinte estrutura de comando é utilizada:

Instalador de Pacotes - pip

Para instalar um pacote é possível usar um dos comandos a seguir:

```
$ pip <argumentos>
```

Para instalar um pacote é possível

```
$ pip install pacote                # Instala a ultima versao
$ pip install pacote==1.0.4        # Instala uma versao
    especifica
$ pip install 'pacote>=1.0.4'      # Instala a partir de
    uma versao minima
```

E para desinstalar um pacote:

```
$ pip uninstall pacote
```

Shell X Script

Para instalar um pacote é possível usar um dos comandos a seguir:

As linguagens de programação, descritas acima como tradutoras entre usuários e máquinas funcionam a base de comandos.

Esses comandos são sequências de palavras reservadas e estruturas de códigos que seguem regras e traduzem o que o usuário quer para instruções de máquina.

Por exemplo, para fazer com que a máquina imprima "Hello World!" na tela, devemos passar para a linguagem algo como "ImprimaNaTela ("Hello World!")".

As regras e sintaxe irão variar de linguagem para linguagem.

Shell X Script

Quando as ações do programa se tornam mais complexas, o código responsável por elas também se torna mais complexo.

Uma característica importante das linguagens de programação é que os códigos, ou seja, as instruções que são passadas, são executados de forma sequencial.

Ou seja, a linha 1 é executada primeiro, em seguida a 2 e assim por diante, seguindo um fluxo linear. A sequencialidade implica que ações como:

```
1 a = 2;  
2 b = a + 1;
```

Irão ser executadas propriamente, enquanto que outras como:

```
1 b = a + 1;  
2 a = 2;
```

São impossíveis de ser executadas, porque a primeira linha depende da execução da segunda.

Variáveis - Tipos

Tipos do Python

DICA: A presença do caractere ponto já indica que o usuário deseja que o valor seja armazenado em float, mesmo que o valor seja inteiro. Exemplo: se digitado $x = 4.$ x será do tipo float.

DICA: O valor de uma variável pode ser impresso digitando apenas o nome dela.
Exemplo:
 $x = 4$
 x
 4

Variáveis - Tipos

Tipos do Python

Os dados podem ser convertidos para outros tipos usando-se o nome do tipo desejado e o valor como argumento.

```
>>> c = str(19)
>>> c
'19'
>>> x = int(c)
>>> x
19
```

Variáveis - Tipos

Tipos do Python

Mesmo os tipos sendo implicitamente definidos, o programador pode definí-los usando o nome do tipo e o valor como argumento, ou seja, fazendo uma conversão:

```
>>> x = float(3)
>>> a = str(x)
>>> x = x + 1
>>> a
'3.0'
>>> print('Valor = ' + a)
Valor = 3.0
>>> print('Valor = ', x)
Valor = 4.0
```

Variáveis - Tipos

Tipos do Python

DICA: Em Python, as aspas duplas também podem ser usadas para definir strings, mas geralmente são usadas aspas simples como boas práticas de programação.

DICA: Note que é possível usar uma para escrever a outra como caractere: ' "' ou "' ".

DICA: Deve-se ter atenção para essa sintaxe, porque apesar de para o Python aspas simples ou duplas terem o mesmo efeito, em outras linguagens elas podem ser fundamentalmente diferentes. Em C, por exemplo, as duplas definem arranjos de caracteres, enquanto que as simples definem caracteres únicos. Em PHP, as duplas definem tipos dinâmicos e as simples tipos estáticos.

(*) PHP é uma linguagem versátil e poderosa para o desenvolvimento de aplicativos web, permitindo a criação de sites dinâmicos e interativos que podem se comunicar com bancos de dados e outros serviços web.

Variáveis - Tipos

Tipos do Python

Valores numéricos são geralmente atribuídos na notação de base decimal, mas é possível atribuir valores na notação de binários, de base oito e de hexadecimais com os prefixos 0b, 0o e 0x, respectivamente.

```
>>> d = 31 #base decimal
>>> d
31
>>> b = 0b11111 #base binaria
>>> b
31
>>> o = 0o37 #base octal
>>> o
31
>>> h = 0x1F #base hexadecimal
>>> h
31
```

Variáveis - Tipos

Tipos do Python

DICA : Valores em bases específicas podem ser dados usando-se o comando `int('valor', base = X)`, sendo que o valor precisa estar obrigatoriamente em string.

Exemplo:

```
>>> x = int('11', base=3)
```

```
>>> x
```

Como as estruturas de dados abordadas aqui são classes, elas possuem métodos. A orientação a objetos será abordada futuramente, mas por hora, os métodos serão tratados como funções e procedimentos inerentes a toda variável.

Variáveis - Tipos

Tipos do Python

Por exemplo, uma string que contém uma frase pode ser separada pelas suas palavras pelo método `split()`. O uso dos métodos se dá da seguinte forma: `Variável.método()`.

```
>>> Frase = 'Uma frase'
>>> Frase
'Uma frase'
>>> Frase.split()
['Uma', 'frase']
>>> a, b = Frase.split()
>>> a
'Uma'
>>> b
'frase'
```

Variáveis - Tipos

Tipos do Python

Para visualizar todos os atributos e métodos da classe deve-se utilizar o comando `dir(classe)` ou `dir(variavel)`.

Python também possui outras classes, estruturas compostas. São elas listas, tuplas, e dicionários, além das strings. Vereemo-las em Estrutura de Dados.

Variáveis - Tipos

Exercícios

1 Em um determinado programa são necessárias variáveis de diferentes tipos para armazenar diferentes informações. Em cada caso a seguir informe qual tipo de dado primitivo seria mais adequado para o armazenamento. Informe também qual o tipo do Python correspondente:

- a) Idade de uma pessoa;
- b) Se a pessoa é maior de idade ou não;
- c) Altura de uma pessoa;
- d) Peso de uma pessoa;
- e) Primeira letra do nome de uma pessoa;
- f) Última letra do nome de uma pessoa.

Variáveis - Tipos

Exercícios

4 Em cada item a seguir diga se pode ser um nome de variável. Em caso negativo, informe o motivo:

- a) x;
- b) aux;
- c) 1aluno;
- d) aluno1;
- e) aluno_1;
- f) aluno 1;
- g) _aluno1;
- h) int;
- i) Int;
- j) int_idade;
- k) class

Variáveis - Tipos

Exercícios

2 Considere as variáveis: $a = 1$; $b = 2$; $c = '1'$; $d = '2'$.

Utilizando o operador "+" observe a diferença entre somar $a + b$ e somar $c + d$.

Qual a diferença e por que isso ocorre?

3 Ainda considerando as variáveis da questão anterior, o que poderia se fazer para que $a + b$ tenha o mesmo resultado de $c + d$ ou vice versa?

Comandos e funções

Entendido as estruturas de dados, é necessário compreender os comandos e funções para melhor utilizá-los.

Conteúdos mais complexos, bem como funções definidas pelo usuário e leitura/escrita de arquivos serão vistos posteriormente.

Os comandos são palavras chaves que executam certas ações e as funções são procedimentos que devolvem dados dependendo de seus argumentos.

Os conceitos são parecidos e as vezes confundidos. No Python, ainda, devido a orientação de objetos, alguns procedimentos, chamados de funções, são ainda métodos de objetos, mas essa distinção não será crucial para esta abordagem inicial.

Comandos e funções

As funções são similares a funções matemáticas, isto é, dependendo do argumento, o resultado é diferente.

O uso de funções em algum momento no script ou no Python interativo recebe o nome de chamada.

Quando uma função é chamada, geralmente recebe argumentos, que são passados como parâmetros, dentro de parênteses.

As respostas da função, quando há, são chamados de retornos. Desta forma temos que:

```
retorno1,retorno2 = funcao(argumento1,argumento2)
```

Comandos e funções

Por exemplo, ao usar a função de raiz quadrada `sqrt` com argumento 25, é retornado o valor 5, ou seja, para:

```
>>> x = sqrt(25)
```

A variável `x` receberá o valor 5.

O próprio usuário pode criar funções para serem utilizadas no código. Esse processo é denominado definição de funções.

A definição de funções será abordada mais a frente, porque, para isso, é necessário o conhecimento de controle de fluxo, isto é, estruturas condicionais, de repetição e aninhamento de estruturas.

Comandos e funções

• Importação

A linguagem Python é conhecida pelas diversas áreas de aplicações. Por vezes, tais áreas necessitam de comandos ou funcionalidades específicas que não estão, por padrão, no Python, elas são advindas de bibliotecas e precisam ser importadas, caso o usuário deseje utilizá-las. Para isso é utilizado o comando `import`.

Por exemplo, a função `sqrt` para cálculo de raízes quadradas está na biblioteca `math`. Portanto, deverá ser impresso um erro ao utilizá-la sem importar a biblioteca.

```
>>> sqrt(9)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'sqrt' is not defined
```

Para corrigir o erro, antes é necessário usar o comando *`import math`*.

Comandos e funções

• Importação

Ainda assim, ao utilizar as funcionalidades advindas da biblioteca deve-se usar o nome da biblioteca. No caso:

```
>>> import math
>>> sqrt(9)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'sqrt' is not defined
>>> math.sqrt(9)
3.0
```

Comandos e funções

• Importação

+ • Importação Otimizada

O comando `import` importa a biblioteca com todas as suas funcionalidades. As vezes, apenas algumas funcionalidades são de interesse, como no exemplo acima, apenas a `sqrt`. Há a opção de importar apenas o que se deseja para economizar memória e para não precisar especificar a biblioteca da qual foi importado. Para isso, em conjunto com o comando `import`, utiliza-se o comando `from`, de forma a ficar como: `from import , ...`. Dessa forma, os métodos são importados diretamente para a pasta, por isso não é necessário explicitar a biblioteca de origem.

```
>>>from math import cos, pi
>>>cos(pi)
-1.0
```

Comandos e funções

• Importação

+ • Importação Otimizada

DICA: Dentro do site oficial do Python, na aba PyPI, é possível acessar o index de pacotes do Python, isto é, todas as bibliotecas e projetos possíveis de se importar. <https://pypi.org/>. É possível criar uma própria biblioteca e disponibilizar na comunidade para livre acesso.

IDE: Ctrl + espaço na IDE PyCharm mostra todas as opções dentro da biblioteca.

IDE: Para gerenciar os pacotes e bibliotecas instalados pela IDE PyCharm, deve-se ir em *Settings* ou *Preferences*, selecionar o projeto e selecionar *Project Interpreter*. Lá, será possível visualizar as bibliotecas instaladas, adicionar ou remover bibliotecas.

Comandos e funções

Importação

Outras Bibliotecas

Algumas bibliotecas são built-in, isto é, já são instaladas por padrão junto com o Python, como a biblioteca math e random. A importação delas é feita naturalmente. No entanto, ao tentar importar bibliotecas que não são built-in, deve-se antes instalá-las na máquina.

Algumas IDEs já fazem o gerenciamento e possuem atalhos para facilitar este processo, podendo importá-las também naturalmente. Mas, se for o caso do usuário não usar IDEs, ou usar o IDLE, por exemplo, deve-se fazer o download do pacote. Para isso, é recomendado usar o pip. Para instalar uma nova biblioteca, navegue até o diretório padrão de instalação do Python, abra o terminal e digite o comando:

```
$ pip install <bib>
```

Comandos e funções

Importação

Outras Bibliotecas

NumPy

NumPy, de Numeric Python, é uma famosa e importante biblioteca de Python, conhecida também como uma biblioteca científica.

É útil pela sua capacidade de processamento vetorial multidimensional homogêneo, estendendo o núcleo básico do Python.

CURIOSIDADE : Por convenção, a biblioteca NumPy é importada com o nome de np.

```
>>> import numpy as np
```


Comandos e funções

Importação

Outras Bibliotecas

NumPy

NumPy, de Numeric Python, é uma famosa e importante biblioteca de Python, conhecida também como uma biblioteca científica.

É útil pela sua capacidade de processamento vetorial multidimensional homogêneo, estendendo o núcleo básico do Python.

Vetores e matrizes podem ser criados pelo comando `array`, usando-se vírgulas e colchetes para identificá-los. Para selecionar elementos de um vetor ou matriz, deve-se usar também colchetes, muito similar ao uso e criação de listas:



Comandos e funções

NumPy

Vetores e matrizes podem ser criados pelo comando array, usando-se vírgulas e colchetes para identificá-los.

Para selecionar elementos de um vetor ou matriz, deve-se usar também colchetes, muito similar ao uso e criação de listas:

```
>>> u = np.array([1, 0, 0])
>>> M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> M
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> u
array([1, 0, 0])
>>> type(M), type(u)
(<class 'numpy.ndarray'>, <class 'numpy.ndarray'>)
>>> u[2]
0
>>> M[1, 2]
6
```

Comandos e funções

NumPy

Em Python, o ndarray (N-dimensional array) é um tipo de objeto que representa um array multidimensional. É uma estrutura de dados muito útil para armazenar e manipular grandes conjuntos de dados numéricos, como imagens, áudio, sinais de sensores, entre outros.

O ndarray é uma parte importante da biblioteca NumPy, que é usada para realizar cálculos numéricos em Python. O ndarray permite que você execute operações matemáticas em grandes conjuntos de dados de uma maneira eficiente e fácil de usar. O ndarray é caracterizado por alguns atributos, como shape (forma), dtype (tipo de dados) e strides (passos), que permitem que você acesse e manipule seus elementos de maneira eficiente.

Comandos e funções

NumPy

Por exemplo, você pode criar um ndarray usando a função `numpy.array` da seguinte maneira:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Este código cria um ndarray de 3 por 3 com os valores de 1 a 9. Você pode acessar os elementos deste ndarray da seguinte maneira:

```
a[0, 0] # retorna 1
a[1, 1] # retorna 5
a[2, 2] # retorna 9
```

Comandos e funções

NumPy

Em resumo, o ndarray é uma estrutura de dados fundamental da biblioteca NumPy que permite manipular eficientemente arrays multidimensionais em Python.

Pode-se especificar o tipo de dado com o parâmetro dtype, e há algumas funções especiais de criação de arranjos:

```
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
>>> np.eye(3, dtype=np.int64)
array([[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]], dtype=int64)
>>> np.zeros((2, 3))
array([[0., 0., 0.],
       [0., 0., 0.]])
>>> np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
>>> np.full((2, 3), 5)
array([[5, 5, 5],
       [5, 5, 5]])
>>> np.random.random((2, 3))
array([[0.36596055, 0.55954819, 0.94322377],
       [0.53376681, 0.00523872, 0.05504161]])
```

Comandos e funções

NumPy

Em Python, `int64` é um tipo de dados de inteiro que pode ser usado para armazenar números inteiros com sinal em um espaço de 64 bits.

O `int64` é um tipo de dados numérico pertencente à biblioteca NumPy, que é amplamente usada para realizar cálculos numéricos em Python.

O `int64` representa um número inteiro que pode variar entre -9223372036854775808 e 9223372036854775807, incluindo os extremos.

Esse tipo de dados é útil em situações em que é necessário trabalhar com grandes números inteiros e não há necessidade de usar números de ponto flutuante.

Comandos e funções

Valores podem ser adicionados em um arranjo pelo comando `append`. É um método usado para adicionar um elemento ao final de uma lista, que é uma das estruturas de dados built-in da linguagem Python.

O método **`append`** (acrescentar) é usado para modificar a lista original, adicionando o novo elemento ao final da lista existente.

```
>>> x = np.random.random((1, 2))
>>> x
array([[0.84375509, 0.04608968]])
>>> x = np.append(x, [1, 2])
>>> x
array([0.84375509, 0.04608968, 1., 2.])
```

Comandos e funções

DICA : Um arranjo vazio pode ser criado pela expressão:

`np.array((0))`

Operações

matemáticas

básicas podem ser

feitas entre arrays

ou entre array, e

número usando-se

os operadores

normalmente

```
>>> x
array([0.84375509, 0.04608968, 1.          , 2.          ])
>>>
>>> x + 10
array([10.84375509, 10.04608968, 11.          , 12.          ])
>>> x - 10
array([-9.15624491, -9.95391032, -9.          , -8.          ])
>>> x * 5
array([ 4.21877543,  0.23044842,  5.          , 10.          ])
>>> x / 2
array([0.42187754, 0.02304484, 0.5          , 1.          ])
>>> y = np.array([1, 2, 3, 4])
>>> x + y
array([1.84375509, 2.04608968, 4.          , 6.          ])
>>> x / y
array([0.84375509, 0.02304484, 0.33333333, 0.5          ])
```


Comandos e funções

Para a multiplicação vetorial que representa o produto escalar entre dois vetores, há o método `dot()`.

O produto escalar entre dois vetores é definido como a soma dos produtos dos elementos correspondentes dos dois vetores.

```
>>> x
array([0.84375509, 0.04608968, 1.          , 2.          ])
>>> y
array([1, 2, 3, 4])
>>> x.dot(y)
11.935934454939305
>>> x.mean()
0.9724611925863914
>>> x.max()
2.0
>>> x.min()
0.04608968459373963
```

Sejam dois vetores v e w com dimensões iguais n , o produto escalar entre eles é representado por $v \cdot w$ e é calculado da seguinte maneira: $v \cdot w = v_1 * w_1 + v_2 * w_2 + \dots + v_n * w_n$. O resultado do produto escalar entre dois vetores é um número que representa a projeção de um vetor sobre o outro. O produto escalar é utilizado em diversas áreas da matemática e da física, como na geometria, álgebra linear, cálculo vetorial, análise de sinais, entre outras. Para extrair o valor máximo, mínimo e a média há os métodos `max()`, `min()` e `mean()`, respectivamente.

Comandos e funções

•Matplotlib

Este é um pacote para plotar figuras de diversos estilos, como gráficos com linhas variadas, histogramas, gráficos de barras, dentre outros.

É, geralmente, usado em conjunto com o NumPy, deixando o Python um pouco mais parecido com o Matlab.

Assim como a literatura convencionou a importação de NumPy como np, também convencionou a importação do módulo pyplot de matplotlib como plt.

Comandos e funções

•Matplotlib

Tendo os arranjos de x e y, pode-se plotar o gráfico usando:

```
>>> plt.plot(x, y)
>>> plt.show()
```

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import math
4 import colorama
5
6 """
7 Estilos de linhas:
8 https://matplotlib.org/3.1.1/api/\_as\_gen/matplotlib
9 .pyplot.plot.html#matplotlib.pyplot.plot
10 """
11 def sine(x):
12     return (np.sin(x))
13
14 def plot(x, y):
15     plt.plot(x,y,'g-')
16     plt.grid(True)
17
18     # Limite dos eixos
19     # plt.axis([0, 10*math.pi, -1, 1.01])
20
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
20
21 # Nome dos eixos
22 plt.ylabel('Eixo y')
23 plt.xlabel('Eixo x')
24
25 # Acentos devem ser usados com o código TeX
26 # plt.title(r'$\ddot{o}\acute{e}\grave{e}\hat{O}$
    \breve{i}\bar{A}\tilde{n}\vec{q}$', fontsize=20)
27
28 # Título do gráfico
29 plt.title(r'$Gr\acute{a}fico$', fontsize = 15)
30
31
32 def cardiod():
33     t = np.arange(0, 2*math.pi, 0.01)
34     r = 1 + np.cos(t)
35     x = r * np.cos(t)
36     y = r * np.sin(t)
37     plot(x,y)
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
38 plt.show()
39
40
41 def butterfly():
42     t = np.arange(0, 1000*math.pi, 0.01)
43     x = np.sin(t) * (np.exp(np.cos(t)) - 2*np.cos(4*t) -
44                     np.power(np.sin(t/12), 5))
45     y = np.cos(t) * (np.exp(np.cos(t)) - 2*np.cos(4*t) -
46                     np.power(np.sin(t/12), 5))
47     plot(x,y)
48     plt.show()
49
50 def tesla():
51     x1 = np.arange(0, 2.0*math.pi, 0.01)
52     x2 = np.arange(-2.0*math.pi/3.0, 4.0*math.pi/3.0, 0.01)
53     x3 = np.arange(-4.0*math.pi/3.0, 2.0*math.pi/3.0, 0.01)
54     y1 = np.sin(x1)
55     y2 = np.sin(x2 + 2.0*math.pi/3.0)
56     y3 = np.sin(x3 + 4.0*math.pi/3.0)
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
56 plot(x1,y1)
57 plot(x2,y2)
58 plot(x3,y3)
59 plt.show()
60
61 def lissajous_curve():
62     a = 4.0
63     b = 5.0
64     kx = 3.0
65     ky = 2.0
66     t = np.arange(0, 2.0*math.pi, 0.01)
67     x = a*np.cos(kx*t)
68     y = b*np.sin(ky*t)
69     plot(x,y)
70     plt.show()
71
72 def begin():
73     print('Curves: \n 1 - Cardioid \n 2 - Butterfly Curve \n
74           3 - Tesla \n 4 - Lissajous Curve')
75     curve = int(input('Insert curve number: '))
```

Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
56 plot(x1,y1)
57 plot(x2,y2)
58 plot(x3,y3)
59 plt.show()
60
61 def lissajous_curve():
62     a = 4.0
63     b = 5.0
64     kx = 3.0
65     ky = 2.0
66     t = np.arange(0, 2.0*math.pi, 0.01)
67     x = a*np.cos(kx*t)
68     y = b*np.sin(ky*t)
69     plot(x,y)
70     plt.show()
71
72 def begin():
73     print('Curves: \n 1 - Cardioid \n 2 - Butterfly Curve \n
74           3 - Tesla \n 4 - Lissajous Curve')
75     curve = int(input('Insert curve number: '))
```


Comandos e funções

Matplotlib

Como por exemplo, pode-se criar um script que plota alguns gráficos como este:

```
75
76 if (curve == 1):
77     cardioid()
78 elif (curve == 2):
79     butterfly()
80 elif (curve == 3):
81     tesla()
82 elif (curve == 4):
83     lissajous_curve()
84 else:
85     print(colorama.Fore.RED + colorama.Back.WHITE +
86           '\nInvalid input, try again')
87     print(colorama.Style.RESET_ALL)
88     begin()
89
90 if __name__ == '__main__':
91     # x = np.arange(0, 10*math.pi, 0.01)
92     # y = sine(x)
93     # y = np.arange(0)
94     # y = math.sin(x)
95     colorama.init()
96     begin()
```

Variáveis e entradas de dados – parte 2

Exercício 3.1 Complete a tabela a seguir, marcando inteiro ou ponto flutuante dependendo do número apresentado.

Número	Tipo numérico
5	☐ inteiro ☐ ponto flutuante
5.0	☐ inteiro ☐ ponto flutuante
4.3	☐ inteiro ☐ ponto flutuante
-2	☐ inteiro ☐ ponto flutuante
100	☐ inteiro ☐ ponto flutuante
1.333	☐ inteiro ☐ ponto flutuante

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Quando formatamos números decimais, podemos utilizar dois valores entre o símbolo de % e a letra f.

O primeiro indica o tamanho total em caracteres a reservar;
O segundo, o número de casas decimais.

Assim, %5.2f diz que estaremos imprimindo um número decimal utilizando cinco posições, sendo que duas são para a parte decimal.

Isso é muito interessante para exibir o resultado de cálculos ou representar dinheiro.

Variáveis e entradas de dados – parte 2

Variáveis do tipo String

Isso é muito interessante para exibir o resultado de cálculos ou representar dinheiro.

Por exemplo, para exibir R\$ 5, você pode utilizar

```
"R$%f" % 5
```

mas o resultado não é bem o que esperamos, pois normalmente utilizamos apenas dois dígitos após a vírgula quando falamos de dinheiro.

Atalhos no Spyder – 125 no total... Aqui alguns

<https://keycombiner.com/collections/spyder/>



Atalhos no Spyder

Aqui estão alguns dos atalhos mais comuns no Spyder, um ambiente de desenvolvimento integrado (IDE) para Python:

- Executar o código: Pressione "F5" ou "Ctrl+Enter"
- Comentar código: Selecione o código e pressione "Ctrl+1"
- Descomentar código: Selecione o código e pressione "Ctrl+1"
- Indentação automática: Selecione o código e pressione "Tab"
- Remover indentação automática: Selecione o código e pressione "Shift+Tab"
- Auto completar: Pressione "Ctrl+Space"
- Ir para a definição da função: Pressione "Ctrl+Click" ou "Ctrl+B"
- Ir para a declaração da variável: Pressione "Ctrl+Click" ou "Ctrl+B"
- Selecionar todas as ocorrências da palavra selecionada: Selecione a palavra e pressione "Ctrl+Shift+G"
- Excluir a linha atual: Pressione "Ctrl+Shift+D"
- Aumentar / diminuir a tela "Ctrl +" "Ctrl -"

Exceções: try - except - else - finally

- Exceção é um erro que ocorre durante a execução de um programa e as exceções podem e devem ser tratadas pelo programador evitando que o programa termine de modo anormal.

Para familiarização com o conceito de exceção existem 2 exemplos básicos

#Exemplo 1 - Divisão por zero

```
divisor = int(input('Digite o divisor: '))  
quoc = 10 / divisor
```

Caso o valor digitado seja zero na execução desse código ocorrerá uma exceção traduzida em uma mensagem como a seguinte:

Traceback (most recente call last):

File ...

Quoc = 10 / divisor

ZeroDivisionError: Division by zero

Exceções: try - except - else - finally

- Exceção é um erro que ocorre durante a execução de um programa e as exceções podem e devem ser tratadas pelo programador evitando que o programa termine de modo anormal.

Para familiarização com o conceito de exceção existem 2 exemplos básicos

#Exemplo 2 – Acesso a um elemento que não existe dentro de uma lista

```
lista = [1, 2, 3]  
print(lista[4])
```

Na tentativa de execução desse código ocorre uma exceção traduzida em uma mensagem como a seguinte:

Traceback (most recent call last):

File ...

```
print(lista[4])
```

IndexError: list index out of range

Exceções: try - except - else - finally

Algumas exceções comuns:

NameError: exceção gerada quando o programa não consegue encontrar um nome local ou global. O nome que não pôde ser encontrado é incluído na mensagem de erro.

TypeError: exceção gerada quando é passado um objeto de um tipo diferente do tipo que a função espera como argumento. Mais detalhes sobre o tipo errado são fornecidos na mensagem de erro.

ValueError: essa exceção ocorre quando um argumento de uma função tem o tipo certo, mas um valor inadequado.

Exceções: try - except - else - finally

Algumas exceções comuns:

`NotImplementedError`: essa exceção é gerada quando um objeto deveria dar suporte a uma operação, mas não foi implementado ainda. Você não deve usar este erro quando determinada função não dá suporte ao tipo de argumento de entrada. Nessas situações, disparar uma exceção `TypeError` é mais apropriado.

`ZeroDivisionError`: exceção gerada quando você fornece um zero como segundo argumento para uma divisão ou módulo.

`FileNotFoundError`: essa exceção é gerada quando o arquivo ou diretório que o programa solicitou não existe.

Exceções: try - except - else - finally

- Na execução de cada um desses exemplos, a ultima linha da mensagem indica o tipo de exceção, ZeroDivisionError e IndexError, seguido de um texto explicativo.

No caso as exceções são predefinidas e podem ser disparadas pelo interpretador ou por funções predefinidas de Python

Tratamento de Exceções:

Ao ocorrer uma exceção, podemos trata-la evitando que o programa termine sua execução de modo anormal. Basicamente, o tratamento é feito pelas clausulas *try* e *except*

Os comandos contidos nas clausulas *try*, *except*, *else* e *finally* aqui serão denominados *bloco try*, *bloco except*, *bloco else* e *bloco finally*.

Exceções: try - except - else - finally

- Os comandos contidos nas cláusulas *try*, *except*, *else* e *finally* aqui serão denominados *bloco try*, *bloco except*, *bloco else* e *bloco finally*. Obedecem ao seguinte formato:

try:

<comandos1> -

comandos que podem gerar uma exceção

except ([tipoExeção, ...]) [as argumento]: -

<comandos2>

Exceções que se pretende capturar

Argumento que terá a informação sobre a causa de uma exceção

else:

<comandos3>

Comandos a serem executados caso ocorra uma exceção

finally:

<comandos4>

Comandos a serem executados caso não ocorra exceção. Devem estar após todos os blocos except

Comandos a serem executados ocorrendo ou não uma exceção.

Exceções: try - except - else - finally

Comando	Explicação
Comandos1	Comandos que podem gerar uma exceção
TipoExceção1, ...	Exceções que se pretende capturar
Argumento	Argumento que terá a informação sobre a causa de uma exceção
Comandos2	Comandos a serem executados caso ocorra uma exceção
Comandos3	Comandos a serem executados caso não ocorra exceção. Devem estar após todos os blocos except
Comandos4	Comandos a serem executados ocorrendo ou não uma exceção.

Exceções: try - except - else - finally

O funcionamento é o seguinte:

Os comandos contidos entre try e except são executados.

Se não ocorrer exceção, o bloco try é concluído e o bloco except, ignorado, e será executado o bloco else, caso exista.

Se ocorrer uma exceção, os comandos do bloco try após aquele que gerou a exceção serão ignorados e o bloco except será executado.

Exceções: try - except - else - finally

#exemplo – usar clausulas try e except com except sem exceções específicas

```
lista = ['janeiro', 'fevereiro', 'março']
```

```
ind = int(input('Digite o número do mês a ser exibido (de 1 a 3): '))
```

```
ind = ind - 1
```

```
try:
```

```
    print(lista[ind])
```

```
    print('não ocorreu exceção')
```

```
except:
```

```
    print('ocorreu uma exceção')
```

```
    print('e o programa continua ...')
```

Exceções: try - except - else - finally

#exemplo – usar clausulas try e except com except sem exceções específicas

```
num1 = 10
```

```
num2 = 0
```

```
try:
```

```
    resultado = num1 / num2
```

```
except ZeroDivisionError:
```

```
    print('Erro: divisão por zero')
```

```
else:
```

```
    print(resultado)
```

```
finally:
```

```
    print('Fim do programa – Lega!!! Entendi')
```


Função no Python

- Usamos função para automatizarmos uma ROTINA, uma REPETIÇÃO que é feita.
- É um bloco de código que executa uma tarefa específica.
- Em Python, você pode definir suas próprias funções para executar tarefas específicas.
- Isso pode tornar seu código mais organizado, mais legível e mais fácil de manter.
- Vamos ver um exemplo simples de como definir uma função em Python.

Função no Python

```
def soma (a, b):
```

```
    resultado = a + b
```

```
    return resultado
```

Nesse exemplo, estamos definindo uma função chamada **soma** que recebe dois argumentos **a** e **b**.

Dentro do corpo da função, estamos adicionando **a** e **b** e armazenando o resultado na variável **resultado**.

Em seguida, estamos retornando o resultado usando a instrução **return**.

```
s = soma (1, 2)
```

```
print(s)
```

Isso significa que, quando a função é chamada, ela retorna o valor da variável **resultado**.

Para chamar uma função, você simplesmente usa seu nome, seguido dos argumentos entre parênteses.

Veja o exemplo ao lado:

Função no Python

Outro exemplo de definicao de função no Python

def **area_triangulo (base, altura):**

 area = (base * altura)/2

base = float(input('coloque o valor da base do triangulo: '))

altura = float(input('coloque o valor da altura do triangulo: '))

mult = **area_triangulo(base, altura)**

print('a area do triangulo é: ', mult)

Função no Python

Outro exemplo ainda de definicao de função no Python sem colocar parâmetros

```
print(' '*30)
```

```
print('      Vinheria Agnello')
```

```
print(' '*30)
```

```
print("")
```

```
print(' '*30)
```

```
print('      Promoção de vinhos - queima de estoque')
```

```
print(' '*30)
```

```
def linha_separacao():
```

```
    return print(' '*30)
```

```
linha_separacao()
```

```
print('      Promoção de vinhos - queima de estoque')
```

```
linha_separacao()
```

```
print('      Promoção de vinhos - Queima de Estoque')
```

```
linha_separacao()
```

Função no Python

Uma função é que um trecho de código que possui uma responsabilidade específica e que damos um nome à ele.

- Como criar funções com a palavra reservada def,
- Como definir seus parâmetros,
- O que são args e o que são kwargs,
- Como retornar dados de funções,
- Como definir funções com uma linha e mais...

Função no Python

Funções são blocos de código que executam funcionalidades específicas.

Normalmente são utilizados para evitar que determinada parte do seu código seja escrito várias vezes.

Em Python sua sintaxe é definida usando def e atribuindo um nome a ela, veja um exemplo:

```
1 def funcao():  
2     print("Bloco de código")
```

Função no Python

Observando essa função, podemos extrair algumas informações, iniciando com a palavra reservada para funções `def` o nome atribuído à função `funcao` e os parênteses `()` utilizado para definição dos dados de entrada da função, também chamados de parâmetros.

Em seguida usa-se dois pontos `:` e abaixo o bloco de código a ser executado, que neste caso é apenas imprimir de uma string.

Para “chamar” uma função, utilizamos o nome que foi definido, dessa forma:

```
1 def funcao():  
2     print("Bloco de código")  
3  
4  
5 funcao()
```

Função no Python

Parâmetros

- Além de executar código, funções também podem **receber** e **retornar** dados.
- Podemos enviar dados para uma função através de seus parâmetros.
- Observe o exemplo:

```
1 def imprime_nome(nome):  
2     print(f"Nome: {nome}")  
3  
4  
5 imprime_nome("Erickson")  
6 imprime_nome("Renan")  
7 imprime_nome("Daniel")
```

```
Nome: Erickson  
Nome: Renan  
Nome: Daniel
```


Função no Python

- Não entendeu essa notação do `print(f"Nome: {nome}")`? Isso se chama F-strings e é uma maneira de formatar código Python e Strings! Quer saber mais sobre esse assunto, então acesse nosso Post completo:

<https://pythonacademy.com.br/blog/f-strings-no-python>

- Quando a função é chamada, passamos uma string como dado de entrada - através do parâmetro `nome` - que é concatenada e impressa dentro da função.
- Caso nenhum valor seja informado ao chamar a função, um erro será gerado. Por exemplo, o seguinte código:

```
1  def imprime_nome(nome):  
2      print(f"Nome: {nome}")  
3  
4  
5  imprime_nome()
```

Função no Python

- Caso nenhum valor seja informado ao chamar a função, um erro será gerado. Por exemplo, o seguinte código:

```
1  def imprime_nome(nome):  
2      print(f"Nome: {nome}")  
3  
4  
5  imprime_nome()
```

Ocasionará o seguinte erro:

Saída do Terminal com Erro

```
TypeError: imprime_nome() missing 1 required positional argument: 'nome'
```

- Podemos resolver esse erro utilizando os “Valores Padrão” e os veremos a seguir

Função no Python

Chamada de Função Posicional *versus* Chamada de Função Nomeada

- Quando chamamos uma função, podemos utilizar a localização dos parâmetros para fazer o casamento entre o que foi chamado e o que foi definido na função.
- Para entender melhor, veja o exemplo a seguir:

```
1 def monta_computador(cpu='', armazenamento=0, memoria=0):
2     print('A configuração é: \n\t- CPU: {cpu}\n\t- Armazenamento: {armazenamento}Tb\n\t- Memória: {memoria}Gb')
3
4
5 monta_computador('Intel Core i9', 4, 64)
```

Resultado

```
A configuração é:
- CPU: Intel Core i9
- Armazenamento: 4Tb
- Memória: 64Gb
```

Função no Python

- O programador que escreveu a chamada da função `monta_computador` está respeitando a posição dos parâmetros, ou seja:
 - O valor "Intel Core i9" é referente ao primeiro parâmetro (`cpu`)
 - O valor 4 é referente ao segundo parâmetro (`armazenamento`)
 - O valor 64 se refere ao terceiro parâmetro (`memoria`)
- Essa é uma chamada de função posicional, ou seja: que respeita a ordem dos parâmetros.
- Outra forma de fazer essa chamada de função é utilizar os nomes dos parâmetros!
- Dessa forma, não é necessário respeitar a ordem de definição dos parâmetros!
- Veja o mesmo exemplo, mas agora utilizando os nomes dos parâmetros:

```
1 monta_computador(memoria=64, armazenamento=4, cpu='Intel Core i9')
```

- A saída será a mesma, pois como utilizamos os nomes, o Python saberá qual valor referência qual parâmetro!

Função no Python

Parâmetro *args

- Caso queira desenvolver uma função que recebe um número variável de parâmetros, pode-se utilizar o parâmetro *args!
- Dessa forma, a função receberá os argumentos em forma de Tupla e você poderá processá-los com um loop for por exemplo!
- Veja o código abaixo para entender melhor:

A função acima irá receber todos os valores passados para função no parâmetro *args e irá iterar sobre eles com um loop for.

```
1  def maior_30(*args):
2      print(args)
3      print(type(args))
4
5      for num in args:
6          if num > 30:
7              print(num)
8
9
10 maior_30(10, 20, 30, 40, 50, 60)
```

Função no Python

Parâmetro *args

- Veja o código ao lado para entender melhor e depois abaixo:

veja a saída:

```
(10, 20, 30, 40, 50, 60)
<class 'tuple'>
40
50
60
```

```
1  def maior_30(*args):
2      print(args)
3      print(type(args))
4
5      for num in args:
6          if num > 30:
7              print(num)
8
9
10 maior_30(10, 20, 30, 40, 50, 60)
```

O nome *args é uma convenção, ou seja uma boa prática entre programadores Python. Contudo, nada te impede de alterar esse nome para *numeros por exemplo. Dessa forma, a definição da função seria:

```
def maior_30(*numeros):
```

VAMOS CODAR!!!

Parâmetros: Reais e formais | default | args

- Funções em Python são blocos de código que executarão algum tipo de tarefa ou manipulação de dados, podendo ou não receber: dados de entrada (parâmetros/argumentos).
- **Parâmetros:** são os nomes dados aos atributos que uma função pode receber. Definem quais argumentos são aceitos por uma função, podendo ou não ter um valor padrão (default).
- **Argumentos:** são os valores que realmente são passados para uma função.

Veja a simples função ao lado:

```
1  def calculadora_salario(valor, horas=220):  
2      return horas * valor  
3  
4  valor_total = calculadora_salario(299.25)  
5  
6  print(valor_total)
```


Parâmetros: Reais e formais | default | args

- No exemplo da função **calculadora_salario**, definimos dois parâmetros que são chamados de valor e horas.
- O parâmetro valor não possui um valor default, então é obrigatório passá-lo como argumento. s
- Já o parâmetro horas possui um valor default, o que o torna a passagem do argumento não obrigatório, assim é assumido o valor 220.
- Caso não passarmos o argumento para o parâmetro valor recebemos o seguinte erro mostrado abaixo:

```
Traceback (most recent call last):  
  File "learn_decorators.py", line 41, in <module>  
    print(calculadora_salario())  
TypeError: calculadora_salario() missing 1 required positional  
argument: 'valor'
```

Parâmetros: Reais e formais | default | args

- Note que os **parâmetros obrigatórios** devem ser colocados antes de qualquer parâmetro default (da esquerda para direita), para que não ocorra uma confusão no interpretador e ocorra o erro:

```
SyntaxError: non-default argument follows default argument
```

- Agora que entendemos obrigatoriedade de parâmetros, vamos analisar o uso dos parâmetros ***args** e ****kwargs**.
- Para começo de conversa, as palavras args e kwargs são apenas usadas como convenção, ou seja, poderiam ter qualquer outro nome, por exemplo: *parms e **kwparms.

Parâmetros: Reais e formais | default | args

*args

- É usado para passar uma lista de argumentos variável sem palavras-chave em forma de tupla, pois a função que o recebe não necessariamente saberá quantos argumentos serão passados.
- Exemplo:

```
1  def func_args(*args):
2      print(f'tipo: {type(args)} conteúdo: {args}')
3      for arg in args:
4          print(f'tipo: {type(arg)} conteúdo: {arg}')
5
6
7  func_args(1, 'A', {'valor': 10})
```

Parâmetros: Reais e formais | default | args

*args

```
1 def func_args(*args):
2     print(f'tipo: {type(args)} conteúdo: {args}')
3     for arg in args:
4         print(f'tipo: {type(arg)} conteúdo: {arg}')
5
6
7 func_args(1, 'A', {'valor': 10})
```

Output:

tipo: <class 'tuple'> conteúdo: (1, 'A', {'valor': 10})

tipo: <class 'int'> conteúdo: 1

tipo: <class 'str'> conteúdo: A

tipo: <class 'dict'> conteúdo: {'valor': 10}

No exemplo, tem-se uma função que recebe um parâmetro do tipo `*args` com alguns argumentos e escreve no terminal o tipo e o conteúdo, junto com o tipo e conteúdo de cada argumento da tupla.

Parâmetros: Reais e formais | default | args

*args

No exemplo, tem-se uma função que recebe um parâmetro do tipo `*args` com alguns argumentos e escreve no terminal o tipo e o conteúdo, junto com o tipo e conteúdo de cada argumento da tupla.

```
# Output:  
tipo: <class 'tuple'> conteúdo: (1, 'A', {'valor': 10})  
  
tipo: <class 'int'> conteúdo: 1  
tipo: <class 'str'> conteúdo: A  
tipo: <class 'dict'> conteúdo: {'valor': 10}
```

Parâmetros: Reais e formais | default | args

****kwargs**

- Como a abreviação sugere, kwargs significa **keyword arguments** (**argumentos de palavras chave**).
- Ele permite passar um dicionário com inúmeras keys para a função.

Exemplo:

```
1  def func_kwargs(**kwargs):
2      print(f'tipo: {type(kwargs)} contudo: {kwargs}')
3      for key, value in kwargs.items():
4          print(f'atributo: {key}, valor: {value}')
5
6  func_kwargs(nome='James', sobrenome='Bond', cargo='Agente 007')
```

Parâmetros: Reais e formais | default | args

****kwargs**

```
1  def func_kwargs(**kwargs):
2      print(f'tipo: {type(kwargs)} conteúdo: {kwargs}')
3      for key, value in kwargs.items():
4          print(f'atributo: {key}, valor: {value}')
5
6  func_kwargs(nome='James', sobrenome='Bond', cargo='Agente 007')
```

Output

```
tipo: <class 'dict'> conteúdo: {'nome': 'James', 'sobrenome': 'Bond',
'cargo': 'Agente 007'}
```

```
atributo: nome, valor: James
```

```
atributo: sobrenome, valor: Bond
```

```
atributo: cargo, valor: Agente 007
```

Parâmetros: Reais e formais | default | args

****kwargs**

- Como visto anteriormente, no output da função usando ** podemos notar que o tipo do argumento é passado como dicionário com chaves e valores.
- Outra forma de passar um dicionário já existente é usando os ** na passagem do parâmetro:

```
parms = {  
    'nome': 'James',  
    'sobrenome': 'Bond',  
    'cargo': 'Agente 007'  
}  
  
func_kwargs(**parms)
```


Parâmetros: Reais e formais | default | args

- Agora para finalizar, veremos como ficaria um função usando todos os parâmetros que vimos.

```
1  def func_missao_dificil(nome, *args, funcao='agente', **kwargs):
2      print(f'Nome do agente: {nome}')
3      print(f'Função: {funcao}')
4      print(args)
5      print(kwargs)
6
7  params = {
8      'id_agente': '007',
9      'proxima_missao': 'Impossível'
10     }
11
12  func_missao_dificil(
13      'James Bond',
14      'Missao 1',
15      'Missão 2',
16      **params
17  )
```

```
# Output
Nome do agente: James Bond
Função: agente
('Missão 1', 'Missão 2')
{'id_agente': '007', 'proxima_missao': 'Impossível'}
```

Parâmetros: Reais e formais | default | args

No exemplo anterior, foi utilizado todos os recursos mencionados:

- O parâmetro nome recebe o valor “James Bond”;
- O parâmetro função utilizou o valor “agente” definido como default;
- O parâmetro args recebeu os valores “Missao 1” e “Missao 2” como tupla;
- O parâmetro kwargs recebeu dois parâmetros nomeados como dicionário:
`{‘id_agente’: ‘007’, ‘proxima_missao’: ‘Impossível’}`
- Por hoje é isso pessoal, aprendemos um pouco sobre esse recurso muito interessante da linguagem Python. Espero ter ajudado no entendimento dessas palavrinhas especiais para construção de funções.

VAMOS CODAR!!!

Dicionários

Este assunto é algo MUITO importante, famosos objetos Dicionários!

São estruturas extremamente flexíveis, que possibilitam adicionar quase de tudo!

Introducing Python in Excel

Harness the power of Python in Microsoft Excel's familiar interface.

[Explore Python in Excel](#)

Dicionários

1. Introdução
2. Criando dicionários
3. Acessando itens
4. Obtendo chaves e valores
5. Adicionando e atualizando itens
6. Excluindo elementos no dicionário
7. Limpando dicionário
8. Copiando Dicionário
9. O método dict.setdefault()
10. Criando Dicionários a partir de lista de valores
11. Operador de “desempacotamento”
12. Dicionários como args e kwargs
13. Juntando dicionários com
**

Dicionários

- Os dicionários são coleções de itens e seus elementos são armazenados de forma não ordenada.
- Seus elementos contém uma chave e valor, isto é:
 - Uma chave que vai servir para indexar (posicionar) determinado elemento no dicionário.
 - Um valor que contém... Bem, um valor :laughing: Este valor aceita diversos tipos: listas, outros dicionários, inteiros, strings e etc.
- Sua sintaxe básica é: {'chave': 'valor'}. Utiliza-se {} para delimitar o dicionário e a chave é separada do valor por dois pontos :.

Dicionários

- Exemplo de sua sintaxe:

```
1  dicio = {'chave': 'valor'}  
2  
3  print(type(dicio))
```

- Quando utilizar type() e a saída for essa abaixo, pode ter certeza que é um dicionário!

```
<class 'dict'>
```


Dicionários

Criando dicionários

Agora vamos ver 6 maneiras de criar um mesmo dicionário!

O modo mais simples de criar um dicionário:

- `dicio_2 = {'primeiro': 1, 'segundo': 2, 'terceiro': 3}`

Também podemos utilizar a função `dict` do próprio Python (built-in function), passando as chaves e valores como parâmetros:

- `dicio = dict(primeiro=1, segundo=2, terceiro=3)`

Dicionários

Criando dicionários

- Utilizando a função zip para concatenar a chave:valor em um elemento do objeto dict:
- `dicio_3 = dict(zip(['primeiro', 'segundo', 'terceiro'], [1, 2, 3]))`
- Utilizando uma lista de tuplas com itens simbolizando chave e valor em um objeto dict:
- `dicio_4 = dict([('primeiro', 1), ('segundo', 2), ('terceiro', 3)])`

Dicionários

Criando dicionários

- Com Dict Comprehensions!
- `dicio_6 = {name: idx + 1 for idx, name in enumerate(('primeiro', 'segundo', 'terceiro'))}`
- Por mais estranho que pareça, podemos tentar transformar uma variável do tipo dicionário em dicionário.
- `dicio_5 = dict({'primeiro': 1, 'segundo': 2, 'terceiro': 3})`

Dicionários

Criando dicionários

- Todos esses exemplos tem o mesmo resultado, que está sendo mostrado abaixo:

```
{'primeiro': 1, 'segundo': 2, 'terceiro': 3}
```

Dicionários

Acessando itens

Para acessar o valor de um dicionário podemos utilizar sua chave. Exemplo:

```
1  pessoa = {'nome': 'Pythonico', 'altura': 1.65, 'idade': 21}
2
3  print(pessoa['nome'])
```

Saída com o valor da chave especificada:

```
Pythonico
```

Dicionários

Dicionários também contam com um método chamado `get()` que fornecerá o mesmo resultado:

```
print(pessoa.get('nome'))
```

A saída será a mesma, o resultado:

```
Pythonico
```

Dicionários

- Uma coisa interessante sobre esse método é que você pode definir um valor *default*, para o caso da chave não ser encontrada, exemplo:

```
print(pessoa.get('peso', 'Chave não encontrada'))
```

- Como não existe nenhuma chave chamada 'peso', ele retorna:

```
Chave não encontrada
```

Dicionários

Obtendo chaves e valores

- Se você deseja obter todas as chaves de um dicionário, use o método `keys()`

```
computador = {'CPU': 'Intel', 'RAM': '8gb', 'SSD': '250bg'}  
  
print(computador.keys())
```

- A saída é um objeto do tipo `dict_keys` que equivale a uma lista com todas as chaves:

```
notas = {'Mat': 5, 'Por': 7, 'His': 8}  
  
print(notas.values())
```


Dicionários

Obtendo chaves e valores

- A saída será uma lista com os valores de cada chave:

```
dict_values([5, 7, 8])
```

Dicionários

Percorrendo os elementos de um Dicionário

Podemos percorrer os elementos de um Dicionário a partir de suas chaves

`dict.keys()` ou a partir de seus valores **`dict.values()`**.

Percorrendo elementos com a função **`dict.keys()`**:

```
computador = {'CPU': 'Intel', 'RAM': '8gb', 'SSD': '250bg'}

for chave in computador.keys():
    print(f'Chave = {chave} e Valor = {computador[chave]}')
```

Dicionários

Percorrendo os elementos de um Dicionário

A saída será:

```
Chave = CPU e Valor = Intel  
Chave = RAM e Valor = 8gb  
Chave = SSD e Valor = 250bg
```

Percorrendo elementos com a função dict.values():

```
notas = {'Mat': 5, 'Por': 7, 'His': 8}  
  
for valor in notas.values():  
    print(f'Valor: {valor}')
```



A saída será:

```
Valor: 5  
Valor: 7  
Valor: 8
```

Dicionários

Percorrendo as chaves e valores de um Dicionário

Uma maneira de obter chaves e valores é utilizando o método `dict.items()`.

Veja como:

```
frutas = {'pera': 10, 'uva': 2, 'maça': 55}
print(frutas.items())
```

Dicionários

Percorrendo as chaves e valores de um Dicionário

- A saída equivale a uma lista com elementos organizados em tuplas, sendo:
 - O primeiro elemento da tupla a chave do dicionário e
 - O segundo elemento, o valor presente naquela chave.

Veja:

```
dict_items([('pera', 10), ('uva', 2), ('maça', 55)])
```

Dicionários

Percorrendo as chaves e valores de um Dicionário

- Uma maneira segura de acessar um elemento de um dicionário é verificando primeiro se a chave existe. Exemplo:

```
dicio_cores = {'amarelo': 10, 'vermelho': 2, 'cinza': 55}

# Verificando se chave existe
if 'amarelo' in dicio_cores:
    print(f"Como a cor desejada existe: {dicio_cores['amarelo']}")

# Verificando se valor existe
if 10 in dicio_cores.values():
    print('O valor desejado existe')
```

Dicionários

+ Percorrendo as chaves e valores de um Dicionário

- A saída será:

```
Como a cor desejada existe: 10  
O valor desejado existe
```

Dicionários

Adicionando e atualizando itens

- A maneira mais simples de adicionar um item a um dicionário é criar uma nova chave e atribuir um valor. Assim:

```
dicio = {'nome': 'Erick'}  
  
dicio['idade'] = 20  
  
print(dicio)
```

- Resultando na adição da chave e valor especificada acima:

```
{'nome': 'Erick', 'idade': 20}
```


Dicionários

Adicionando e atualizando itens

- O método **update()** tanto pode criar, como atualizar dados:

```
dicio = {'nome': 'Erick'}

# Atualiza o elemento de chave 'nome'
dicio.update({'nome': 'Matheus'})

# Cria os elementos de chave 'idade' e 'tamanho'
dicio.update({'idade': 18})
dicio.update(tamanho=1.60)

print(dicio)
```

Dicionários

Adicionando e atualizando itens

- Saída:

```
{ 'nome': 'Matheus', 'idade': 18, 'tamanho': 1.60 }
```

Dicionários

Excluindo elementos no dicionário

Você pode excluir um elemento do dicionário com a keyword `del` do Python, especificando sua chave:

```
peessoa = {'nome': 'Matheus', 'idade': 18, 'tamanho': 1.60}

del peessoa['tamanho']

print(peessoa)
```

Resultando na exclusão do dado especificado:

```
{'nome': 'Matheus', 'idade': 18}
```

Dicionários

Excluindo elementos no dicionário

Excluindo e retornando o elemento excluído

O método **pop()** remove o item cujo a chave foi especificada:

```
sacola = {'maça': 2, 'ovos': 6, 'farinha': 2}

ovos = sacola.pop('ovos')

print(ovos)
print(sacola)
```

Saida:

```
6
{'maça': 2, 'farinha': 2}
```

Dicionários

Excluindo elementos no dicionário

Excluindo último item:

- O método **popitem()** também remove o último elemento do dicionário.
- Contudo, diferente do método `pop()` que apenas retorna o valor daquele elemento, `popitem()` retorna o elemento todo, contendo chave e valor.
- Veja usando o mesmo exemplo do método `pop()`:

```
sacola = {'maça': 2, 'ovos': 6, 'farinha': 2}

farinha = sacola.popitem()

print(farinha)
print(sacola)
```

Dicionários

Excluindo elementos no dicionário

Veja que o retorno foi uma tupla e o último item foi removido:

```
('farinha', 2)
{'maça': 2, 'ovos': 6}
```

Limpando dicionário

Uma maneira de esvaziar um dicionário é utilizando o método `clear()`, dessa forma:

```
dicio = {'nome': 'F9', 'motor': 'v8', 'ano': 2019}
dicio.clear()

print(dicio)
```

O retorno é um dicionário **vazio**: `{}`.

Dicionários

Copiando Dicionário

- Em Python, você não pode simplesmente digitar `dict1 = dict2` para copiar um dicionário.
- Dessa forma você está apenas copiando a referência do `dict1` ao `dict2`.
- A maneira correta de criar uma cópia do dicionário é utilizando o método **`copy()`**, assim:

```
dicio = {"operacao": "web scraping", "dados": 250}

copia = dicio.copy()

print(copia)
print(dicio)
```

Dicionários

Copiando Dicionário

- Resultando na cópia exata do dicionário

```
{'operacao': 'web scraping', 'dados': 250}  
{'operacao': 'web scraping', 'dados': 250}
```


Dicionários

Copiando Dicionário

- O método **dict.setdefault()**
- O método dict.setdefault() retorna o valor da chave especificada, e, caso a chave não exista, a cria com o valor especificado.
- Vamos ao exemplo:

```
dicio = {'coleta': 'scrapy', 'dados': 200}

set_dados = dicio.setdefault('dados')

print(set_dados)
print(dicio)
```

Dicionários

Copiando Dicionário

- Como a chave dados existe, ela é retornada:

```
200  
{'coleta': 'scrapy', 'dados': 200}
```

Dicionários

Copiando Dicionário

- Agora vamos adicionar apenas a chave sem valor, e outro com chave e valor:

```
dicio = {'coleta': 'scrapy', 'dados': 200}

set_data = dicio.setdefault('data')
set_teste = dicio.setdefault('teste', 1)

print(set_data)
print(set_teste)
print(dicio)
```

Dicionários

Copiando Dicionário

- A saída resulta em:

```
None
1
{'coleta': 'scrapy', 'dados': 200, 'data': None, 'teste': 1}
```

- O **set_data** que teve apenas atribuída uma chave, teve seu valor criado como None, enquanto set_teste teve o valor adicionado.

Dicionários

Criando Dicionários a partir de lista de valores

- Com o método **dict.fromkeys()** é possível criar um dicionário a partir de uma lista de chaves e um valor, que será usado em todas as chaves.
- Veja o exemplo:

```
chaves = ['chave1', 'chave2', 'chave3']  
valor = 0  
  
dicio = dict.fromkeys(chaves , valor)  
  
print(dicio)
```

Dicionários

Criando Dicionários a partir de lista de valores

- Veja que o valor 0 foi atribuído em todas as chaves:

```
{'chave1': 0, 'chave2': 0, 'chave3': 0}
```

Dicionários

Operador de “desempacotamento”

- O Python introduziu em sua linguagem um operador de “unpacking”, ou operador de desempacotamento.
- Para utilizar esse operador, devemos usar dois asteriscos antes da variável que vai ser desconstruída (**variavel).
- Pode ser utilizado em tuplas, listas e em nossos queridos dicionários.

Dicionários

Operador de “desempacotamento”

- No caso do Dicionário, o que ele faz é desconstruir cada elemento do dicionário em chave e valor, tentando casar o nome do argumento da função com a chave do dicionário. Vamos ao exemplo!
- Suponha a seguinte função:

```
def funcao(argumento):  
    print(argumento)
```


Dicionários

Operador de “desempacotamento”

- Agora vamos chamá-la duas vezes, uma sem o operador de desempacotamento e outra com ele:

```
dicionario = {'argumento': 1}
```

```
funcao(dicionario)
```

```
funcao(**dicionario)
```

- Agora veja a saída de cada um:

```
{'argumento': 1}
```

```
1
```

Dicionários

Operador de “desempacotamento”

- Na primeira chamada, sem o operador, a saída foi apenas o print daquilo que foi passado. No caso, um dicionário.
- Já na segunda, o operador fez com que a função tentasse “encaixar” a chave argumento do dicionário (de valor 1) com o argumento da função.
- Dessa forma, apenas o valor foi printado!

Dicionários

Dicionários como args e kwargs

- Como vimos aqui em cima, dicionários podem ser passados como valores para funções.
- Além disso, podemos usar o operador de desempacotamento para casar as chaves do dicionário com os argumentos da função:

```
regulagem = {'max': 10, 'meio': 5, 'min': 0}
```

```
def funcao(max, meio, min):  
    print(max)  
    print(meio)  
    print(min)
```

```
funcao(**regulagem)
```

Dicionários

Dicionários como args e kwargs

- Resultando em:

```
10  
5  
0
```

Dicionários

Dicionários como args e kwargs

- Se caso você desconheça as chaves de um dicionário, basta adicionar em sua função o atributo kwargs (Keyword Arguments ou Argumentos Nomeados).
- O atributo kwargs recebe todos os argumento que tenham sido passados com nome.
- Exemplo: `funcao(arg=1)`. Nesse caso, `arg` é um Argumento Nomeado e pode ser acessado pela variável `kwargs`.

Dicionários

Dicionários como args e kwargs

- Percorrendo o kwargs temos acesso às chaves, onde podemos, portanto, buscar os valores que desejamos através do método .get(). Veja o exemplo:

```
# Definição da Função recebendo kwargs
def funcao(**kwargs):
    # Percorrendo argumento nomeados
    for chave in kwargs:
        print(f"Acessando Chave '{chave}', valor = {kwargs.get(chave)}")

regulagem = {'max': 10, 'meio': 5, 'min': 0}

funcao(**regulagem)
```

- Saída:



```
Acessando Chave 'max', valor = 10
Acessando Chave 'meio', valor = 5
Acessando Chave 'min', valor = 0
```

Dicionários

Dicionários como args e kwargs

- Já o argumento args possui os Argumentos Não-Nomeados.
- Se necessário pegar apenas as chaves do dicionário, podemos utilizar args para isso e chamando a função passando um asterisco em nosso dicionário, da seguinte maneira:

```
def funcao(*args):  
    for chave in args:  
        print(chave)  
  
funcao(*regulagem)
```

Veja que saída contém apenas as chaves:



```
max  
meio  
min
```

Dicionários

Juntando dicionários com **

- Outra coisa que podemos fazer com o operador de desempacotamento é juntar dicionários.
- Como esse operador desconstrói um dicionário em chave e valor, podemos utilizá-lo para juntar dicionários, da seguinte forma:

```
regulagem = {'max': 10, 'meio': 5, 'min': 0}
extra = {'passo': 2}

# Junção de dicionários com **
juncao_dicio = {**regulagem, **extra}

print(juncao_dicio)
```

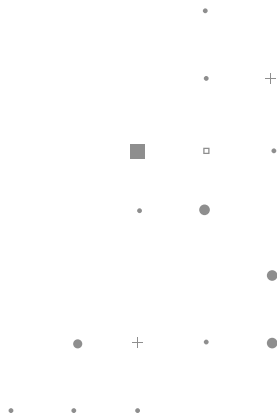
- Saída:



```
{'max': 10, 'meio': 5, 'min': 0, 'passo': 2}
```




VAMOS CODAR!!!



Set (Conjuntos)

+ Sets (conjuntos) são usados para armazenamento de uma coleção de dados.

- É definido entre chaves { } .
- É uma coleção **não ordenada** e **não indexada**
- conjunto = {"maçã", "banana", "laranja"}
- print(conjunto)
- {'banana', 'laranja', 'maça'}



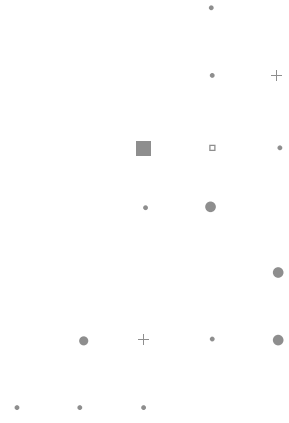
Set (Conjuntos)



Conjuntos não são ordenados

- Os itens de um conjunto não possuem uma ordem definida
 - Eles podem aparecer em uma ordem diferente sempre que você os utiliza
- 

Conjuntos não são indexados

- Os itens não podem ser referenciados por índice ou chave
- 

Set (Conjuntos)

Itens não repetidos

- Sets não podem ter itens com o mesmo valor.
- Valores duplicados serão ignorados.

```
conjunto = {"maçã", "banana", "laranja", "maçã"}
```

- `print(conjunto)`
- `{'banana', 'laranja', 'maçã'}`

Set (Conjuntos)

Conjuntos são heterogêneos:

- Podem armazenar itens de tipos diferente
 - True e 1 são considerados o mesmo valor

```
conjunto = {"abc", 34, 40, "nome" , 1, 1, True}
```

```
print(conjunto)
```

```
{1, 34, 40, 'nome', 'abc'}
```

Set (Conjuntos)

Tamanho do conjunto

A função **len()** é utilizada para obter o tamanho de um conjunto.

```
conjunto = {"maçã", "banana", "laranja", "maçã"}
```

```
print(len(conjunto))
```

Set (Conjuntos)

Acessando itens

- Não é possível acessar itens consultando um índice ou uma chave.
- É possível percorrer os itens usando um **for** ou verificar se um valor está presente em um conjunto usando a instrução **in**.

```
conjunto = {"maçã", "banana", "laranja"}
```

```
for x in conjunto:
```

```
    print(x)
```

```
if "maçã" in conjunto:
```

```
    print("OK")
```

```
laranja
```

```
banana
```

```
maçã
```

```
OK
```

Set (Conjuntos)

Inserindo itens

- Itens podem ser adicionados ao conjunto usando a função **add()**.
- Ao adicionar um item repetido, ele será ignorado.

```
conjunto = {"maçã", "banana", "laranja"}
```

```
conjunto.add("abacaxi")
```

```
conjunto.add("maçã")
```

```
print(conjunto)
```

```
{'banana', 'laranja', 'abacaxi', 'maçã'}
```


Set (Conjuntos)

Removendo itens

- Itens podem ser removidos utilizando a função **remove()** ou **discard()**.
- Se o item a ser removido não existir, remove() gerará um erro.
- Se o item a ser removido não existir, discard() NÃO gerará um erro.

```
nomes = {'Paulo', 'Ana', 'Pedro', 'Maria'}
```

```
nomes.remove('Ana')
```

```
nomes.discard('Maria')
```

```
print(nomes)
```

```
{'Paulo', 'Pedro'}
```

Set (Conjuntos)

Preenchendo conjuntos com entrada do usuário

- Podemos preencher um conjunto com dados inseridos pelo usuário utilizando uma estrutura for .
- Para criar um conjunto vazio utilize a função set()

```
numeros = set() # conjunto vazio
```

```
for i in range(5):
```

```
    n = int(input('Informe um Número: '))
```

Informe um Número: 1

Informe um Número: 2

Informe um Número: 3

Informe um Número: 4

Informe um Número: 5

{1, 2, 3, 4, 5}

Set (Conjuntos)

Unindo conjuntos:

- Existem várias formas de unir dois ou mais conjuntos em Python.
- Utilizando o método **union()** retorna um novo conjunto contendo todos os itens de ambos os conjuntos.

```
Set_1 = {"a", "b", "c"}
```

```
Set_2 = {1, 2, 3}
```

```
Set_3 = Set_1.union(Set_2)
```

```
print(Set_3)
```

```
{1, 2, 3, 'c', 'a', 'b'}
```

Set (Conjuntos)

Unindo conjuntos (cont):

- O método **intersection()** retorna um conjunto que contém os itens que existem em ambos os conjuntos.

```
x = {"apple", "banana", "laranja"}
```

```
y = {"google", "microsoft", "apple"}
```

```
z = x.intersection(y)
```

```
print(z)
```

```
{'apple'}
```

Set (Conjuntos)

Unindo conjuntos (cont):

- O método **difference()** retorna um conjunto que contém os itens que existem apenas em um conjunto e não existem no outro

```
x = {"apple", "banana", "cherry"}
```

```
y = {"google", "microsoft", "apple"}
```

```
z = x.difference(y)
```

```
print(z)
```

```
{'banana', 'cherry'}
```

Set (Conjuntos)

Função set()

- A função set pode ser utilizada para copiar os itens de uma lista, tupla ou string para um conjunto.
- Valores duplicados serão ignorados.

```
texto = 'bananeira'
```

```
conjunto = set(texto)
```

```
print(conjunto)
```

```
lista = ["apple", "banana", "cherry", "apple"]
```

```
conjunto = set(lista)
```

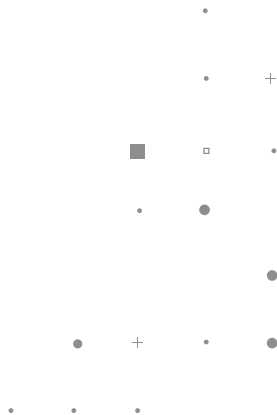
```
print(conjunto)
```

`{'e', 'r', 'i', 'n', 'b', 'a'}`

`{'cherry', 'banana', 'apple'}`

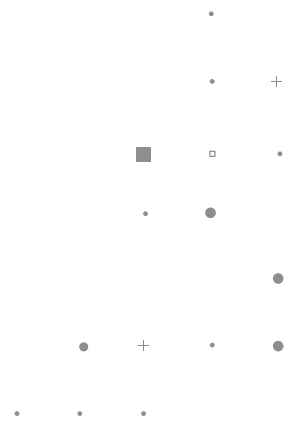


VAMOS CODAR!!!





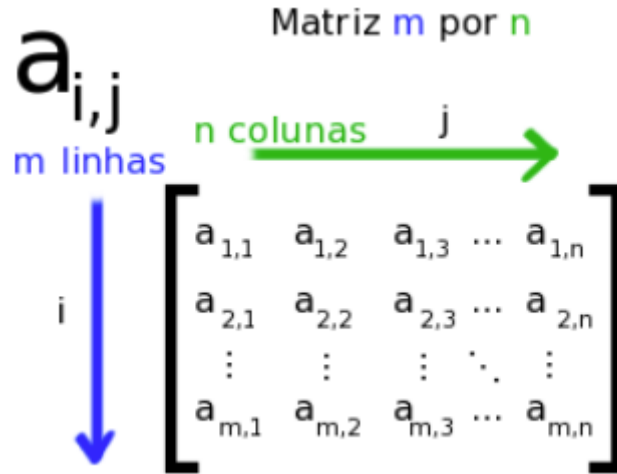
Matrices



Matrizes

Matriz

- É uma coleção de itens dispostos em uma tabela retangular organizada em linhas e colunas. É muito utilizado nas áreas de engenharia, física, estatística e jogos



Matrizes

Criando uma Matriz

- Para a criação de uma matriz basta criar uma lista na qual seus elementos são listas, compostos pelos valores de cada coluna, dentro da linha.

```
#Criando uma matriz 3 X 5
```

```
matriz = [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10],[11, 12, 13, 14, 15]]
```

```
print(matriz)
```

```
matriz = [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10],[11, 12, 13, 14, 15]]
```

```
print(matriz)
```

```
[[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15]]
```

Seleção de itens da matriz

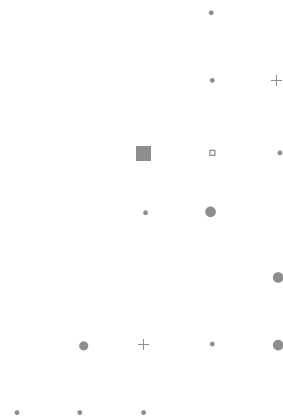
- Para acessar uma linha da matriz, informamos o índice da linha.
- Para acessar um elemento específico da matriz, é necessário informar o índice de linha e de coluna.

```
n [3]: matriz = [[1, 2, 3],  
                 [4, 5, 6]]
```

```
print(matriz[l] [e] )
```

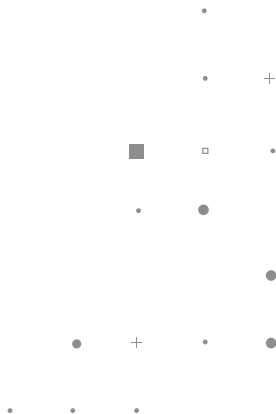


VAMOS CODAR!!!





Manipulacao de arquivos de texto



Manipulação de arquivos txt

- Manipulação de arquivo é um recurso muito utilizado além ser muito versátil e poderoso, que pode realizar uma quantidade muito grande de operações.
- Como exemplos podemos citar:
 - Criar arquivos de log
 - Armazenar dados
 - Ser utilizado para integrar aplicações

Manipulação de arquivos

- Antes de se fazer uma operação qualquer no arquivo (leitura ou gravação), necessário abri-lo
- Para tal utilizaremos a função **open()**
 - Parâmetros:
 - Caminho e nome do arquivo
 - Modo de abertura

```
arquivo = open('nomedoarquivo.txt', 'r')
```

Manipulação de arquivos

• Modos básicos de abertura:

- **r (read)** – abre um arquivo existente para uma operação de leitura de dados. Caso o mesmo não seja encontrado, irá gerar uma exceção.
- **w (write)** – Abre um arquivo para uma operacao de escrita de dados. Se o arquivo já existe, **ele será substituído**; se não existir, será criado.
- **a (add)** – abre um arquivo para operacao de anexar dados. Se o arquivo já existir **ele NÃO será substituído**; se o arquivo não existir, o mesmo será criado.

Leitura de Arquivos

Ex 1: Ler conteúdo de arquivo e copiar todos os caracteres para uma string.

```
1 # um arquivo chamado "nomearquivo.txt" será aberto para leitura.
2 arquivo = open('nomearquivo.txt', 'r')
3
4 # a função read() irá copiar todo o conteúdo do arquivo para uma string
5 texto = arquivo.read()
6 print(texto)
7
8 # fecha o arquivo e libera da memória
9 arquivo.close()
```

Leitura de Arquivos

- Ex 2: Ler conteúdo de um arquivo e fazer um loop para percorrer cada linha dentro do arquivo.

```
1 # um arquivo chamado "nomearquivo.txt" será aberto para leitura.
2 arquivo = open('nomearquivo.txt', 'r')
3
4 # o for irá percorrer cada linha no arquivo
5 for linha in arquivo:
6     print(linha)
7
8 # fecha o arquivo e libera da memória
9 arquivo.close()
```

Escrita de Arquivos

Ex 3: Criar um arquivo e escrever um texto no arquivo criado.

- Quando o arquivo é aberto em modo 'w', caso o arquivo já exista, **ele será sobrescrito.**

```
1 # um arquivo chamado "nomearquivo.txt" será criado e aberto para escrita.
2 arquivo = open('nomearquivo.txt', 'w')
3
4 # escreve uma string no arquivo com a função write()
5 arquivo.write("Este texto será escrito no arquivo\n")
6
7 # fecha o arquivo e libera da memória
8 arquivo.close()
```

Escrita de Arquivos

Ex 4: Criar um arquivo e escrever dados informados pelo usuário.

- Quando o arquivo é aberto em modo 'w', caso o arquivo já **ele será sobrescrito.**

```
1  # um arquivo chamado "nomearquivo.txt" será criado e aberto para escrita.
2  arquivo = open('nomearquivo.txt', 'w')
3
4  # escreve o valor de uma variável str no arquivo
5  nome = input('Digite seu nome: ')
6  arquivo.write(nome + '\n')
7
8  # escreve o valor de uma variável int no arquivo (deve converte para str)
9  idade = int(input('Digite sua idade: '))
10 arquivo.write(str(idade) + '\n')
11
12 # fecha o arquivo e libera da memória
13 arquivo.close()
```

Escrita de Arquivos

Ex 5: Adicionar texto em um arquivo existente.

- Quando o arquivo é aberto em modo 'a', o texto será adicionado **ao final do arquivo existente**

```
1 # um arquivo chamado "nomearquivo.txt" será aberto para adicionar conteúdo.
2 arquivo = open('nomearquivo.txt', 'a')
3
4 # escreve uma string no arquivo com a função write()
5 arquivo.write("Este texto será escrito no arquivo\n")
6
7 # fecha o arquivo e libera da memória
8 arquivo.close()
```

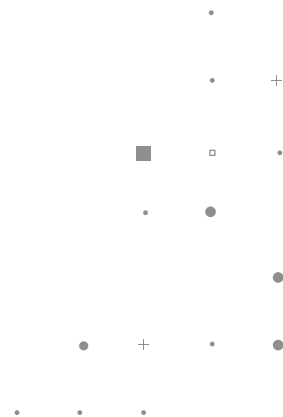
Fechamento de Arquivos

- A função `close()` fecha o arquivo e libera o espaço de memória ocupado por esse arquivo.
- Deve ser utilizado quando o arquivo não é mais necessário ou se for aberto em um modo de arquivo diferente.

```
1 arquivo = open('nomearquivo.txt', 'r')
2
3 # ...
4
5 arquivo.close()
```

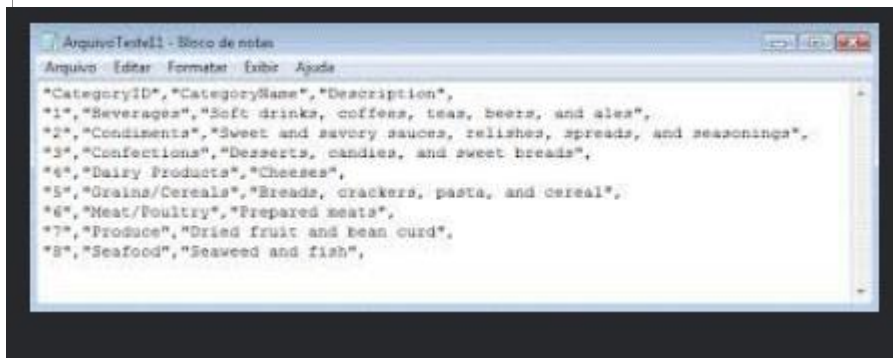


VAMOS CODAR!!!



Arquivos CSV

- Um arquivos CSV nada mais é um tipo de arquivo de texto que tem osvalores separados por vírgula (Comma Separated Values). Exemplo:



```
ArquivoIntelli - Bloco de notas
Arquivo Editar Formatar Inserir Ajuda

"CategoryID","CategoryName","Description",
"1","Beverages","Soft drinks, coffees, teas, beers, and ales",
"2","Condiments","Sweet and savory sauces, relishes, spreads, and seasonings",
"3","Confections","Desserts, candies, and sweet breads",
"4","Dairy Products","Cheeses",
"5","Grains/Cereals","Breads, crackers, pasta, and cereal",
"6","Meat/Poultry","Prepared meats",
"7","Produce","Dried fruit and bean curd",
"8","Seafood","Seaweed and fish",
```



```
Dados de exemplo do arquivo C:\PROG\DATA\CSV\PEOPLE.CSV
1 Number,Gender,Name,Surname,Title,GivenName,MiddleInitial,Suffix,StreetAddress,City,State,ZipCode
2 1,Male,Brazil,Mr.,Leonardo,E,Azevedo,"Rua Luper Segel 1855","Belo Horizonte",MG,"Minas Ger
3 2,Female,Brazil,Mrs.,Rodrigo,D,Melo,"Praça Tarriver Toledo Carlos Filipe 2044","204a Paulo",SP,"S
4 3,Male,Brazil,Mr.,Arthur,B,Branco,"Avenida Colombo 1574",Bomazama,PR,Sapopó,87403-130,PR,B
5 4,Male,Brazil,Mr.,Maurício,B,Pinto,"Rua Adone Horta 1767",Hercílio,Zé,Alagoinha,57025-120,RR,I
6 5,Female,Brazil,Mrs.,Rafaela,E,Lima,"Teatro das Nações 1458",Cruzeiro,CE,Ceará,63113-450,SE,Bre
7 6,Male,Brazil,Mrs.,Ryan,F,Rocha,"Rua MC 3 1940",Goiânia,GO,Goiana,74320-450,RR,Brazil,Sydney
8 7,Female,Brazil,Mrs.,Tânia,Z,Carvalho,"Avenida Quilom de Pontões 1655",Fátima,SE,"Rio de Jan
9 8,Female,Brazil,Mrs.,Capitana,E,Rodrigues,"Rua Francisco de Paula 107",Hercílio,Zé,Alagoinha,57025-120,RR,I
10 9,Male,Brazil,Mrs.,Marcos,Z,Bonafina,"Rua São 482",Bomazama,PR,Sapopó,87403-130,PR,B
11 10,Female,Brazil,Mrs.,Rafaela,C,Rodrigues,"Rua Adalberto 1713","Rio de Janeiro",RJ,"Rio de Jan
12 11,Male,Brazil,Mrs.,Vilfredo,F,Bastos,"Rua Luis Jacarati 1237","São Paulo",SP,"São Paulo",00
13 12,Female,Brazil,Mrs.,Emily,F,Capozzo,"Quinta Dextra 104 Conjunto 62 312","Bomazama,PR,Sapopó,87403-130,PR,B
14 13,Female,Brazil,Mrs.,Rafaela,E,Melo,"Rua Adalberto 1713",Fátima,SE,"Rio de Janeiro",RJ,"Rio de Janeiro",00
```


Manipulação de arquivos CSV

- A manipulação de arquivos CSV é uma tarefa rotineira para os programadores de Python, para processamento de dados, análise de dados, a geração de reports entre outros.
- É importante saber manipular arquivos CSV (ler, escrever e manipular) para permitir a automação de várias tarefas, e processar grandes quantidades de dados de forma eficiente.

Manipulação de arquivos CSV

- Um arquivo CSV é um arquivo que armazena dados em formato de tabela, usando vírgulas para separar cada valor.
- Eles são usados para transferir dados entre diferentes programas, como planilhas eletrônicas, bancos de dados ou sistemas de gerenciamento de dados. Arquivos CSV podem ser abertos e editados por vários aplicativos, como Python, Microsoft Excel, Google Sheets, OpenOffice Calc ou editores de texto simples.

Manipulação de arquivos CSV

- Para abrir um arquivo CSV em um programa de planilha, você pode clicar duas vezes no arquivo e escolher o programa desejado. Ou você pode abrir o programa e selecionar a opção de abrir ou importar um arquivo CSV.
- Você pode precisar ajustar algumas configurações, como o delimitador usado, o formato dos dados ou o idioma dos caracteres.

Manipulação de arquivos CSV

- Para abrir um arquivo CSV em um editor de texto, você pode clicar com o botão direito do mouse no arquivo e escolher a opção de abrir com o editor de sua preferência.
- Você pode abrir o editor e selecionar a opção de abrir um arquivo de texto. Você verá os dados em formato de texto, separados por vírgulas e quebras de linha.

Manipulação de arquivos CSV – Leitura

- No Python, utilizaremos o módulo **csv** embutido na biblioteca padrão. O módulo **csv** nos fornece várias funcionalidades para trabalhar com arquivos de formato **CSV**, de forma fácil e eficiente.
- Para ler um arquivo CSV, primeiro precisamos abrir o arquivo usando a função **open()**.
- Em seguida, o objeto de arquivo retornado para criar um leitor de CSV usando a função **csv.reader**.

Manipulação de arquivos CSV – Leitura

- O exemplo a seguir demonstra como ler um arquivo CSV chamado **dados.csv**:

```
3582     Exercício 109 - Leitura de um arquivo CSV
3583
3584     '''
3585
3586     import csv
3587
3588     with open('dados.csv', 'r') as arquivo:
3589         leitor_csv = csv.reader(arquivo)
3590         for linha in leitor_csv:
3591             print(linha)
3592
```

Manipulação de arquivos CSV – Leitura

- Neste exemplo, usamos um **with statement** para garantir que o arquivo seja fechado após a leitura.
- Na sequência é criado Em seguida, criamos um leitor de CSV usando o objeto de arquivo retornado pela função open.
- Por fim, iteramos sobre as linhas do arquivo usando um loop for e imprimimos cada linha.

Manipulando dados CSV

- A manipulação de dados CSV envolve várias tarefas como, por exemplo, filtrar linhas, seleção de colunas e calcular estatísticas.
- O exemplo abaixo demonstra como filtrar linhas:

Exercicio 110 - Filtrando Linhas em um arquivo CSV

'''

import csv

with open('dados.csv', 'r') as arquivo:

leitor_csv = csv.reader(arquivo)

next(leitor_csv) # pula a primeira linha (cabeçalho)

for linha in leitor_csv:

if linha[2] == 'Brasil':

print(linha)

Manipulando dados CSV

- No exemplo anterior, usamos a função `next()` para pular a primeira linha do arquivo (geralmente é o cabeçalho).
- Em seguida, verificamos se o valor da terceira coluna (índice 2) é igual a “Brasil” e, se for, imprimimos a linha.
- Dessa forma, filtramos apenas as linhas que atendem a essa condição.
- Além disso, podemos selecionar colunas específicas em um arquivo CSV usando a indexação adequada.

• • • • • +

• • • • • •

• + Manipulando dados CSV

+ •

- Neste exemplo, imprimimos apenas as colunas 1 e 3 de cada linha do arquivo.
- As colunas são indexadas a partir de zero, então a primeira coluna é representada pelo índice 0, a segunda coluna pelo índice 1 e assim sucessivamente

|
+

•
• +
■ □ •
• •
•
• + • •
• • •

Escrevendo em arquivos CSV

- Também podemos escrever dados em um arquivo CSV no Python. Para escrever em um arquivo CSV, usamos a função `csv.writer()` juntamente com o objeto de arquivo retornado pela função `open()`. Veja o exemplo:

```
3622     """
3623     Exercício 112 - Escrevendo um arquivo CSV
3624     """
3625
3626
3627
3628     import csv
3629
3630     dados = [
3631         ['Nome', 'Idade', 'País'],
3632         ['João', '25', 'Brasil'],
3633         ['Maria', '30', 'Portugal'],
3634         ['José', '35', 'Espanha']
3635     ]
3636
3637     with open('dados.csv', 'w', newline='') as arquivo:
3638         escritor_csv = csv.writer(arquivo)
3639         for linha in dados:
3640             escritor_csv.writerow(linha)
3641
3642
```

Como automatizar planilhas do Excel utilizando Python



Enterprise

Pricing

Resources

About

Contact Sales

Introducing Python in Excel

Harness the power of Python in Microsoft Excel's familiar interface.



Explore Python in Excel



Como automatizar planilhas do Excel utilizando Python

- O Excel é uma ferramenta poderosa para manipular e fazer analisar dados. Entretanto, realizar tarefas repetitivas e demoradas no Excel pode consumir muito tempo.
- Uma solução para isso é automatizar essas tarefas utilizando a linguagem de programação Python.
- Para este intento utilizaremos a biblioteca openpyxl para automatizar tarefas no Excel (Criar, Ler, Atualizar e Formatar Planilhas).

Como automatizar planilhas do Excel utilizando Python

Automatizar tarefas no Excel com Python pode trazer diversos benefícios. Alguns deles incluem:

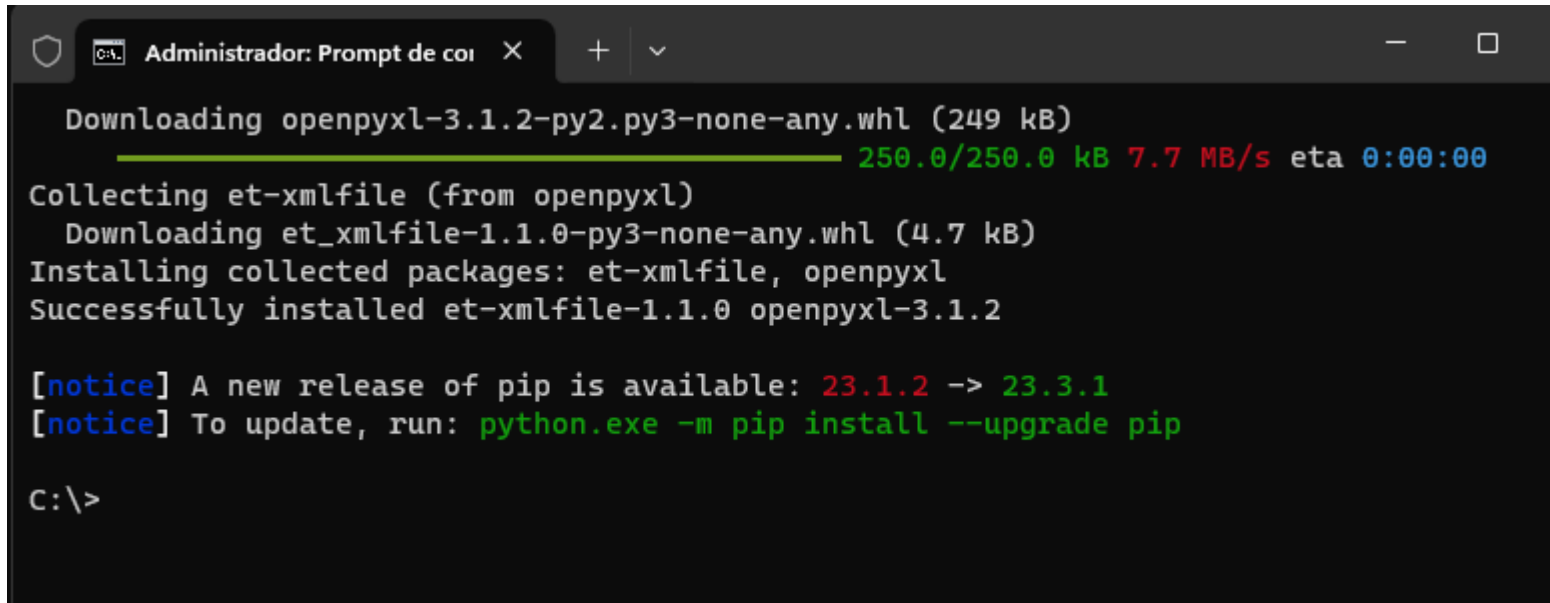
- **Economia de tempo:** Ao automatizar tarefas, você pode reduzir significativamente o tempo necessário para executá-las manualmente. Isso é particularmente útil quando você precisa lidar com grandes volumes de dados ou realizar manipulações complexas.
- **Precisão:** Ao utilizar o Python para manipular dados no Excel, você reduz o risco de erros humanos, como digitar valores incorretos ou aplicar fórmulas erradas.
- **Flexibilidade:** O Excel possui limitações em relação a certas tarefas de processamento e análise de dados. Com o Python, você pode implementar lógica personalizada e realizar manipulações mais complexas nos dados.

Como automatizar planilhas do Excel utilizando Python

- Instalando a biblioteca openpyxl:

No terminal vamos utilizar o comando:

pip install openpyxl:



```
Administrador: Prompt de comando
Downloading openpyxl-3.1.2-py2.py3-none-any.whl (249 kB)
250.0/250.0 kB 7.7 MB/s eta 0:00:00
Collecting et-xmlfile (from openpyxl)
  Downloading et_xmlfile-1.1.0-py3-none-any.whl (4.7 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-1.1.0 openpyxl-3.1.2

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\>
```

Como automatizar planilhas do Excel utilizando Python

- Vamos criar uma planilha em branco – Exemplo.xlsx:

```
3648  Exercício 113 - Criando uma planilha nova
3649
3650  '''
3651
3652  # inicialmente NO TERMINAL use: pip install openpyxl
3653
3654  from openpyxl import Workbook
3655
3656  # Cria uma nova planilha em branco
3657  wb = Workbook()
3658
3659  # Seleciona a planilha ativa
3660  planilha = wb.active
3661
3662  # Define o valor da célula A1
3663  planilha['A1'] = 'Olá, Excel com Python!'
3664
3665  # Salva o arquivo
3666  wb.save('Exemplo.xlsx')
```


Como automatizar planilhas do Excel utilizando Python

- Importamos então a classe Workbook da biblioteca openpyxl, criamos um objeto **wb** do tipo Workbook, que representa a planilha em branco.
- Depois selecionamos a planilha ativa usando o atributo active e definimos o valor da célula 'A1' da planilha usando a notação de índice 'A1', e atribuímos **Excel com Python!** para a célula A1.
- Finalmente, o arquivo é salvo com nome **"Exemplo.xlsx"** usando o método **save** do objeto wb.

Como automatizar planilhas do Excel utilizando Python

- Importamos então a classe Workbook da biblioteca openpyxl, criamos um objeto **wb** do tipo Workbook, que representa a planilha em branco.
- Depois selecionamos a planilha ativa usando o atributo active e definimos o valor da célula 'A1' da planilha usando a notação de índice 'A1', e atribuímos **Excel com Python!** para a célula A1.
- Finalmente, o arquivo é salvo com nome **"Exemplo.xlsx"** usando o método **save** do objeto wb.

Lendo dados de uma planilha existente

Além de criar uma nova planilha, também pode-se usar o Python para ler dados de uma planilha existente.

Para fazer isso, utilizamos a seguinte sequência de comandos:

```
3669 Exercicio 114 - Lendo dados de uma planilha existente
3670
3671 '''
3672
3673 from openpyxl import load_workbook
3674
3675 # Carrega o arquivo existente
3676 wb = load_workbook('Exemplo.xlsx')
3677
3678 # Seleciona a planilha ativa
3679 planilha = wb.active
3680
3681 # Lê o valor da célula A1
3682 valor_celula = planilha['A1'].value
3683
3684 # Imprime o valor na tela
3685 print(valor_celula)
```

Lendo dados de uma planilha existente

- Neste exemplo, importamos a função `load_workbook` da biblioteca `openpyxl`.
- Em seguida, carregamos o arquivo “Exemplo.xlsx” usando a função `load_workbook` e atribuímos o objeto resultante à variável `wb`.
- Depois disso, selecionamos a planilha ativa utilizando o atributo `active()`.
- Em seguida, lemos o valor da célula A1, utilizando a notação de índice `'A1'`, e atribuímos o valor à variável `valor_celula`.
- Por fim, imprimimos o valor da célula A1 na tela utilizando a função `print()`.

Atualizando dados em uma planilha existente

- Além de ler dados de uma planilha existente, também podemos atualizar os dados usando o Python.
- Para isso, utilizamos a seguinte sequência de comandos:

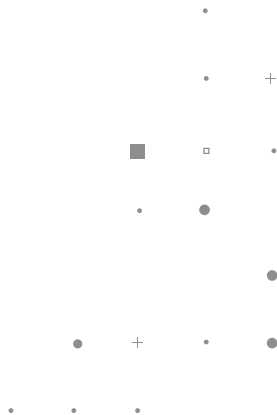
```
3687  
3688 '''  
3689 Exercicio 115 - Atualizando dados em uma planilha existente  
3690 '''  
3691  
3692  
3693 from openpyxl import load_workbook  
3694  
3695 # Carrega o arquivo existente  
3696 wb = load_workbook('Exemplo.xlsx')  
3697  
3698 # Seleciona a planilha ativa  
3699 planilha = wb.active  
3700  
3701 # Atualiza o valor da célula A1  
3702 planilha['A1'] = 'Novo valor'  
3703  
3704 # Salva o arquivo  
3705 wb.save('Exemplo.xlsx')  
3706  
3707
```

Atualizando dados em uma planilha existente

- Neste exemplo, importamos a função `load_workbook` da biblioteca `openpyxl`.
- Em seguida, carregamos o arquivo “Exemplo.xlsx” usando a função `load_workbook` e atribuímos o objeto resultante à variável `wb`.
- Depois disso, selecionamos a planilha ativa utilizando o atributo `active`. Em seguida, atualizamos o valor da célula A1, utilizando a notação de índice `'A1'` e atribuímos o novo valor. Então salvamos o arquivo usando o método `save` do objeto `wb`.



VAMOS CODAR!!!



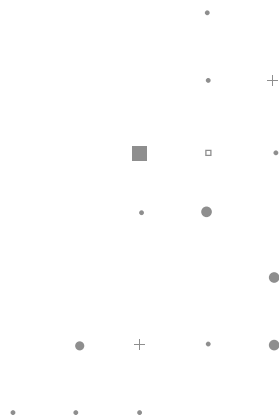


OBRIGADO

FIAP

Copyright © 2022 | Professor Allen Fernando

Todos os direitos reservados. Reprodução ou divulgação total ou parcial deste documento, é expressamente proibido sem consentimento formal, por escrito, do professor/autor.





FIAP

